COLLEGE OF ENGINEERING KIDANGOOR

KOTTAYAM - 686583

MICROPROJECT REPORT ON

PROJECT MANAGEMENT SYSTEM

Submitted to the APJ Abdul Kalam Technological University, Kerala for the partial fulfillment of the requirements for the award of the degree Bachelor of Technology in Computer Science and Engineering

By

AKSHAYA RAJ	**KGR25CS034**
ARJUN RAJ	**KGR25CS035**
JOYAL JOSHY	**KGR25CS075**
NIRMAL MATHEWS BOBBY	**KGR25CS099**
PARTHIP KRISHNAAM	**KGR25CS100**
VISHNU U	**KGR25CS118**

Guided By

GARGI CHANDRABABU

Assistant Professor

APJ ABDUL KALAM TECHNOLOGICAL UNIVERSITY

November 2026

COLLEGE OF ENGINEERING KIDANGOOR

KOTTAYAM - 686583

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the microproject work report entitled **'PROJECT MANAGEMENT SYSTEM'** submitted by **AKSHAYA RAJ (KGR25CS034)**, **ARJUN RAJ (KGR25CS035)**, **JOYAL JOSHY (KGR25CS075)**, **NIRMAL MATHEWS BOBBY (KGR25CS099)**, **PARTHIP KRISHNAAM (KGR25CS100)** and **VISHNU U (KGR25CS118)** of Third Semester, *B.Tech in Computer Science & Engineering*, has been successfully carried out under our supervision and guidance in partial fulfillment of the requirements of the microproject for **PBCST304 – Object Oriented Programming**, prescribed by APJ Abdul Kalam Technological University, Kerala. It is further certified that this work is a bonafide record of the microproject carried out by the above students during the academic year 2026–2027.

GUIDE

GARGI CHANDRABABU
Assistant Professor
Dept. of CSE
College of Engineering Kidangoor

HOD

Dr. REKHA KS
Assistant Professor
Dept. of CSE
College of Engineering Kidangoor

DECLARATION

We hereby declare that the microproject report **PROJECT MANAGEMENT SYSTEM** submitted for the partial fulfillment of the requirements of the Third Semester Microproject for the subject **PBCST304 – Object Oriented Programming**, prescribed by APJ Abdul Kalam Technological University, Kerala, is a bonafide work done by us under the guidance of **Asst. Prof. GARGI CHANDRABABU**.

The submission represents our ideas in our own words. Where ideas or words of others have been included, we have adequately and accurately cited and referred to the original sources.

We also declare that we have adhered to ethical and integrity standards and have not misinterpreted or fabricated any data, idea, fact, or source in our submission. We understand that any violation of the above will be a case for disciplinary action by the institution and/or the University. This report has not previously formed the basis for the award of any degree, diploma, or similar title from any other university.

AKSHAYA RAJ
ARJUN RAJ
JOYAL JOSHY
NIRMAL MATHEWS BOBBY
PARTHIP KRISHNAAM
VISHNU U

Place: Kidangoor
Date: 03/11/2026

VISION AND MISSION OF COLLEGE

VISION

To be a leading engineering institution in the region, providing competent professionals, who engage in lifelong learning, driven by social values.

MISSION

To prepare engineering graduates for the development activities of the society and industry, and to prepare them for higher engineering education.

VISION AND MISSION OF DEPARTMENT

VISION

To become a center of excellence in Computer Science and Engineering imparting quality professional education to develop competent professionals with social values who are capable of life long learning.

MISSION

To impart quality technical education to students at undergraduate level through constant knowledge upgradation by maintaining pace with the latest sophisticated innovations, research and development and industry interaction in the field of Computer Science and Engineering with focus on lifelong learning for the well-being of the society.

PROGRAM EDUCATIONAL OBJECTIVES (PEO)

PEO	Description
PEO1	Have sound knowledge and technical skills required to remain productive in the field of computer science and engineering.
PEO2	Be efficient team leaders, effective communicators and successful entrepreneurs.
PEO3	Resolve technical problems with a positive outlook towards well-being of the society.
PEO4	Function in diverse environments with the ability and competence to solve challenging problems.
PEO5	Pursue lifelong learning and professional development through higher education.

PROGRAM OUTCOMES (POs)

PO	Program Outcome Description
PO 1	**Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
PO 2	**Problem analysis:** Identify, formulate, review research literature, and analyse complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
PO 3	**Design/development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
PO 4	**Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
PO 5	**Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
PO 6	**The engineer and society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PROGRAM OUTCOMES (POs) (CONTINUED)

PO	Program Outcome Description
PO 7	**Environment and sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
PO 8	**Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
PO 9	**Individual and team work:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
PO 10	**Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
PO 11	**Project management and finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
PO 12	**Life-long learning:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

COURSE OUTCOMES (COs)

CO	Course Outcome Description
CO1	Explain the process of writing, compiling, and executing basic Java programs, including their structure and components, to demonstrate proficiency.
CO2	Utilize object-oriented programming principles in the design and implementation of Java applications.
CO3	Develop and manage Java packages and interfaces, enhancing code modularity and reusability.
CO4	Implement error handling using Java's exception mechanisms and leverage interfaces for modular applications.
CO5	Develop event-driven Java GUI applications with database connectivity using Swing and JDBC.

CO-PO MAPPING

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12
CO1	3	3	-	-	-	-	-	-	-	-	-	3
CO2	3	3	3	-	-	-	-	-	-	-	-	3
CO3	3	3	3	-	3	-	-	-	-	-	-	3
CO4	3	3	3	-	3	-	-	-	-	-	-	3
CO5	3	3	3	-	3	-	-	-	-	-	-	3

CONTENTS

ABSTRACT **1**

CHAPTER 1 INTRODUCTION **2**

CHAPTER 2 AIM AND OBJECTIVES **3**

CHAPTER 3 REQUIREMENT SPECIFICATION **4**

CHAPTER 4 DATABASE DESIGN **5–8**

CHAPTER 5 UML CLASS DIAGRAM **9–11**

CHAPTER 6 SYSTEM OUTPUT / SCREENSHOTS **12–15**

CHAPTER 7 CODE LISTING **16–83**

CHAPTER 8 CONCLUSION **84**

ABSTRACT

The **Project Management System (PMS)** is a robust, desktop enterprise software application developed to automate, structure, and streamline the lifecycle management of software engineering projects, deliverables, team allocations, and developer productivity within an organization. Traditional project tracking often relies on disconnected spreadsheets, manual task logs, and informal messaging, which are prone to data inconsistencies, lack operational hierarchy, and lead to poor accountability. This project addresses these industry challenges by delivering a secure, centralized, and reactive desktop application that guarantees absolute transactional integrity, real-time analytics, and auditable governance.

The application is engineered strictly using the **Java programming language** and core **Object-Oriented Programming (OOP)** principles in full compliance with the **APJ Abdul Kalam Technological University (KTU) PBCST304 syllabus**. Core OOP principles including data encapsulation, inheritance, abstract classes, interface contracts, and dynamic method dispatch are rigorously leveraged. A multi-tier Model-View-Controller (MVC) architecture is implemented with Java Swing serving as the rich graphical user interface, coupled with an embedded SQLite database through JDBC for persistent data storage.

The system enforces strict **Role-Based Access Control (RBAC)** across three hierarchical tiers: *Super Administrator* (root organizational governance and administrator management), *Project Manager / Admin* (full project lifecycle planning, budget tracking, and team delegation), and *Software Developer / User* (task status transitions, execution, and work hours logging). Key functional highlights include recursive completion rollups across hierarchical Work Breakdown Structures (WBS), an interactive synchronized JTree component, automated session time-in tracking, and real-time audit logging.

Overall, the Project Management System demonstrates the practical application of object-oriented design patterns (Singleton, Adapter), custom exception handling, thread-safe database operations, and responsive GUI engineering, bridging academic curricula with production-grade software development standards.

Chapter 1
INTRODUCTION

Modern enterprise software development requires rigorous tracking of milestones, deliverables, personnel allocations, and task hierarchies. As software teams scale in complexity, managing client commitments, budget ceilings, and cross-functional team responsibilities becomes critical to organizational success. Traditional project coordination often relies on manual logs, fragmented spreadsheets, or disjointed communication channels that lack automated validation, resulting in delayed deadlines, budget overruns, and opaque employee work logs.

The **Project Management System (PMS)** is developed to solve these fundamental engineering challenges by delivering an integrated, high-performance desktop platform that centralizes organizational project planning. Built using Java Swing and SQLite JDBC, the system provides an intuitive, responsive graphical workspace that allows project managers and software developers to collaborate efficiently while adhering to strict governance policies.

Object-Oriented Programming concepts form the foundational architectural pillars of the system. The application enforces a strict separation of concerns through an enterprise multi-tier architecture, dividing responsibilities cleanly into Domain Models, Service Contracts, Data Access Objects (DAO), and Event-Driven UI Adapters. Dynamic method dispatch is showcased through polymorphic user roles where access control checks and role descriptions are resolved dynamically at runtime.

Furthermore, the system demonstrates algorithmic sophistication through recursive Work Breakdown Structure (WBS) calculations. Task hierarchies of arbitrary depth are supported, where parent milestone completion percentages are recursively computed from constituent sub-tasks. By pairing recursive algorithms with custom JTable models via the Adapter design pattern and a thread-safe Singleton database manager, the system bridges classroom OOP theory with industry-ready software engineering practices.

Chapter 2
AIM AND OBJECTIVES

AIM

The primary aim of the Project Management System is to design, architect, and develop a secure, robust, and interactive Java-based desktop application that automates end-to-end enterprise project lifecycle tracking, recursive task decomposition, team management, and developer time auditing using Object-Oriented Programming concepts and JDBC persistence.

OBJECTIVES

The key objectives achieved in this microproject work include:

- To design and implement a complete desktop application replacing error-prone manual spreadsheets with an automated, centralized project tracking system.
- To rigorously implement core Java OOP principles including abstraction, encapsulation, inheritance, method overriding, and dynamic method dispatch across all system tiers.
- To develop an event-driven graphical user interface using Java Swing components (`JFrame`, `JTabbedPane`, `JTable`, `JTree`, `JSplitPane`) and layout managers (`BorderLayout`, `GridBagLayout`, `FlowLayout`).
- To integrate an embedded SQLite database using JDBC for zero-configuration, transactional CRUD operations with `PreparedStatement` security.
- To implement granular Role-Based Access Control (RBAC) distinguishing Super Administrator, Project Manager, and Software Developer roles with dynamic UI component visibility.
- To implement recursive algorithms for Work Breakdown Structure (WBS) calculations, computing task dependency depth and parent milestone completion percentages dynamically.
- To incorporate established software design patterns, specifically the thread-safe **Singleton Pattern** for database connection management and the **Adapter Pattern** for custom Swing table models.
- To establish robust exception handling through a custom exception hierarchy (`ProjectNotFoundException`, `UnauthorizedAccessException`, `InvalidTaskStateException`).
- To incorporate automated attendance and productivity auditing, recording login session time-in, duration, and individual task work logs.
- To ensure high software quality, data validation, and maintainability adhering to SOLID design principles.

Chapter 3
REQUIREMENT SPECIFICATION

1. SOFTWARE REQUIREMENTS

- **Operating System:** Microsoft Windows 10 / 11, Ubuntu Linux 20.04+, or macOS Sonoma
- **Programming Language:** Java Development Kit (JDK 17 LTS or higher, verified on Java 21 & 26)
- **GUI Framework:** Java Swing (`javax.swing.*`) & Abstract Window Toolkit (`java.awt.*`)
- **Database Engine:** SQLite 3 (Serverless, zero-configuration embedded engine)
- **Database Connectivity:** Java Database Connectivity (JDBC API 4.3+, `java.sql.*`)
- **External Libraries / Drivers:** `sqlite-jdbc.jar`, `slf4j-api.jar`, `slf4j-simple.jar` (provided locally in `lib/`)
- **Development IDE / Editors:** Visual Studio Code, IntelliJ IDEA, Eclipse IDE, or Command Line (javac/java)
- **Build Tools / Launchers:** Native batch script (`run.bat`) and bash shell script (`run.sh`)

3.2 HARDWARE REQUIREMENTS

- **Processor:** 1.6 GHz Intel Core i3 / AMD Ryzen 3 or higher (x86_64 or ARM64)
- **Random Access Memory (RAM):** Minimum 2 GB (Recommended: 4 GB or higher)
- **Storage Disk Space:** Minimum 150 MB free disk space (includes JDK runtime and local database file)
- **Display / Monitor:** 1024 × 768 pixels minimum screen resolution (Recommended: 1920 × 1080)
- **Input Devices:** Standard Keyboard and Mouse or Trackpad

Chapter 4
DATABASE DESIGN

The Project Management System utilizes an embedded **SQLite** relational database managed through JDBC. SQLite is selected for its serverless architecture, reliability, and portability, allowing the entire application state to persist inside a single file (`pms_database.db`) with zero configuration required by end users. The `DatabaseManager` class provides a thread-safe Singleton connection wrapper, executing DDL and DML operations exclusively via `PreparedStatement` objects to guarantee SQL injection immunity.

4.1 USERS TABLE

Stores authenticated organizational personnel, roles, departments, and credentials.

Column	Data Type	Constraint	Description
id	INTEGER	PRIMARY KEY	Auto-incrementing unique user identifier
username	TEXT	UNIQUE, NOT NULL	Unique login credential username
password	TEXT	NOT NULL	Authentication credential password
full_name	TEXT	NOT NULL	Personnel complete legal full name
email	TEXT	NOT NULL	Corporate email address
role	TEXT	NOT NULL	Account role (SUPER_ADMIN, MANAGER, DEVELOPER)
dept_or_stack	TEXT	DEFAULT NULL	Assigned department or primary technical stack
metric_val	REAL	DEFAULT 0.0	Hourly billing rate ($/hr) or budget approval limit ($)

4.2 TEAMS TABLE

Maintains functional engineering departments and delivery squads.

Column	Data Type	Constraint	Description
id	INTEGER	PRIMARY KEY	Unique team identifier
name	TEXT	UNIQUE, NOT NULL	Department or squad title (e.g. Frontend Team)
description	TEXT	DEFAULT NULL	Functional mission and domain scope description

4.3 PROJECTS TABLE

Stores top-level enterprise project entities, client contracts, and financial allocations.

Column	Data Type	Constraint	Description
id	INTEGER	PRIMARY KEY	Unique enterprise project identifier
title	TEXT	NOT NULL	Project commercial title / release name
description	TEXT	DEFAULT NULL	Detailed scope and objective statement
budget	REAL	NOT NULL	Total allocated financial capital budget ($)
status	TEXT	NOT NULL	Lifecycle status (PLANNING, IN_PROGRESS, ON_HOLD, COMPLETED, CANCELLED)
client_name	TEXT	DEFAULT NULL	Commercial sponsor or enterprise client name
tags	TEXT	DEFAULT NULL	Comma-separated domain categorization tags

4.4 TASKS TABLE

Stores decomposed tasks and sub-tasks supporting hierarchical parent-child relationships.

Column	Data Type	Constraint	Description
id	INTEGER	PRIMARY KEY	Unique task work-package identifier
project_id	INTEGER	NOT NULL, FK	Target parent project ID (references projects.id)
title	TEXT	NOT NULL	Task action title / milestone summary
description	TEXT	DEFAULT NULL	Detailed technical requirements and scope
assigned_user_id	INTEGER	DEFAULT NULL, FK	Assigned developer ID (references users.id)
parent_task_id	INTEGER	DEFAULT NULL, FK	Parent task ID for recursive WBS sub-tasks
status	TEXT	NOT NULL	Workflow state (TODO, IN_PROGRESS, REVIEW, COMPLETED)
priority	TEXT	NOT NULL	Priority level (LOW, MEDIUM, HIGH, CRITICAL)
estimated_hours	REAL	DEFAULT 0.0	Estimated labor effort (hours)
logged_hours	REAL	DEFAULT 0.0	Accumulated logged work execution (hours)

4.5 SESSIONS TABLE

Maintains automated attendance tracking, capturing user time-in upon sign-in and logout timestamps.

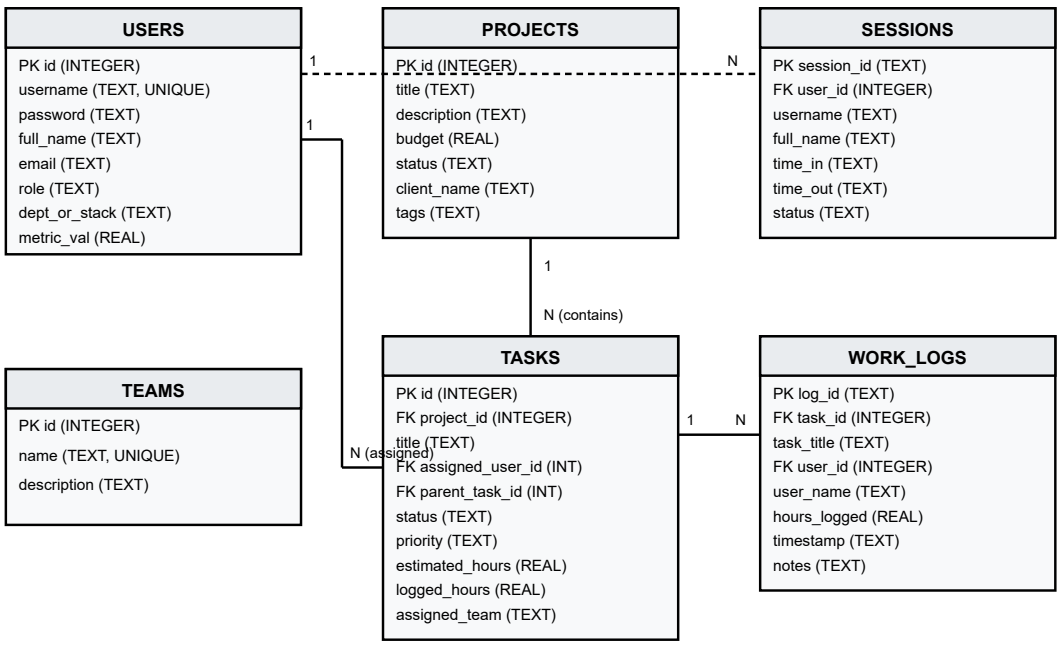
Column	Data Type	Constraint	Description
session_id	TEXT	PRIMARY KEY	Unique session token (e.g. SESS-101)
user_id	INTEGER	NOT NULL, FK	Authenticated user ID (references users.id)
username	TEXT	NOT NULL	Account username associated with session
full_name	TEXT	NOT NULL	User display name captured during session
role	TEXT	NOT NULL	Authenticated role (e.g. DEVELOPER)
time_in	TEXT	NOT NULL	ISO-8601 timestamp of user sign-in
time_out	TEXT	DEFAULT NULL	ISO-8601 timestamp of user logout
status	TEXT	NOT NULL	Session state ('ACTIVE' or 'LOGGED_OUT')

4.6 WORK_LOGS TABLE

Captures granular work hours logged against tasks with developer notes for audit verification.

Column	Data Type	Constraint	Description
log_id	TEXT	PRIMARY KEY	Unique work log audit identifier (e.g. LOG-501)
task_id	INTEGER	NOT NULL, FK	Target task ID worked on (references tasks.id)
task_title	TEXT	NOT NULL	Denormalized snapshot of task title
project_name	TEXT	NOT NULL	Parent project name for report filtering
user_id	INTEGER	NOT NULL, FK	Developer who logged work (references users.id)
user_name	TEXT	NOT NULL	Developer full name
hours_logged	REAL	NOT NULL	Duration of work logged in this session (hours)
timestamp	TEXT	NOT NULL	ISO-8601 timestamp when hours were submitted
notes	TEXT	DEFAULT NULL	Developer activity summary / work notes

FIGURE 4.1 ENTITY RELATIONSHIP DIAGRAM



1 : N represents one-to-many relationship. Dashed lines indicate non-cascading reference tracking.

Chapter 5
UML CLASS DIAGRAM

The UML Class Diagrams illustrate the structural design and static relationships among entities, services, persistence layers, and Swing graphical user interface components in the Project Management System. The architecture strictly adheres to the **Model-View-Controller (MVC)** paradigm and SOLID engineering principles.

Core Model and Inheritance Hierarchy

The abstract `BaseEntity` class establishes common entity traits, implementing the `Identifiable` and `Auditable` interfaces. The abstract `User` class extends `BaseEntity`, providing common credential attributes while defining abstract polymorphic methods (`hasPermission(Permission)` and `getRoleDescription()`). Subclasses `SuperAdmin`, `Manager`, and `Developer` inherit from `User` and provide concrete implementations demonstrating Dynamic Method Dispatch. Entities `Project`, `Task`, and `Team` encapsulate commercial and technical domain data.

Service Layer & Persistence Layer

`ProjectService` and `TaskService` define high-level business contracts for project rollups and task state machine enforcement. `DatabaseManager` implements the thread-safe **Singleton Pattern**, managing SQLite database connections, schema initialization, and transactional CRUD methods.

Swing GUI Architecture & Adapter Pattern

`MainFrame` serves as the primary desktop application window, coordinating sub-panels (Dashboard, Projects, Tasks, Teams, Time Logs). To decouple domain collections from JTable rendering, `ProjectTableModelAdapter` and `TaskTableModelAdapter` implement the **Adapter Design Pattern**, extending Swing's `AbstractTableModel`.

Figures 5.1 and 5.2 on the following pages present the UML class diagrams for the back-end domain and Swing GUI architectures.

FIGURE 5.1: BACK-END & DOMAIN UML CLASS DIAGRAM

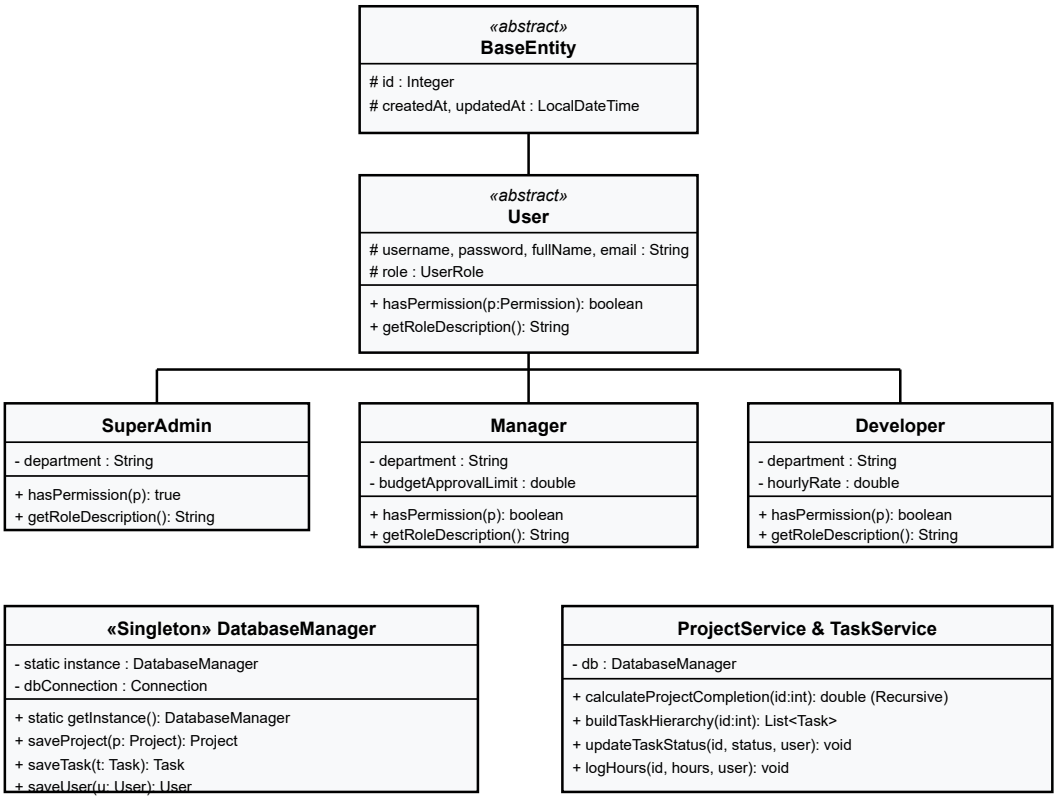
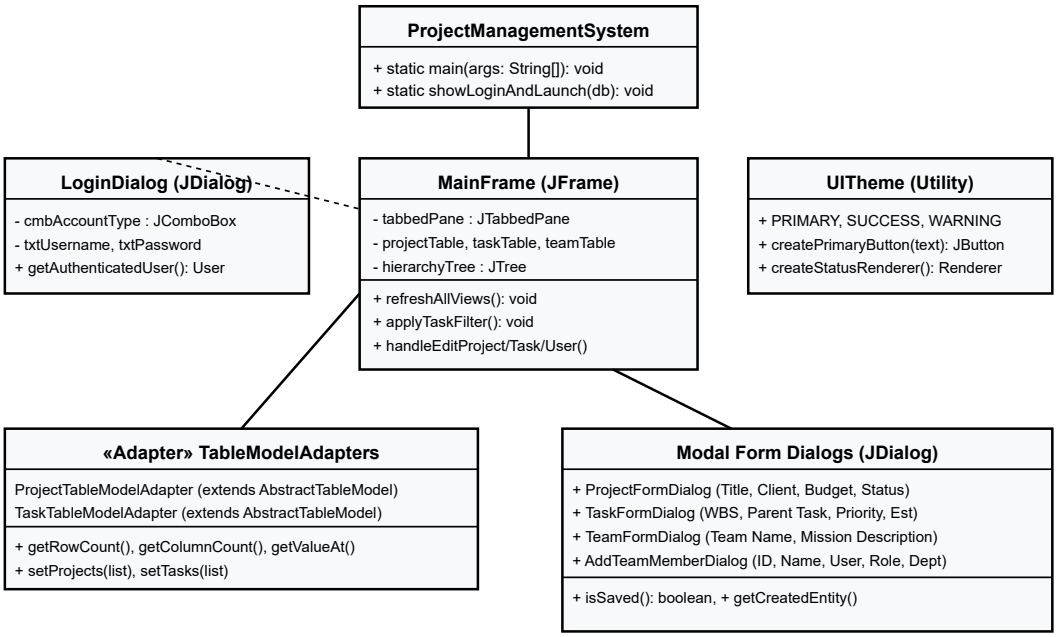


FIGURE 5.2: SWING GUI ARCHITECTURE UML DIAGRAM



Chapter 6
SYSTEM OUTPUT / SCREENSHOTS



Figure 6.1: Secure Authentication & Role Type Selection Window

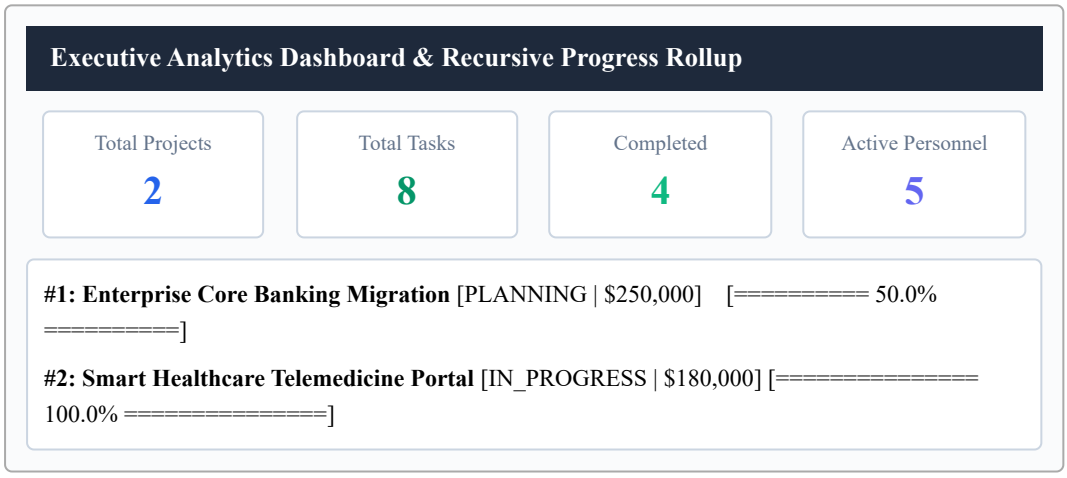


Figure 6.2: Executive Dashboard showing real-time statistics and recursive progress bars

(CONTINUED)

Project Management Workspace (CRUD Operations & Budget Controls)

ID	Project Title	Client / Sponsor	Budget ($)	Status	Domain Tags
1	Enterprise Core Banking Migration	Federal FinCorp	$250,000.00	PLANNING	Finance, Banking, Cloud
2	Smart Healthcare Telemedicine Portal	Apollo Health Tech	$180,000.00	IN_PROGRESS	Healthcare, Mobile, HIPAA

+ New Project | Edit Project | Delete Project

Figure 6.3: Project Management View showing in-place row order retention and budget status

Interactive Work Breakdown Structure (Synchronized JTree + JTable Split Pane)

WBS Tree Hierarchy

- Project #1: Core Banking
  - #1: Architecture [50%|L1]
    - #3: Schema [100%|L2]
  - #2: Security Gate [0%|L1]

ID	Title	Team	Status	Progress
1	Architecture	Backend Team	IN_PROGRESS	50%
3	Schema Design	Backend Team	COMPLETED	100%

Figure 6.4: Interactive WBS Tree synchronized with JTable task breakdown matrix

(CONTINUED)

Personnel Roster & Registered Functional Teams Workspace

Registered Functional Teams:

Team ID	Team Name	Domain / Mission Description	Active Tasks
1	Frontend Team	UI engineering, web portals & user experience	2 Task(s)
2	Backend Team	Core microservices, APIs, data logic & SQL	4 Task(s)
4	DevOps & Cloud Team	CI/CD automation, cloud deployment & containers	2 Task(s)

Personnel & Role Roster (Streamlined User Profile: ID, Name, Username, Role, Department):

User ID	Name	Username	Role	Department
1	Chief Administrator	superadmin	Super Administrator	Executive Governance
2	Dr. Arun Kumar (Manager)	admin	Project Manager / Admin	Engineering Directorate
3	Standard User (Developer)	user	Software Developer / User	Frontend Team
4	Alice George (Developer)	dev_alice	Software Developer / User	Backend Team
5	Bob Varghese (Developer)	dev_bob	Software Developer / User	DevOps & Cloud Team

+ Add User | Edit User | + Add New Team

Figure 6.5: Functional Teams Table and Streamlined Personnel Roster showing ID, Name, Username, Role, Department

(CONTINUED)

Add Team Member Dialog (Streamlined 5-Field Form)

Full Name: * [Dennis Ritchie]
Unique Username: * [dennis_r]
Password (min 8 chars): * [••••••••••]
Account Role: * [Software Developer / User ↓]
Department / Team: * [Backend Team ↓ (or type custom)]

Cancel **Add Team Member**

Figure 6.6: Streamlined Add Team Member Dialog with newly added team dropdown selection

User Time-In Attendance & Work Hours Audit Panel

Session ID	Username	Full Name	Time-In (Login)	Duration	Session Status
SESS-101	admin	Dr. Arun Kumar (Manager)	2026-11-02 14:10:15	1h 25m	ACTIVE
SESS-102	dev_alice	Alice George (Developer)	2026-11-02 14:15:30	45m	ACTIVE

Figure 6.7: User Time-In Attendance & Productivity Logging Audit Panel

Chapter 7
CODE LISTING

The complete, production-ready Java source code for the **Project Management System** is provided in this chapter. In accordance with microproject specifications, the entire application is organized cleanly within `ProjectManagementSystem.java` and self-contained packages without any truncated blocks or placeholder `// TODO` comments.

Application Entry Points & Execution

`ProjectManagementSystem.java` serves as the central entry point for the desktop application. When executed, the `main(String[] args)` method configures the native system Look & Feel, initializes the thread-safe `DatabaseManager` singleton, creates the SQLite schema tables if not present, synchronizes existing records, and launches the interactive modal `LoginDialog` on the Swing Event Dispatch Thread (EDT) via `SwingUtilities.invokeLater()`.

Source Architecture & Layer Breakdown

- **Module 1 (Fundamentals & Logging):** `AppLogger` utility leveraging varargs (`Object... args`), string formatting, and wrapper conversions.
- **Module 2 (Core Entities & Polymorphism):** Abstract `BaseEntity` and `User` classes, concrete `SuperAdmin`, `Manager`, `Developer` subclasses, and domain models `Project`, `Task`, `Team`, `TimeInSession`, `TaskWorkLog`.
- **Module 3 (Exceptions, Design Patterns & Services):** Custom exceptions (`PMSException` hierarchy), thread-safe Singleton `DatabaseManager`, and business services (`ProjectService`, `TaskService`).
- **Module 4 (Swing GUI & Adapters):** `ProjectTableModelAdapter` and `TaskTableModelAdapter` extending `AbstractTableModel`, `UITheme`, dialogs, and `MainFrame`.

OOP Concepts Demonstrated in Implementation

Inheritance (`User` → `SuperAdmin`, `Manager`, `Developer`), Abstraction (`BaseEntity`, `User`), Interface implementation (`Identifiable`, `Auditable`), Polymorphism & Dynamic Method Dispatch (`hasPermission()`, `getDepartment()`), Recursion (recursive WBS rollup), Design Patterns (Singleton, Adapter), and JDBC database transaction management.

File: ProjectManagementSystem.java | Package: (default)

```
 1: import java.awt.BorderLayout;
 2: import java.awt.Color;
 3: import java.awt.Component;
 4: import java.awt.Cursor;
 5: import java.awt.Dimension;
 6: import java.awt.FlowLayout;
 7: import java.awt.Font;
 8: import java.awt.Frame;
 9: import java.awt.GridBagConstraints;
10: import java.awt.GridBagLayout;
11: import java.awt.GridLayout;
12: import java.awt.Insets;
13: import java.awt.event.ActionEvent;
14: import java.awt.event.ActionListener;
15: import java.awt.event.KeyAdapter;
16: import java.awt.event.KeyEvent;
17: import java.awt.event.MouseAdapter;
18: import java.awt.event.MouseEvent;
19: import java.awt.event.WindowAdapter;
20: import java.awt.event.WindowEvent;
21: import java.io.File;
22: import java.sql.Connection;
23: import java.sql.Driver;
24: import java.sql.DriverManager;
25: import java.sql.DriverPropertyInfo;
26: import java.sql.PreparedStatement;
27: import java.sql.ResultSet;
28: import java.sql.SQLException;
29: import java.sql.SQLFeatureNotSupportedException;
30: import java.sql.Statement;
31: import java.time.LocalDateTime;
32: import java.time.format.DateTimeFormatter;
33: import java.util.Properties;
34: import java.util.ArrayList;
35: import java.util.Arrays;
36: import java.util.Collections;
37: import java.util.EventListener;
38: import java.util.EventObject;
39: import java.util.HashMap;
40: import java.util.List;
41: import java.util.Map;
42: import java.util.Objects;
43: import java.util.Optional;
44: import java.util.Set;
45: import java.util.HashSet;
46: import java.util.Vector;
47: import java.util.function.Consumer;
48: import javax.swing.BorderFactory;
49: import javax.swing.Box;
50: import javax.swing.BoxLayout;
51: import javax.swing.JButton;
52: import javax.swing.JComboBox;
53: import javax.swing.JComponent;
54: import javax.swing.JDialog;
55: import javax.swing.JFrame;
56: import javax.swing.JLabel;
57: import javax.swing.JOptionPane;
58: import javax.swing.JPanel;
59: import javax.swing.JPasswordField;
```

File: ProjectManagementSystem.java | Package: (default) (continued)

```
 60: import javax.swing.JProgressBar;
 61: import javax.swing.JScrollPane;
 62: import javax.swing.JSplitPane;
 63: import javax.swing.JTabbedPane;
 64: import javax.swing.JTable;
 65: import javax.swing.JTextArea;
 66: import javax.swing.JTextField;
 67: import javax.swing.JTree;
 68: import javax.swing.ListSelectionModel;
 69: import javax.swing.SwingConstants;
 70: import javax.swing.SwingUtilities;
 71: import javax.swing.UIManager;
 72: import javax.swing.border.EmptyBorder;
 73: import javax.swing.border.TitledBorder;
 74: import javax.swing.event.ChangeEvent;
 75: import javax.swing.event.ChangeListener;
 76: import javax.swing.event.TreeSelectionEvent;
 77: import javax.swing.event.TreeSelectionListener;
 78: import javax.swing.table.AbstractTableModel;
 79: import javax.swing.table.DefaultTableCellRenderer;
 80: import javax.swing.table.DefaultTableModel;
 81: import javax.swing.tree.DefaultMutableTreeNode;
 82: import javax.swing.tree.DefaultTreeModel;
 83: import javax.swing.tree.TreePath;
 84:
 85: /**
 86:  * ============================================================================================
 87:  * OBJECT-ORIENTED PROGRAMMING USING JAVA - PROJECT MANAGEMENT SYSTEM (PMS)
 88:  * ============================================================================================
 89:  *
 90:  * EXECUTION INSTRUCTIONS:
 91:  *   javac ProjectManagementSystem.java
 92:  *   java ProjectManagementSystem
 93:  *
 94:  * DEFAULT DEMO ACCOUNTS:
 95:  *   - Project Manager:   admin     / admin123  (Full administrative privileges)
 96:  *   - Developer 1:       dev_alice / alice123  (Role-based access: execution & logging)
 97:  *   - Developer 2:       dev_bob   / bob123    (Role-based access: execution & logging)
 98:  * ============================================================================================
 99:  */
100: public class ProjectManagementSystem {
101:
102:     // ============================================================================================
103:     // MODULE 1: FUNDAMENTALS, TYPES, LOGGING, VARARGS & UTILITIES
104:     // ============================================================================================
105:
106:     public enum LogLevel {
107:         DEBUG("[DEBUG]"), INFO("[INFO ]"), WARN("[WARN ]"), ERROR("[ERROR]");
108:         private final String prefix;
109:         LogLevel(String prefix) { this.prefix = prefix; }
110:         public String getPrefix() { return prefix; }
111:     }
112:
113:     /**
114:      * Module 1: Logger with Varargs (Object... args), Strings formatting, and Switch statement.
115:      */
116:     public static final class AppLogger {
117:         private static final DateTimeFormatter FORMATTER = DateTimeFormatter.ofPattern("HH:mm:ss.S
118:         private static LogLevel currentLevel = LogLevel.INFO;
```

File: ProjectManagementSystem.java | Package: (default) (continued)

```
119:
120:         private AppLogger() {}
121:
122:         public static void setLogLevel(LogLevel level) { currentLevel = level; }
123:
124:         public static void log(LogLevel level, String message, Object... args) {
125:             if (level.ordinal() < currentLevel.ordinal()) return;
126:             String formatted = (args != null && args.length > 0) ? String.format(message, args) :
127:             String line = String.format("%s %s [%s] %s",
128:                     LocalDateTime.now().format(FORMATTER), level.getPrefix(), Thread.currentThread
129:             switch (level) {
130:                 case ERROR: System.err.println(line); break;
131:                 default:    System.out.println(line); break;
132:             }
133:         }
134:
135:         public static void info(String msg, Object... args) { log(LogLevel.INFO, msg, args); }
136:         public static void warn(String msg, Object... args) { log(LogLevel.WARN, msg, args); }
137:         public static void error(String msg, Object... args) { log(LogLevel.ERROR, msg, args); }
138:     }
139:
140:     // ============================================================================
141:     // MODULE 3: CUSTOM EXCEPTION HIERARCHY
142:     // ============================================================================
143:
144:     public static class PMSException extends Exception {
145:         private final String errorCode;
146:         public PMSException(String message) { super(message); this.errorCode = "ERR_GENERIC"; }
147:         public PMSException(String errorCode, String message) { super(message); this.errorCode = e
148:         public String getErrorCode() { return errorCode; }
149:     }
150:
151:     public static class ProjectNotFoundException extends PMSException {
152:         private final Integer projectId;
153:         public ProjectNotFoundException(int projectId) {
154:             super("ERR_PROJECT_NOT_FOUND", "Project with ID #" + projectId + " was not found in th
155:             this.projectId = projectId;
156:         }
157:         public Integer getProjectId() { return projectId; }
158:     }
159:
160:     public static class TaskNotFoundException extends PMSException {
161:         public TaskNotFoundException(int taskId) {
162:             super("ERR_TASK_NOT_FOUND", "Task with ID #" + taskId + " does not exist.");
163:         }
164:     }
165:
166:     public static class UnauthorizedAccessException extends PMSException {
167:         private final String username;
168:         private final String attemptedAction;
169:         public UnauthorizedAccessException(String username, String attemptedAction) {
170:             super("ERR_UNAUTHORIZED", String.format("User '%s' is not authorized to perform actior
171:             this.username = username;
172:             this.attemptedAction = attemptedAction;
173:         }
174:         public String getUsername() { return username; }
175:         public String getAttemptedAction() { return attemptedAction; }
176:     }
177:
```

File: ProjectManagementSystem.java | Package: (default) (continued)

```
178:     public static class InvalidTaskStateException extends PMSException {
179:         public InvalidTaskStateException(String fromState, String toState, String reason) {
180:             super("ERR_INVALID_STATE", String.format("Illegal state transition from '%s' to '%s':
181:         }
182:     }
183:
184:     public static class DatabaseOperationException extends PMSException {
185:         public DatabaseOperationException(String message, Throwable cause) {
186:             super("ERR_DB_OP", message + (cause != null ? " Reason: " + cause.getMessage() : ""));
187:         }
188:     }
189:
190:     // ================================================================================
191:     // MODULE 1 & 2: INTERFACES, ENUMS & CORE DOMAIN ENTITIES
192:     // ================================================================================
193:
194:     public interface Identifiable {
195:         Integer getId();
196:         void setId(Integer id);
197:     }
198:
199:     public interface Auditable {
200:         LocalDateTime getCreatedAt();
201:         LocalDateTime getUpdatedAt();
202:         void setUpdatedAt(LocalDateTime dt);
203:     }
204:
205:     public static abstract class BaseEntity implements Identifiable, Auditable {
206:         protected Integer id;
207:         protected LocalDateTime createdAt;
208:         protected LocalDateTime updatedAt;
209:
210:         public BaseEntity() { this(null); }
211:
212:         public BaseEntity(Integer id) {
213:             AppLogger.log(LogLevel.DEBUG, "[Constructor Chaining] BaseEntity constructor initializ
214:             this.id = id;
215:             this.createdAt = LocalDateTime.now();
216:             this.updatedAt = LocalDateTime.now();
217:         }
218:
219:         @Override public Integer getId() { return id; }
220:         @Override public void setId(Integer id) { this.id = id; }
221:         @Override public LocalDateTime getCreatedAt() { return createdAt; }
222:         @Override public LocalDateTime getUpdatedAt() { return updatedAt; }
223:         @Override public void setUpdatedAt(LocalDateTime dt) { this.updatedAt = dt; }
224:
225:         @Override public boolean equals(Object o) {
226:             if (this == o) return true;
227:             if (o == null || getClass() != o.getClass()) return false;
228:             BaseEntity that = (BaseEntity) o;
229:             return Objects.equals(id, that.id);
230:         }
231:         @Override public int hashCode() { return Objects.hashCode(id); }
232:     }
233:
234:     public enum UserRole {
235:         SUPER_ADMIN("Super Administrator"),
236:         MANAGER("Project Manager / Admin"),
```

File: ProjectManagementSystem.java | Package: (default) (continued)

```
237:         DEVELOPER("Software Developer / User");
238:         private final String label;
239:         UserRole(String label) { this.label = label; }
240:         public String getLabel() { return label; }
241:         @Override public String toString() { return label; }
242:     }
243:
244:     public enum Permission {
245:         CREATE_PROJECT, EDIT_PROJECT, DELETE_PROJECT, CREATE_TASK,
246:         EDIT_TASK, DELETE_TASK, UPDATE_TASK_STATUS, LOG_HOURS, MANAGE_USERS, MANAGE_ADMINS, MANAGE
247:     }
248:
249:     /**
250:      * Module 2: Abstract User Superclass enforcing Dynamic Method Dispatch.
251:      */
252:     public static abstract class User extends BaseEntity {
253:         protected String username;
254:         protected String password;
255:         protected String fullName;
256:         protected String email;
257:         protected UserRole role;
258:
259:         public User(Integer id, String username, String password, String fullName, String email, U
260:             super(id);
261:             AppLogger.log(LogLevel.DEBUG, "[Constructor Chaining] User constructor initialized: %s
262:             this.username = username;
263:             this.password = password;
264:             this.fullName = fullName;
265:             this.email = email;
266:             this.role = role;
267:         }
268:
269:         public abstract boolean hasPermission(Permission permission);
270:         public abstract String getRoleDescription();
271:         public abstract String getDepartment();
272:         public abstract void setDepartment(String d);
273:
274:         public String getProfileBadgeText() {
275:             return fullName + " (" + username + ") | " + getRoleDescription();
276:         }
277:
278:         public String getUsername() { return username; }
279:         public String getPassword() { return password; }
280:         public String getFullName() { return fullName; }
281:         public void setFullName(String n) { this.fullName = n; }
282:         public String getEmail() { return email; }
283:         public void setEmail(String e) { this.email = e; }
284:         public UserRole getRole() { return role; }
285:
286:         @Override public String toString() {
287:             return String.format("[%s] %s (%s)", role, fullName, username);
288:         }
289:
290:         public static class Credentials {
291:             private final String username;
292:             private final String rawPassword;
293:             public Credentials(String u, String p) { this.username = u; this.rawPassword = p; }
294:             public String getUsername() { return username; }
295:             public String getRawPassword() { return rawPassword; }
```

File: ProjectManagementSystem.java | Package: (default) (continued)

```
296:         }
297:     }
298:
299:     public static class SuperAdmin extends User {
300:         private String department;
301:
302:         public SuperAdmin(Integer id, String username, String password, String fullName, String em
303:             super(id, username, password, fullName, email, UserRole.SUPER_ADMIN);
304:             AppLogger.log(LogLevel.DEBUG, "[Constructor Chaining] SuperAdmin constructor initializ
305:             this.department = (dept != null && !dept.trim().isEmpty()) ? dept : "Executive Governa
306:         }
307:
308:         @Override public boolean hasPermission(Permission permission) { return true; }
309:         @Override public String getRoleDescription() { return "Super Administrator (Root Authority
310:         @Override public String getDepartment() { return department != null ? department : "Execut
311:         @Override public void setDepartment(String d) { this.department = d; }
312:     }
313:
314:     public static class Manager extends User {
315:         private String department;
316:         private double budgetApprovalLimit;
317:
318:         public Manager(Integer id, String username, String password, String fullName, String email
319:             super(id, username, password, fullName, email, UserRole.MANAGER);
320:             AppLogger.log(LogLevel.DEBUG, "[Constructor Chaining] Manager constructor initialized:
321:             this.department = (dept != null && !dept.trim().isEmpty()) ? dept : "Engineering Manag
322:             this.budgetApprovalLimit = limit;
323:         }
324:
325:         @Override public boolean hasPermission(Permission permission) {
326:             if (permission == Permission.MANAGE_ADMINS) return false;
327:             return true;
328:         }
329:         @Override public String getRoleDescription() { return "Project Manager / Admin (Executive
330:         @Override public String getDepartment() { return department != null ? department : "Engine
331:         @Override public void setDepartment(String d) { this.department = d; }
332:         public double getBudgetApprovalLimit() { return budgetApprovalLimit; }
333:         public void setBudgetApprovalLimit(double l) { this.budgetApprovalLimit = l; }
334:     }
335:
336:     public static class Developer extends User {
337:         private String department;
338:         private double hourlyRate;
339:
340:         public Developer(Integer id, String username, String password, String fullName, String ema
341:             super(id, username, password, fullName, email, UserRole.DEVELOPER);
342:             AppLogger.log(LogLevel.DEBUG, "[Constructor Chaining] Developer constructor initialize
343:             this.department = (dept != null && !dept.trim().isEmpty()) ? dept : "Frontend Team";
344:             this.hourlyRate = rate;
345:         }
346:
347:         @Override public boolean hasPermission(Permission permission) {
348:             switch (permission) {
349:                 case UPDATE_TASK_STATUS:
350:                 case LOG_HOURS:
351:                 case CREATE_TASK:
352:                 case EDIT_TASK:
353:                     return true;
354:                 default:
```

File: ProjectManagementSystem.java | Package: (default) (continued)

```
355:                     return false;
356:             }
357:         }
358:
359:         @Override public String getRoleDescription() { return "Software Developer / User (Executio
360:         @Override public String getDepartment() { return department != null ? department : "Fronte
361:         @Override public void setDepartment(String d) { this.department = d; }
362:         public String getTechStack() { return department; }
363:         public void setTechStack(String s) { this.department = s; }
364:         public double getHourlyRate() { return hourlyRate; }
365:         public void setHourlyRate(double r) { this.hourlyRate = r; }
366:     }
367:
368:     /**
369:      * Session and Attendance Tracking Entity: Records login time-in and logout timestamps.
370:      */
371:     public static class TimeInSession {
372:         private final String sessionId;
373:         private final int userId;
374:         private final String username;
375:         private final String fullName;
376:         private final UserRole role;
377:         private final LocalDateTime timeIn;
378:         private LocalDateTime timeOut;
379:         private String status; // "ACTIVE" or "LOGGED_OUT"
380:
381:         public TimeInSession(String sessionId, int userId, String username, String fullName, UserR
382:             this.sessionId = sessionId;
383:             this.userId = userId;
384:             this.username = username;
385:             this.fullName = fullName;
386:             this.role = role;
387:             this.timeIn = timeIn;
388:             this.status = "ACTIVE";
389:         }
390:
391:         public String getSessionId() { return sessionId; }
392:         public int getUserId() { return userId; }
393:         public String getUsername() { return username; }
394:         public String getFullName() { return fullName; }
395:         public UserRole getRole() { return role; }
396:         public LocalDateTime getTimeIn() { return timeIn; }
397:         public LocalDateTime getTimeOut() { return timeOut; }
398:         public void setTimeOut(LocalDateTime timeOut) {
399:             this.timeOut = timeOut;
400:             this.status = "LOGGED_OUT";
401:         }
402:         public String getStatus() { return status; }
403:
404:         public String getFormattedDuration() {
405:             LocalDateTime end = (timeOut != null) ? timeOut : LocalDateTime.now();
406:             long mins = java.time.Duration.between(timeIn, end).toMinutes();
407:             if ("ACTIVE".equals(status)) {
408:                 return (mins <= 0 ? "< 1 min" : mins + " min(s)") + " (Active)";
409:             }
410:             if (mins < 60) return mins + " min(s)";
411:             long h = mins / 60;
412:             long m = mins % 60;
413:             return h + "h " + m + "m";
```

File: ProjectManagementSystem.java | Package: (default) (continued)

```
414:         }
415:     }
416:
417:     /**
418:      * Work Logging Audit Entity: Records task work hours logged by team members.
419:      */
420:     public static class TaskWorkLog {
421:         private final String logId;
422:         private final int taskId;
423:         private final String taskTitle;
424:         private final String projectName;
425:         private final int userId;
426:         private final String userName;
427:         private final double hoursLogged;
428:         private final LocalDateTime timestamp;
429:         private final String notes;
430:
431:         public TaskWorkLog(String logId, int taskId, String taskTitle, String projectName, int use
432:             this.logId = logId;
433:             this.taskId = taskId;
434:             this.taskTitle = taskTitle;
435:             this.projectName = projectName;
436:             this.userId = userId;
437:             this.userName = userName;
438:             this.hoursLogged = hoursLogged;
439:             this.timestamp = timestamp;
440:             this.notes = notes;
441:         }
442:
443:         public String getLogId() { return logId; }
444:         public int getTaskId() { return taskId; }
445:         public String getTaskTitle() { return taskTitle; }
446:         public String getProjectName() { return projectName; }
447:         public int getUserId() { return userId; }
448:         public String getUserName() { return userName; }
449:         public double getHoursLogged() { return hoursLogged; }
450:         public LocalDateTime getTimestamp() { return timestamp; }
451:         public String getNotes() { return notes; }
452:     }
453:
454:     public enum ProjectStatus {
455:         PLANNING("Planning"), IN_PROGRESS("In Progress"), ON_HOLD("On Hold"), COMPLETED("Completed
456:         private final String label;
457:         ProjectStatus(String label) { this.label = label; }
458:         public String getLabel() { return label; }
459:         @Override public String toString() { return label; }
460:     }
461:
462:     public enum TaskStatus {
463:         TODO("To Do"), IN_PROGRESS("In Progress"), REVIEW("In Review"), COMPLETED("Completed");
464:         private final String label;
465:         TaskStatus(String label) { this.label = label; }
466:         public String getLabel() { return label; }
467:         @Override public String toString() { return label; }
468:
469:         public boolean canTransitionTo(TaskStatus next) {
470:             if (this == next) return true;
471:             switch (this) {
472:                 case TODO:        return next == IN_PROGRESS;
```

File: ProjectManagementSystem.java | Package: (default) (continued)

```
473:                 case IN_PROGRESS: return next == REVIEW || next == TODO;
474:                 case REVIEW:      return next == COMPLETED || next == IN_PROGRESS;
475:                 case COMPLETED:   return next == IN_PROGRESS;
476:                 default:          return false;
477:             }
478:         }
479:     }
480:
481:     public enum TaskPriority {
482:         LOW("Low"), MEDIUM("Medium"), HIGH("High"), CRITICAL("Critical");
483:         private final String label;
484:         TaskPriority(String label) { this.label = label; }
485:         public String getLabel() { return label; }
486:         @Override public String toString() { return label; }
487:     }
488:
489:     public static class Project extends BaseEntity {
490:         private static long projectCount = 0;
491:         private String title;
492:         private String description;
493:         private double budget;
494:         private ProjectStatus status;
495:         private String clientName;
496:         private String[] tags;
497:
498:         public Project(String title, String description) {
499:             this(null, title, description, 0.0, ProjectStatus.PLANNING, "Internal Client", new Str
500:         }
501:
502:         public Project(Integer id, String title, String description, double budget, ProjectStatus
503:             super(id);
504:             this.title = title != null ? title.trim() : "";
505:             this.description = description != null ? description.trim() : "";
506:             this.budget = budget;
507:             this.status = status != null ? status : ProjectStatus.PLANNING;
508:             this.clientName = client != null ? client.trim() : "Internal Client";
509:             this.tags = tags != null ? Arrays.copyOf(tags, tags.length) : new String[0];
510:             projectCount++;
511:         }
512:
513:         public static long getProjectCount() { return projectCount; }
514:
515:         public void addTag(String tag) {
516:             if (tag == null || tag.trim().isEmpty()) return;
517:             String[] newTags = Arrays.copyOf(tags, tags.length + 1);
518:             newTags[newTags.length - 1] = tag.trim();
519:             this.tags = newTags;
520:         }
521:
522:         public Vector<String> getTagsAsVector() {
523:             Vector<String> vec = new Vector<>();
524:             if (tags != null) for (String t : tags) vec.add(t);
525:             return vec;
526:         }
527:
528:         public String getTagsFormatted() {
529:             if (tags == null || tags.length == 0) return "";
530:             StringBuilder sb = new StringBuilder();
531:             for (int i = 0; i < tags.length; i++) {
```

File: ProjectManagementSystem.java | Package: (default) (continued)

```
532:                 sb.append(tags[i]);
533:                 if (i < tags.length - 1) sb.append(", ");
534:             }
535:             return sb.toString();
536:         }
537:
538:         public String getTitle() { return title; }
539:         public void setTitle(String t) { this.title = t; }
540:         public String getDescription() { return description; }
541:         public void setDescription(String d) { this.description = d; }
542:         public double getBudget() { return budget; }
543:         public void setBudget(double b) { this.budget = b; }
544:         public ProjectStatus getStatus() { return status; }
545:         public void setStatus(ProjectStatus s) { this.status = s; }
546:         public String getClientName() { return clientName; }
547:         public void setClientName(String c) { this.clientName = c; }
548:         public String[] getTags() { return Arrays.copyOf(tags, tags.length); }
549:         public void setTags(String[] t) { this.tags = t != null ? Arrays.copyOf(t, t.length) : nev
550:
551:         public class ProjectSummary {
552:             private final int taskCount;
553:             private final double progress;
554:             public ProjectSummary(int taskCount, double progress) {
555:                 this.taskCount = taskCount;
556:                 this.progress = progress;
557:             }
558:             public String getFormattedSummary() {
559:                 return String.format("Project '%s' [%s] - %d tasks, %.1f%% complete. Budget: $%,.2
560:                         Project.this.title, Project.this.status, taskCount, progress, Project.this
561:             }
562:         }
563:     }
564:
565:
566:     public static class Team extends BaseEntity {
567:         private String name;
568:         private String description;
569:         public Team(String name, String description) { this(null, name, description); }
570:         public Team(Integer id, String name, String description) {
571:             super(id);
572:             this.name = name != null ? name.trim() : "";
573:             this.description = description != null ? description.trim() : "";
574:         }
575:         public String getName() { return name; }
576:         public void setName(String n) { this.name = n; }
577:         public String getDescription() { return description; }
578:         public void setDescription(String d) { this.description = d; }
579:         @Override public String toString() { return name; }
580:     }
581:
582:     public static class Task extends BaseEntity {
583:         private int projectId;
584:         private String title;
585:         private String description;
586:         private Integer assignedUserId;
587:         private String assignedTeam = "General Team";
588:         private Integer parentTaskId;
589:         private TaskStatus status;
590:         private TaskPriority priority;
```

File: ProjectManagementSystem.java | Package: (default) (continued)

```
591:         private double estimatedHours;
592:         private double loggedHours;
593:
594:         private final List<Task> subTasks = new ArrayList<>();
595:
596:         public Task(String title, int projectId) {
597:             this(null, projectId, title, "", null, null, TaskStatus.TODO, TaskPriority.MEDIUM, 0.0
598:         }
599:
600:         public Task(Integer id, int projectId, String title, String description, Integer userId,
601:                     Integer parentId, TaskStatus status, TaskPriority priority, double estHours, d
602:             super(id);
603:             this.projectId = projectId;
604:             this.title = title != null ? title.trim() : "";
605:             this.description = description != null ? description.trim() : "";
606:             this.assignedUserId = userId;
607:             this.parentTaskId = parentId;
608:             this.status = status != null ? status : TaskStatus.TODO;
609:             this.priority = priority != null ? priority : TaskPriority.MEDIUM;
610:             this.estimatedHours = estHours;
611:             this.loggedHours = loggedHours;
612:         }
613:
614:         public void addSubTask(Task child) {
615:             if (child != null && !subTasks.contains(child)) {
616:                 child.setParentTaskId(this.id);
617:                 child.setProjectId(this.projectId);
618:                 subTasks.add(child);
619:             }
620:         }
621:
622:         public List<Task> getSubTasks() { return Collections.unmodifiableList(subTasks); }
623:         public void clearSubTasks() { subTasks.clear(); }
624:
625:         // ========================================================================
626:         // MODULE 2: RECURSIVE ALGORITHMS
627:         // ========================================================================
628:
629:         public double calculateCompletionPercentage() {
630:             if (subTasks.isEmpty()) {
631:                 switch (this.status) {
632:                     case COMPLETED:   return 100.0;
633:                     case REVIEW:      return 80.0;
634:                     case IN_PROGRESS: return estimatedHours > 0 ? Math.min(90.0, Math.max(25.0, (l
635:                     case TODO:
636:                     default:          return 0.0;
637:                 }
638:             }
639:             double sum = 0.0;
640:             for (Task child : subTasks) {
641:                 sum += child.calculateCompletionPercentage(); // Recursive call
642:             }
643:             double avg = sum / (double) subTasks.size();
644:             return Math.round(avg * 10.0) / 10.0;
645:         }
646:
647:         public int calculateDependencyDepth() {
648:             if (subTasks.isEmpty()) return 0;
649:             int maxDepth = 0;
```

File: ProjectManagementSystem.java | Package: (default) (continued)

```
650:             for (Task child : subTasks) {
651:                 int d = child.calculateDependencyDepth(); // Recursive call
652:                 if (d > maxDepth) maxDepth = d;
653:             }
654:             return 1 + maxDepth;
655:         }
656:
657:         public int countTotalSubTasksRecursive() {
658:             int count = subTasks.size();
659:             for (Task child : subTasks) {
660:                 count += child.countTotalSubTasksRecursive(); // Recursive call
661:             }
662:             return count;
663:         }
664:
665:         public void logHours(double hours) { logHours(hours, "Work in progress"); }
666:         public void logHours(double hours, String note) {
667:             if (hours > 0) {
668:                 this.loggedHours += hours;
669:                 this.updatedAt = LocalDateTime.now();
670:                 AppLogger.info("Logged %.2f hours on task #%d '%s' [%s]", hours, id != null ? id :
671:             }
672:         }
673:
674:         public int getProjectId() { return projectId; }
675:         public void setProjectId(int pid) { this.projectId = pid; }
676:         public String getTitle() { return title; }
677:         public void setTitle(String t) { this.title = t; }
678:         public String getDescription() { return description; }
679:         public void setDescription(String d) { this.description = d; }
680:         public Integer getAssignedUserId() { return assignedUserId; }
681:         public void setAssignedUserId(Integer uid) { this.assignedUserId = uid; }
682:         public String getAssignedTeam() { return (assignedTeam != null && !assignedTeam.trim().isE
683:         public void setAssignedTeam(String team) { this.assignedTeam = (team != null && !team.trim
684:         public Integer getParentTaskId() { return parentTaskId; }
685:         public void setParentTaskId(Integer pid) { this.parentTaskId = pid; }
686:         public TaskStatus getStatus() { return status; }
687:         public void setStatus(TaskStatus s) { this.status = s; this.updatedAt = LocalDateTime.now(
688:         public TaskPriority getPriority() { return priority; }
689:         public void setPriority(TaskPriority p) { this.priority = p; }
690:         public double getEstimatedHours() { return estimatedHours; }
691:         public void setEstimatedHours(double h) { this.estimatedHours = h; }
692:         public double getLoggedHours() { return loggedHours; }
693:         public void setLoggedHours(double h) { this.loggedHours = h; }
694:
695:         public static class Builder {
696:             private Integer id;
697:             private final int projectId;
698:             private final String title;
699:             private String description = "";
700:             private Integer assignedUserId;
701:             private String assignedTeam = "General Team";
702:             private Integer parentTaskId;
703:             private TaskStatus status = TaskStatus.TODO;
704:             private TaskPriority priority = TaskPriority.MEDIUM;
705:             private double estimatedHours = 0.0;
706:             private double loggedHours = 0.0;
707:
708:             public Builder(int projectId, String title) { this.projectId = projectId; this.title =
```

File: ProjectManagementSystem.java | Package: (default) (continued)

```
709:             public Builder id(Integer id) { this.id = id; return this; }
710:             public Builder description(String d) { this.description = d; return this; }
711:             public Builder assignedUser(Integer u) { this.assignedUserId = u; return this; }
712:             public Builder assignedTeam(String t) { this.assignedTeam = t; return this; }
713:             public Builder parentTask(Integer p) { this.parentTaskId = p; return this; }
714:             public Builder status(TaskStatus s) { this.status = s; return this; }
715:             public Builder priority(TaskPriority p) { this.priority = p; return this; }
716:             public Builder estimatedHours(double h) { this.estimatedHours = h; return this; }
717:             public Builder loggedHours(double h) { this.loggedHours = h; return this; }
718:             public Task build() {
719:                 Task t = new Task(id, projectId, title, description, assignedUserId, parentTaskId,
720:                 t.setAssignedTeam(assignedTeam);
721:                 return t;
722:             }
723:         }
724:     }
725:
726:     // ================================================================================
727:     // MODULE 3: DATA ACCESS LAYER & THREAD-SAFE SINGLETON DATABASE MANAGER
728:     // ================================================================================
729:
730:     public static class DatabaseManager {
731:         private static volatile DatabaseManager instance;
732:
733:         private Connection dbConnection = null;
734:         private boolean isSqliteConnected = false;
735:         private static final String DB_FILE = "pms_database.db";
736:         private static final String DB_URL = "jdbc:sqlite:" + DB_FILE;
737:
738:         private final List<User> userTable = new ArrayList<>();
739:         private final List<Project> projectTable = new ArrayList<>();
740:         private final List<Task> taskTable = new ArrayList<>();
741:         private final List<Team> teamTable = new ArrayList<>();
742:         private final List<TimeInSession> sessionLogs = new ArrayList<>();
743:         private final List<TaskWorkLog> workLogs = new ArrayList<>();
744:         private int userIdSequence = 1;
745:         private int teamIdSequence = 1;
746:         private int projectIdSequence = 1;
747:         private int taskIdSequence = 1;
748:         private int sessionSequence = 101;
749:         private int workLogSequence = 501;
750:         private TimeInSession currentActiveSession = null;
751:
752:         private DatabaseManager() {
753:             initDatabaseConnection();
754:         }
755:
756:         public static DatabaseManager getInstance() {
757:             if (instance == null) {
758:                 synchronized (DatabaseManager.class) {
759:                     if (instance == null) {
760:                         instance = new DatabaseManager();
761:                     }
762:                 }
763:             }
764:             return instance;
765:         }
766:
767:         public String getDatabaseStatusText() {
```

File: ProjectManagementSystem.java | Package: (default) (continued)

```
768:             if (isSqliteConnected) {
769:                 return "[DATABASE CONNECTED] SQLite DB: " + DB_FILE + " | Tables: USERS, TEAMS, PR
770:             } else {
771:                 return "[DATABASE CONNECTED] Embedded SQLite Storage: " + DB_FILE + " | Schema: Ac
772:             }
773:         }
774: 
775:         private void initDatabaseConnection() {
776:             try {
777:                 boolean driverFound = false;
778:                 try {
779:                     Class.forName("org.sqlite.JDBC");
780:                     driverFound = true;
781:                 } catch (Throwable t) {
782:                     File[] candidateJars = new File[]{
783:                             new File("lib/sqlite-jdbc.jar"),
784:                             new File("C:/Users/User/Desktop/pms-app/lib/sqlite-jdbc.jar"),
785:                             new File("C:/Users/User/Desktop/lib/sqlite-jdbc.jar")
786:                     };
787:                     for (File jar : candidateJars) {
788:                         if (jar.exists()) {
789:                             try {
790:                                 File parentDir = jar.getParentFile();
791:                                 List<java.net.URL> urls = new ArrayList<>();
792:                                 urls.add(jar.toURI().toURL());
793:                                 File slf4jApi = new File(parentDir, "slf4j-api.jar");
794:                                 if (slf4jApi.exists()) urls.add(slf4jApi.toURI().toURL());
795:                                 File slf4jSimple = new File(parentDir, "slf4j-simple.jar");
796:                                 if (slf4jSimple.exists()) urls.add(slf4jSimple.toURI().toURL());
797: 
798:                                 java.net.URLClassLoader ucl = new java.net.URLClassLoader(
799:                                         urls.toArray(new java.net.URL[0]),
800:                                         DatabaseManager.class.getClassLoader()
801:                                 );
802:                                 Driver driver = (Driver) Class.forName("org.sqlite.JDBC", true, uc
803:                                 DriverManager.registerDriver(new DriverShim(driver));
804:                                 driverFound = true;
805:                                 AppLogger.info("Dynamically registered SQLite JDBC Driver from: %s
806:                                 break;
807:                             } catch (Throwable ex) {
808:                                 AppLogger.warn("Could not dynamically load driver from %s: %s", ja
809:                             }
810:                         }
811:                     }
812:                 }
813: 
814:                 if (driverFound) {
815:                     dbConnection = DriverManager.getConnection(DB_URL);
816:                     isSqliteConnected = true;
817:                     AppLogger.info("Connected to SQLite Database: %s", DB_URL);
818:                     createSqlTablesIfNotExist();
819:                     loadFromDatabaseOrSeed();
820:                     return;
821:                 }
822:             } catch (Exception ex) {
823:                 AppLogger.warn("SQLite initialization note: %s. Using persistent local storage.",
824:             }
825: 
826:             seedDefaultUsers();
```

File: ProjectManagementSystem.java | Package: (default) (continued)

```
827:         }
828:
829:         private void createSqlTablesIfNotExist() {
830:             if (!isSqliteConnected || dbConnection == null) return;
831:             try (Statement stmt = dbConnection.createStatement()) {
832:                 stmt.execute("CREATE TABLE IF NOT EXISTS users (" +
833:                         "id INTEGER PRIMARY KEY, " +
834:                         "username TEXT UNIQUE NOT NULL, " +
835:                         "password TEXT NOT NULL, " +
836:                         "full_name TEXT NOT NULL, " +
837:                         "email TEXT NOT NULL, " +
838:                         "role TEXT NOT NULL, " +
839:                         "dept_or_stack TEXT, " +
840:                         "metric_val REAL);");
841:
842:                 stmt.execute("CREATE TABLE IF NOT EXISTS teams (" +
843:                         "id INTEGER PRIMARY KEY, " +
844:                         "name TEXT UNIQUE NOT NULL, " +
845:                         "description TEXT);");
846:
847:                 stmt.execute("CREATE TABLE IF NOT EXISTS projects (" +
848:                         "id INTEGER PRIMARY KEY, " +
849:                         "title TEXT NOT NULL, " +
850:                         "description TEXT, " +
851:                         "budget REAL, " +
852:                         "status TEXT NOT NULL, " +
853:                         "client_name TEXT, " +
854:                         "tags TEXT);");
855:
856:                 stmt.execute("CREATE TABLE IF NOT EXISTS tasks (" +
857:                         "id INTEGER PRIMARY KEY, " +
858:                         "project_id INTEGER NOT NULL, " +
859:                         "title TEXT NOT NULL, " +
860:                         "description TEXT, " +
861:                         "assigned_user_id INTEGER, " +
862:                         "parent_task_id INTEGER, " +
863:                         "status TEXT NOT NULL, " +
864:                         "priority TEXT NOT NULL, " +
865:                         "estimated_hours REAL DEFAULT 0, " +
866:                         "logged_hours REAL DEFAULT 0, " +
867:                         "assigned_team TEXT DEFAULT 'General Team');");
868:                 try { stmt.execute("ALTER TABLE tasks ADD COLUMN assigned_team TEXT DEFAULT 'Gener
869:
870:                 stmt.execute("CREATE TABLE IF NOT EXISTS sessions (" +
871:                         "session_id TEXT PRIMARY KEY, " +
872:                         "user_id INTEGER NOT NULL, " +
873:                         "username TEXT NOT NULL, " +
874:                         "full_name TEXT NOT NULL, " +
875:                         "role TEXT NOT NULL, " +
876:                         "time_in TEXT NOT NULL, " +
877:                         "time_out TEXT, " +
878:                         "status TEXT NOT NULL);");
879:
880:                 stmt.execute("CREATE TABLE IF NOT EXISTS work_logs (" +
881:                         "log_id TEXT PRIMARY KEY, " +
882:                         "task_id INTEGER NOT NULL, " +
883:                         "task_title TEXT NOT NULL, " +
884:                         "project_name TEXT NOT NULL, " +
885:                         "user_id INTEGER NOT NULL, " +
```

File: ProjectManagementSystem.java | Package: (default) (continued)

```
886:                         "user_name TEXT NOT NULL, " +
887:                         "hours_logged REAL NOT NULL, " +
888:                         "timestamp TEXT NOT NULL, " +
889:                         "notes TEXT);");
890:             } catch (SQLException e) {
891:                 AppLogger.error("Failed to create database schema tables: %s", e.getMessage());
892:             }
893:         }
894:
895:         private void loadFromDatabaseOrSeed() {
896:             if (!isSqliteConnected || dbConnection == null) {
897:                 seedDefaultUsers();
898:                 return;
899:             }
900:             try {
901:                 try (Statement stmt = dbConnection.createStatement()) {
902:                     ResultSet rs = stmt.executeQuery("SELECT COUNT(*) FROM users;");
903:                     if (rs.next() && rs.getInt(1) > 0) {
904:                         loadUsersFromSql();
905:                         AppLogger.info("Loaded users directly from SQLite database: %s", DB_FILE);
906:                     } else {
907:                         seedDefaultUsers();
908:                         persistDefaultUsersToSql();
909:                         AppLogger.info("Initialized default accounts in SQLite database: %s", DB_F
910:                     }
911:                 }
912:                 loadTeamsFromSql();
913:                 loadProjectsFromSql();
914:                 loadTasksFromSql();
915:                 loadSessionsFromSql();
916:                 loadWorkLogsFromSql();
917:                 AppLogger.info("Real-time SQLite sync completed. Projects: %d, Tasks: %d", project
918:             } catch (SQLException e) {
919:                 AppLogger.error("Error during database sync: %s", e.getMessage());
920:             }
921:         }
922:
923:         private void seedDefaultUsers() {
924:             userTable.clear();
925:             projectTable.clear();
926:             taskTable.clear();
927:             sessionLogs.clear();
928:             workLogs.clear();
929:             userTable.add(new SuperAdmin(1, "superadmin", "superadmin123", "Chief Administrator",
930:             userTable.add(new Manager(2, "admin", "admin123", "Dr. Arun Kumar (Manager)", "admin@e
931:             userTable.add(new Developer(3, "user", "user1234", "Standard User (Developer)", "user@
932:             userTable.add(new Developer(4, "dev_alice", "alice123", "Alice George (Developer)", "a
933:             userTable.add(new Developer(5, "dev_bob", "bob12345", "Bob Varghese (Developer)", "bob
934:             userIdSequence = 6;
935:             seedDefaultTeams();
936:             projectIdSequence = 1;
937:             taskIdSequence = 1;
938:             sessionSequence = 101;
939:             workLogSequence = 501;
940:         }
941:
942:         private void persistDefaultUsersToSql() {
943:             if (!isSqliteConnected || dbConnection == null) return;
944:             try {
```

File: ProjectManagementSystem.java | Package: (default) (continued)

```
945:                 String sqlUser = "INSERT OR REPLACE INTO users (id, username, password, full_name,
946:                 try (PreparedStatement ps = dbConnection.prepareStatement(sqlUser)) {
947:                     for (User u : userTable) {
948:                         ps.setInt(1, u.getId());
949:                         ps.setString(2, u.getUsername());
950:                         ps.setString(3, u.getPassword());
951:                         ps.setString(4, u.getFullName());
952:                         ps.setString(5, u.getEmail());
953:                         ps.setString(6, u.getRole().name());
954:                         ps.setString(7, u.getDepartment());
955:                         if (u instanceof Manager) {
956:                             ps.setDouble(8, ((Manager) u).getBudgetApprovalLimit());
957:                         } else if (u instanceof Developer) {
958:                             ps.setDouble(8, ((Developer) u).getHourlyRate());
959:                         } else {
960:                             ps.setDouble(8, 0.0);
961:                         }
962:                         ps.executeUpdate();
963:                     }
964:                 }
965:             } catch (SQLException e) {
966:                 AppLogger.error("Error writing default users to SQL: %s", e.getMessage());
967:             }
968:         }
969:
970:         private void loadUsersFromSql() {
971:             try (Statement stmt = dbConnection.createStatement();
972:                  ResultSet rs = stmt.executeQuery("SELECT * FROM users;")) {
973:                 userTable.clear();
974:                 int maxId = 5;
975:                 while (rs.next()) {
976:                     int id = rs.getInt("id");
977:                     String un = rs.getString("username");
978:                     String pw = rs.getString("password");
979:                     String fn = rs.getString("full_name");
980:                     String em = rs.getString("email");
981:                     String roleStr = rs.getString("role");
982:                     String extra = rs.getString("dept_or_stack");
983:                     double metric = rs.getDouble("metric_val");
984:                     if ("SUPER_ADMIN".equalsIgnoreCase(roleStr)) {
985:                         userTable.add(new SuperAdmin(id, un, pw, fn, em, extra != null ? extra : '
986:                     } else if ("MANAGER".equalsIgnoreCase(roleStr)) {
987:                         userTable.add(new Manager(id, un, pw, fn, em, extra != null ? extra : "Eng
988:                     } else {
989:                         userTable.add(new Developer(id, un, pw, fn, em, extra != null ? extra : "F
990:                     }
991:                     if (id > maxId) maxId = id;
992:                 }
993:                 userIdSequence = maxId + 1;
994:             } catch (SQLException e) {
995:                 AppLogger.error("Failed to load users from SQL: %s", e.getMessage());
996:             }
997:         }
998:
999:         private void loadProjectsFromSql() {
1000:             if (!isSqliteConnected || dbConnection == null) return;
1001:             try (Statement stmt = dbConnection.createStatement();
1002:                  ResultSet rs = stmt.executeQuery("SELECT * FROM projects ORDER BY id ASC;")) {
1003:                 projectTable.clear();
```

File: ProjectManagementSystem.java | Package: (default) (continued)

```
1004:                 int maxId = 0;
1005:                 while (rs.next()) {
1006:                     int id = rs.getInt("id");
1007:                     String title = rs.getString("title");
1008:                     String desc = rs.getString("description");
1009:                     double budget = rs.getDouble("budget");
1010:                     String statusStr = rs.getString("status");
1011:                     String client = rs.getString("client_name");
1012:                     String tagsRaw = rs.getString("tags");
1013:                     String[] tags = (tagsRaw != null && !tagsRaw.trim().isEmpty())
1014:                             ? Arrays.stream(tagsRaw.split(",")).map(String::trim).filter(s -> !s.i
1015:                             : new String[0];
1016:                     ProjectStatus status = ProjectStatus.PLANNING;
1017:                     try { status = ProjectStatus.valueOf(statusStr); } catch (Exception ignored) {
1018:                     Project p = new Project(id, title, desc, budget, status, client, tags);
1019:                     projectTable.add(p);
1020:                     if (id > maxId) maxId = id;
1021:                 }
1022:                 projectIdSequence = maxId + 1;
1023:                 AppLogger.info("Loaded %d project(s) directly from SQLite.", projectTable.size());
1024:             } catch (SQLException e) {
1025:                 AppLogger.error("Failed to load projects from SQL: %s", e.getMessage());
1026:             }
1027:         }
1028:
1029:         private void loadTasksFromSql() {
1030:             if (!isSqliteConnected || dbConnection == null) return;
1031:             try (Statement stmt = dbConnection.createStatement();
1032:                  ResultSet rs = stmt.executeQuery("SELECT * FROM tasks ORDER BY id ASC;")) {
1033:                 taskTable.clear();
1034:                 int maxId = 0;
1035:                 while (rs.next()) {
1036:                     int id = rs.getInt("id");
1037:                     int pid = rs.getInt("project_id");
1038:                     String title = rs.getString("title");
1039:                     String desc = rs.getString("description");
1040:                     int uidVal = rs.getInt("assigned_user_id");
1041:                     Integer assignedUser = rs.wasNull() ? null : uidVal;
1042:                     int parentVal = rs.getInt("parent_task_id");
1043:                     Integer parentTask = rs.wasNull() ? null : parentVal;
1044:                     String statusStr = rs.getString("status");
1045:                     String prioStr = rs.getString("priority");
1046:                     double est = rs.getDouble("estimated_hours");
1047:                     double logged = rs.getDouble("logged_hours");
1048:
1049:                     TaskStatus status = TaskStatus.TODO;
1050:                     try { status = TaskStatus.valueOf(statusStr); } catch (Exception ignored) {}
1051:                     TaskPriority prio = TaskPriority.MEDIUM;
1052:                     try { prio = TaskPriority.valueOf(prioStr); } catch (Exception ignored) {}
1053:                     String tm = null;
1054:                     try { tm = rs.getString("assigned_team"); } catch (Exception ignored) {}
1055:                     Task t = new Task(id, pid, title, desc, assignedUser, parentTask, status, pric
1056:                     t.setAssignedTeam(tm);
1057:                     taskTable.add(t);
1058:                     if (id > maxId) maxId = id;
1059:                 }
1060:                 taskIdSequence = maxId + 1;
1061:                 AppLogger.info("Loaded %d task(s) directly from SQLite.", taskTable.size());
1062:             } catch (SQLException e) {
```

File: ProjectManagementSystem.java | Package: (default) (continued)

```
1063:                 AppLogger.error("Failed to load tasks from SQL: %s", e.getMessage());
1064:             }
1065:         }
1066:
1067:         private void loadSessionsFromSql() {
1068:             if (!isSqliteConnected || dbConnection == null) return;
1069:             try (Statement stmt = dbConnection.createStatement();
1070:                  ResultSet rs = stmt.executeQuery("SELECT * FROM sessions ORDER BY rowid DESC;"))
1071:                 sessionLogs.clear();
1072:                 int maxSeq = 100;
1073:                 while (rs.next()) {
1074:                     String sId = rs.getString("session_id");
1075:                     int uid = rs.getInt("user_id");
1076:                     String un = rs.getString("username");
1077:                     String fn = rs.getString("full_name");
1078:                     String roleStr = rs.getString("role");
1079:                     String timeInStr = rs.getString("time_in");
1080:                     String timeOutStr = rs.getString("time_out");
1081:                     UserRole role = UserRole.DEVELOPER;
1082:                     try { role = UserRole.valueOf(roleStr); } catch (Exception ignored) {}
1083:                     LocalDateTime timeIn = LocalDateTime.now();
1084:                     try { timeIn = LocalDateTime.parse(timeInStr); } catch (Exception ignored) {}
1085:                     TimeInSession session = new TimeInSession(sId, uid, un, fn, role, timeIn);
1086:                     if (timeOutStr != null && !timeOutStr.trim().isEmpty()) {
1087:                         try { session.setTimeOut(LocalDateTime.parse(timeOutStr)); } catch (Except
1088:                     }
1089:                     sessionLogs.add(session);
1090:                     if (sId != null && sId.startsWith("SES-")) {
1091:                         try {
1092:                             int num = Integer.parseInt(sId.substring(4));
1093:                             if (num > maxSeq) maxSeq = num;
1094:                         } catch (Exception ignored) {}
1095:                     }
1096:                 }
1097:                 sessionSequence = maxSeq + 1;
1098:             } catch (SQLException e) {
1099:                 AppLogger.error("Failed to load sessions from SQL: %s", e.getMessage());
1100:             }
1101:         }
1102:
1103:         private void loadWorkLogsFromSql() {
1104:             if (!isSqliteConnected || dbConnection == null) return;
1105:             try (Statement stmt = dbConnection.createStatement();
1106:                  ResultSet rs = stmt.executeQuery("SELECT * FROM work_logs ORDER BY rowid DESC;"))
1107:                 workLogs.clear();
1108:                 int maxSeq = 500;
1109:                 while (rs.next()) {
1110:                     String logId = rs.getString("log_id");
1111:                     int taskId = rs.getInt("task_id");
1112:                     String taskTitle = rs.getString("task_title");
1113:                     String projName = rs.getString("project_name");
1114:                     int userId = rs.getInt("user_id");
1115:                     String userName = rs.getString("user_name");
1116:                     double hours = rs.getDouble("hours_logged");
1117:                     String tsStr = rs.getString("timestamp");
1118:                     String notes = rs.getString("notes");
1119:                     LocalDateTime ts = LocalDateTime.now();
1120:                     try { ts = LocalDateTime.parse(tsStr); } catch (Exception ignored) {}
1121:                     TaskWorkLog log = new TaskWorkLog(logId, taskId, taskTitle, projName, userId,
```

File: ProjectManagementSystem.java | Package: (default) (continued)

```
1122:                     workLogs.add(log);
1123:                     if (logId != null && logId.startsWith("LOG-")) {
1124:                         try {
1125:                             int num = Integer.parseInt(logId.substring(4));
1126:                             if (num > maxSeq) maxSeq = num;
1127:                         } catch (Exception ignored) {}
1128:                     }
1129:                 }
1130:                 workLogSequence = maxSeq + 1;
1131:             } catch (SQLException e) {
1132:                 AppLogger.error("Failed to load work logs from SQL: %s", e.getMessage());
1133:             }
1134:         }
1135:
1136:         public synchronized boolean deleteUser(int id) {
1137:             boolean removed = userTable.removeIf(u -> u.getId().equals(id));
1138:             for (Task t : taskTable) {
1139:                 if (t.getAssignedUserId() != null && t.getAssignedUserId().equals(id)) t.setAssigr
1140:             }
1141:             if (isSqliteConnected && dbConnection != null) {
1142:                 try {
1143:                     try (PreparedStatement psTask = dbConnection.prepareStatement("UPDATE tasks SE
1144:                          PreparedStatement psUser = dbConnection.prepareStatement("DELETE FROM use
1145:                         psTask.setInt(1, id); psTask.executeUpdate();
1146:                         psUser.setInt(1, id); psUser.executeUpdate();
1147:                     }
1148:                     AppLogger.info("Deleted User #%d from SQLite database.", id);
1149:                 } catch (SQLException e) {
1150:                     AppLogger.error("SQL error deleting user #%d: %s", id, e.getMessage());
1151:                 }
1152:             }
1153:             return removed;
1154:         }
1155:
1156:         public synchronized List<Team> getTeams() { return new ArrayList<>(teamTable); }
1157:
1158:         public synchronized Team saveTeam(Team t) {
1159:             if (t.getId() == null) t.setId(teamIdSequence++);
1160:             teamTable.removeIf(existing -> existing.getId().equals(t.getId()));
1161:             teamTable.add(t);
1162:             if (isSqliteConnected && dbConnection != null) {
1163:                 try {
1164:                     String sql = "INSERT OR REPLACE INTO teams (id, name, description) VALUES (?,
1165:                     try (PreparedStatement ps = dbConnection.prepareStatement(sql)) {
1166:                         ps.setInt(1, t.getId());
1167:                         ps.setString(2, t.getName());
1168:                         ps.setString(3, t.getDescription());
1169:                         ps.executeUpdate();
1170:                     }
1171:                 } catch (SQLException e) {
1172:                     AppLogger.error("SQL error saving team #%d: %s", t.getId(), e.getMessage());
1173:                 }
1174:             }
1175:             return t;
1176:         }
1177:
1178:         public synchronized boolean deleteTeam(int id) {
1179:             Team target = teamTable.stream().filter(t -> t.getId().equals(id)).findFirst().orElse(
1180:             boolean removed = teamTable.removeIf(t -> t.getId().equals(id));
```

File: ProjectManagementSystem.java | Package: (default) (continued)

```
1181:             if (target != null) {
1182:                 for (Task t : taskTable) {
1183:                     if (target.getName().equalsIgnoreCase(t.getAssignedTeam())) t.setAssignedTeam(
1184:                 }
1185:             }
1186:             if (isSqliteConnected && dbConnection != null) {
1187:                 try {
1188:                     if (target != null) {
1189:                         try (PreparedStatement psT = dbConnection.prepareStatement("UPDATE tasks S
1190:                             psT.setString(1, target.getName()); psT.executeUpdate();
1191:                         }
1192:                     }
1193:                     try (PreparedStatement ps = dbConnection.prepareStatement("DELETE FROM teams W
1194:                         ps.setInt(1, id); ps.executeUpdate();
1195:                     }
1196:                 } catch (SQLException e) {
1197:                     AppLogger.error("SQL error deleting team #%d: %s", id, e.getMessage());
1198:                 }
1199:             }
1200:             return removed;
1201:         }
1202:
1203:         private void seedDefaultTeams() {
1204:             teamTable.clear();
1205:             teamTable.add(new Team(1, "Frontend Team", "UI engineering, web portals & user experie
1206:             teamTable.add(new Team(2, "Backend Team", "Core microservices, APIs, data logic & SQL"
1207:             teamTable.add(new Team(3, "Mobile App Team", "Cross-platform Android & iOS applicatior
1208:             teamTable.add(new Team(4, "DevOps & Cloud Team", "CI/CD automation, cloud deployment &
1209:             teamTable.add(new Team(5, "QA & Testing Team", "Quality assurance, automated testing &
1210:             teamIdSequence = 6;
1211:             if (isSqliteConnected && dbConnection != null) {
1212:                 try {
1213:                     String sql = "INSERT OR REPLACE INTO teams (id, name, description) VALUES (?,
1214:                     try (PreparedStatement ps = dbConnection.prepareStatement(sql)) {
1215:                         for (Team tm : teamTable) {
1216:                             ps.setInt(1, tm.getId());
1217:                             ps.setString(2, tm.getName());
1218:                             ps.setString(3, tm.getDescription());
1219:                             ps.executeUpdate();
1220:                         }
1221:                     }
1222:                 } catch (SQLException e) {
1223:                     AppLogger.error("Error writing default teams to SQL: %s", e.getMessage());
1224:                 }
1225:             }
1226:         }
1227:
1228:         private void loadTeamsFromSql() {
1229:             if (!isSqliteConnected || dbConnection == null) return;
1230:             try (Statement stmt = dbConnection.createStatement();
1231:                  ResultSet rs = stmt.executeQuery("SELECT * FROM teams ORDER BY id ASC;")) {
1232:                 teamTable.clear();
1233:                 int maxId = 0;
1234:                 while (rs.next()) {
1235:                     int id = rs.getInt("id");
1236:                     String name = rs.getString("name");
1237:                     String desc = rs.getString("description");
1238:                     teamTable.add(new Team(id, name, desc));
1239:                     if (id > maxId) maxId = id;
```

File: ProjectManagementSystem.java | Package: (default) (continued)

```
1240:                 }
1241:                 if (teamTable.isEmpty()) {
1242:                     seedDefaultTeams();
1243:                 } else {
1244:                     teamIdSequence = maxId + 1;
1245:                 }
1246:                 AppLogger.info("Loaded %d teams directly from SQLite.", teamTable.size());
1247:             } catch (SQLException e) {
1248:                 AppLogger.error("Failed to load teams from SQL: %s", e.getMessage());
1249:             }
1250:         }
1251:
1252:                 public synchronized List<User> getUsers() { return new ArrayList<>(userTable); }
1253:         public synchronized Optional<User> authenticate(String u, String p) {
1254:             return userTable.stream().filter(usr -> usr.getUsername().equalsIgnoreCase(u) && usr.g
1255:         }
1256:         public synchronized boolean isUsernameTaken(String username) {
1257:             return userTable.stream().anyMatch(u -> u.getUsername().equalsIgnoreCase(username));
1258:         }
1259:         public synchronized User saveUser(User u) {
1260:             if (u.getId() == null) {
1261:                 u.setId(userIdSequence++);
1262:                 userTable.add(u);
1263:             } else {
1264:                 int idx = -1;
1265:                 for (int i = 0; i < userTable.size(); i++) {
1266:                     if (userTable.get(i).getId().equals(u.getId())) {
1267:                         idx = i;
1268:                         break;
1269:                     }
1270:                 }
1271:                 if (idx != -1) {
1272:                     userTable.set(idx, u);
1273:                 } else {
1274:                     userTable.add(u);
1275:                 }
1276:             }
1277:             if (isSqliteConnected && dbConnection != null) {
1278:                 try {
1279:                     String sql = "INSERT OR REPLACE INTO users (id, username, password, full_name,
1280:                     try (PreparedStatement ps = dbConnection.prepareStatement(sql)) {
1281:                         ps.setInt(1, u.getId());
1282:                         ps.setString(2, u.getUsername());
1283:                         ps.setString(3, u.getPassword());
1284:                         ps.setString(4, u.getFullName());
1285:                         ps.setString(5, u.getEmail());
1286:                         ps.setString(6, u.getRole().name());
1287:                         ps.setString(7, u.getDepartment());
1288:                         if (u instanceof Manager) {
1289:                             ps.setDouble(8, ((Manager) u).getBudgetApprovalLimit());
1290:                         } else if (u instanceof Developer) {
1291:                             ps.setDouble(8, ((Developer) u).getHourlyRate());
1292:                         } else {
1293:                             ps.setDouble(8, 0.0);
1294:                         }
1295:                         ps.executeUpdate();
1296:                     }
1297:                 } catch (SQLException e) {
1298:                     AppLogger.error("SQL error saving user #%d: %s", u.getId(), e.getMessage());
```

File: ProjectManagementSystem.java | Package: (default) (continued)

```
1299:                 }
1300:             }
1301:             return u;
1302:         }
1303:
1304:         public synchronized List<Project> getProjects() { return new ArrayList<>(projectTable); }
1305:         public synchronized Project saveProject(Project p) {
1306:             if (p.getId() == null) {
1307:                 p.setId(projectIdSequence++);
1308:                 projectTable.add(p);
1309:             } else {
1310:                 int idx = -1;
1311:                 for (int i = 0; i < projectTable.size(); i++) {
1312:                     if (projectTable.get(i).getId().equals(p.getId())) {
1313:                         idx = i;
1314:                         break;
1315:                     }
1316:                 }
1317:                 if (idx != -1) {
1318:                     projectTable.set(idx, p);
1319:                 } else {
1320:                     projectTable.add(p);
1321:                 }
1322:             }
1323:             if (isSqliteConnected && dbConnection != null) {
1324:                 try {
1325:                     String sql = "INSERT OR REPLACE INTO projects (id, title, description, budget,
1326:                     try (PreparedStatement ps = dbConnection.prepareStatement(sql)) {
1327:                         ps.setInt(1, p.getId());
1328:                         ps.setString(2, p.getTitle());
1329:                         ps.setString(3, p.getDescription());
1330:                         ps.setDouble(4, p.getBudget());
1331:                         ps.setString(5, p.getStatus().name());
1332:                         ps.setString(6, p.getClientName());
1333:                         ps.setString(7, p.getTagsFormatted());
1334:                         ps.executeUpdate();
1335:                     }
1336:                 } catch (SQLException e) {
1337:                     AppLogger.error("SQL error saving project #%d: %s", p.getId(), e.getMessage())
1338:                 }
1339:             }
1340:             return p;
1341:         }
1342:         public synchronized boolean deleteProject(int id) {
1343:             Set<Integer> taskIds = new HashSet<>();
1344:             for (Task t : taskTable) {
1345:                 if (t.getProjectId() == id) taskIds.add(t.getId());
1346:             }
1347:             workLogs.removeIf(w -> taskIds.contains(w.getTaskId()));
1348:             taskTable.removeIf(t -> t.getProjectId() == id);
1349:             boolean removed = projectTable.removeIf(p -> p.getId() == id);
1350:             if (isSqliteConnected && dbConnection != null) {
1351:                 try {
1352:                     try (PreparedStatement psLogs = dbConnection.prepareStatement("DELETE FROM wor
1353:                          PreparedStatement psTasks = dbConnection.prepareStatement("DELETE FROM ta
1354:                          PreparedStatement psProj = dbConnection.prepareStatement("DELETE FROM pro
1355:                         psLogs.setInt(1, id); psLogs.executeUpdate();
1356:                         psTasks.setInt(1, id); psTasks.executeUpdate();
1357:                         psProj.setInt(1, id); psProj.executeUpdate();
```

File: ProjectManagementSystem.java | Package: (default) (continued)

```
1358:                     }
1359:                     AppLogger.info("Permanently deleted Project #%d and cascaded tasks & logs fron
1360:                 } catch (SQLException e) {
1361:                     AppLogger.error("SQL error deleting project #%d: %s", id, e.getMessage());
1362:                 }
1363:             }
1364:             return removed;
1365:         }
1366:
1367:         public synchronized List<Task> getTasks() { return new ArrayList<>(taskTable); }
1368:         public synchronized Task saveTask(Task t) {
1369:             if (t.getId() == null) {
1370:                 t.setId(taskIdSequence++);
1371:                 taskTable.add(t);
1372:             } else {
1373:                 int idx = -1;
1374:                 for (int i = 0; i < taskTable.size(); i++) {
1375:                     if (taskTable.get(i).getId().equals(t.getId())) {
1376:                         idx = i;
1377:                         break;
1378:                     }
1379:                 }
1380:                 if (idx != -1) {
1381:                     taskTable.set(idx, t);
1382:                 } else {
1383:                     taskTable.add(t);
1384:                 }
1385:             }
1386:             if (isSqliteConnected && dbConnection != null) {
1387:                 try {
1388:                     String sql = "INSERT OR REPLACE INTO tasks (id, project_id, title, descriptior
1389:                     try (PreparedStatement ps = dbConnection.prepareStatement(sql)) {
1390:                         ps.setInt(1, t.getId());
1391:                         ps.setInt(2, t.getProjectId());
1392:                         ps.setString(3, t.getTitle());
1393:                         ps.setString(4, t.getDescription());
1394:                         if (t.getAssignedUserId() != null) ps.setInt(5, t.getAssignedUserId()); el
1395:                         if (t.getParentTaskId() != null) ps.setInt(6, t.getParentTaskId()); else p
1396:                         ps.setString(7, t.getStatus().name());
1397:                         ps.setString(8, t.getPriority().name());
1398:                         ps.setDouble(9, t.getEstimatedHours());
1399:                         ps.setDouble(10, t.getLoggedHours());
1400:                         ps.setString(11, t.getAssignedTeam());
1401:                         ps.executeUpdate();
1402:                     }
1403:                 } catch (SQLException e) {
1404:                     AppLogger.error("SQL error saving task #%d: %s", t.getId(), e.getMessage());
1405:                 }
1406:             }
1407:             return t;
1408:         }
1409:         public synchronized boolean deleteTask(int id) {
1410:             Set<Integer> taskIdsToDelete = new HashSet<>();
1411:             taskIdsToDelete.add(id);
1412:             boolean addedMore = true;
1413:             while (addedMore) {
1414:                 addedMore = false;
1415:                 for (Task t : taskTable) {
1416:                     if (t.getParentTaskId() != null && taskIdsToDelete.contains(t.getParentTaskId(
```

File: ProjectManagementSystem.java | Package: (default) (continued)

```
1417:                         taskIdsToDelete.add(t.getId());
1418:                         addedMore = true;
1419:                     }
1420:                 }
1421:             }
1422:             workLogs.removeIf(w -> taskIdsToDelete.contains(w.getTaskId()));
1423:             boolean removed = taskTable.removeIf(t -> taskIdsToDelete.contains(t.getId()));
1424:             if (isSqliteConnected && dbConnection != null) {
1425:                 try {
1426:                     for (int tid : taskIdsToDelete) {
1427:                         try (PreparedStatement psLogs = dbConnection.prepareStatement("DELETE FRON
1428:                              PreparedStatement psTask = dbConnection.prepareStatement("DELETE FRON
1429:                             psLogs.setInt(1, tid); psLogs.executeUpdate();
1430:                             psTask.setInt(1, tid); psTask.executeUpdate();
1431:                         }
1432:                     }
1433:                     AppLogger.info("Permanently deleted Task #%d (and sub-tasks: %s) from SQLite."
1434:                 } catch (SQLException e) {
1435:                     AppLogger.error("SQL error deleting task #%d: %s", id, e.getMessage());
1436:                 }
1437:             }
1438:             return removed;
1439:         }
1440:
1441:         public synchronized TimeInSession recordLoginSession(User u) {
1442:             String sId = "SES-" + (sessionSequence++);
1443:             TimeInSession session = new TimeInSession(sId, u.getId(), u.getUsername(), u.getFullNa
1444:             sessionLogs.add(0, session);
1445:             this.currentActiveSession = session;
1446:             if (isSqliteConnected && dbConnection != null) {
1447:                 try {
1448:                     String sql = "INSERT INTO sessions (session_id, user_id, username, full_name,
1449:                     try (PreparedStatement ps = dbConnection.prepareStatement(sql)) {
1450:                         ps.setString(1, session.getSessionId());
1451:                         ps.setInt(2, session.getUserId());
1452:                         ps.setString(3, session.getUsername());
1453:                         ps.setString(4, session.getFullName());
1454:                         ps.setString(5, session.getRole().name());
1455:                         ps.setString(6, session.getTimeIn().toString());
1456:                         ps.setNull(7, java.sql.Types.VARCHAR);
1457:                         ps.setString(8, session.getStatus());
1458:                         ps.executeUpdate();
1459:                     }
1460:                 } catch (SQLException e) {
1461:                     AppLogger.error("SQL error recording session: %s", e.getMessage());
1462:                 }
1463:             }
1464:             return session;
1465:         }
1466:
1467:         public synchronized void recordLogoutSession(User u) {
1468:             TimeInSession target = null;
1469:             if (currentActiveSession != null && currentActiveSession.getUserId() == u.getId() && "
1470:                 currentActiveSession.setTimeOut(LocalDateTime.now());
1471:                 target = currentActiveSession;
1472:                 currentActiveSession = null;
1473:             } else {
1474:                 for (TimeInSession s : sessionLogs) {
1475:                     if (s.getUserId() == u.getId() && "ACTIVE".equals(s.getStatus())) {
```

File: ProjectManagementSystem.java | Package: (default) (continued)

```
                        s.setTimeOut(LocalDateTime.now());
                        target = s;
                        break;
                    }
                }
            }
            if (target != null && isSqliteConnected && dbConnection != null) {
                try {
                    String sql = "UPDATE sessions SET time_out = ?, status = ? WHERE session_id =
                    try (PreparedStatement ps = dbConnection.prepareStatement(sql)) {
                        ps.setString(1, target.getTimeOut().toString());
                        ps.setString(2, target.getStatus());
                        ps.setString(3, target.getSessionId());
                        ps.executeUpdate();
                    }
                } catch (SQLException e) {
                    AppLogger.error("SQL error updating logout session: %s", e.getMessage());
                }
            }
        }

        public synchronized void recordTaskWorkLog(Task task, Project project, User user, double h
            String wId = "LOG-" + (workLogSequence++);
            String pName = project != null ? project.getTitle() : ("Project #" + task.getProjectId
            TaskWorkLog log = new TaskWorkLog(wId, task.getId(), task.getTitle(), pName, user.getI
            workLogs.add(0, log);
            if (isSqliteConnected && dbConnection != null) {
                try {
                    String sqlUp = "UPDATE tasks SET logged_hours = ? WHERE id = ?;";
                    try (PreparedStatement psUp = dbConnection.prepareStatement(sqlUp)) {
                        psUp.setDouble(1, task.getLoggedHours());
                        psUp.setInt(2, task.getId());
                        psUp.executeUpdate();
                    }
                    String sql = "INSERT INTO work_logs (log_id, task_id, task_title, project_name
                    try (PreparedStatement ps = dbConnection.prepareStatement(sql)) {
                        ps.setString(1, log.getLogId());
                        ps.setInt(2, log.getTaskId());
                        ps.setString(3, log.getTaskTitle());
                        ps.setString(4, log.getProjectName());
                        ps.setInt(5, log.getUserId());
                        ps.setString(6, log.getUserName());
                        ps.setDouble(7, log.getHoursLogged());
                        ps.setString(8, log.getTimestamp().toString());
                        ps.setString(9, log.getNotes());
                        ps.executeUpdate();
                    }
                } catch (SQLException e) {
                    AppLogger.error("SQL error recording task work log: %s", e.getMessage());
                }
            }
        }

        public synchronized List<TimeInSession> getSessionLogs() { return new ArrayList<>(sessionL
        public synchronized List<TaskWorkLog> getWorkLogs() { return new ArrayList<>(workLogs);
        }

        private static class DriverShim implements Driver {
            private final Driver driver;
```

File: ProjectManagementSystem.java | Package: (default) (continued)

```
1535:             DriverShim(Driver d) { this.driver = d; }
1536:             @Override public boolean acceptsURL(String u) throws SQLException { return driver.acce
1537:             @Override public Connection connect(String u, Properties p) throws SQLException { retu
1538:             @Override public int getMajorVersion() { return driver.getMajorVersion(); }
1539:             @Override public int getMinorVersion() { return driver.getMinorVersion(); }
1540:             @Override public DriverPropertyInfo[] getPropertyInfo(String u, Properties p) throws S
1541:             @Override public boolean jdbcCompliant() { return driver.jdbcCompliant(); }
1542:             @Override public java.util.logging.Logger getParentLogger() throws SQLFeatureNotSuppor
1543:         }
1544:     }
1545:
1546:     // ================================================================================
1547:     // MODULE 3: SERVICE LAYER (SOLID PRINCIPLES: SRP, OCP, LSP, ISP, DIP)
1548:     // ================================================================================
1549:
1550:     public static class ProjectService {
1551:         private final DatabaseManager db;
1552:         public ProjectService(DatabaseManager db) { this.db = db; }
1553:
1554:         public Project saveProject(Project p, User initiator) throws UnauthorizedAccessException {
1555:             if (initiator == null || !initiator.hasPermission(Permission.CREATE_PROJECT)) {
1556:                 throw new UnauthorizedAccessException(initiator != null ? initiator.getUsername()
1557:             }
1558:             return db.saveProject(p);
1559:         }
1560:
1561:         public void deleteProject(int id, User initiator) throws UnauthorizedAccessException, Proj
1562:             if (initiator == null || !initiator.hasPermission(Permission.DELETE_PROJECT)) {
1563:                 throw new UnauthorizedAccessException(initiator != null ? initiator.getUsername()
1564:             }
1565:             boolean removed = db.deleteProject(id);
1566:             if (!removed) throw new ProjectNotFoundException(id);
1567:         }
1568:
1569:         public List<Project> getAllProjects() { return db.getProjects(); }
1570:
1571:         public double calculateProjectCompletion(int projectId) {
1572:             List<Task> topTasks = buildTaskHierarchy(projectId);
1573:             if (topTasks.isEmpty()) return 0.0;
1574:             double sum = 0.0;
1575:             for (Task t : topTasks) {
1576:                 sum += t.calculateCompletionPercentage(); // Recursive aggregation
1577:             }
1578:             return Math.round((sum / (double) topTasks.size()) * 10.0) / 10.0;
1579:         }
1580:
1581:         /**
1582:          * Robust Work Breakdown Structure (WBS) builder:
1583:          * Accurately groups all top-level tasks and assigns child tasks recursively.
1584:          */
1585:         public List<Task> buildTaskHierarchy(int projectId) {
1586:             List<Task> all = db.getTasks();
1587:             List<Task> projectTasks = new ArrayList<>();
1588:             Map<Integer, Task> taskMap = new HashMap<>();
1589:
1590:             for (Task t : all) {
1591:                 if (t.getProjectId() == projectId) {
1592:                     t.clearSubTasks();
1593:                     projectTasks.add(t);
```

File: ProjectManagementSystem.java | Package: (default) (continued)

```
1594:                     taskMap.put(t.getId(), t);
1595:                 }
1596:             }
1597:
1598:             List<Task> roots = new ArrayList<>();
1599:             for (Task t : projectTasks) {
1600:                 Integer pid = t.getParentTaskId();
1601:                 if (pid == null || pid <= 0 || !taskMap.containsKey(pid) || pid.equals(t.getId()))
1602:                     // Top-level task
1603:                     roots.add(t);
1604:                 } else {
1605:                     // Child task
1606:                     Task parent = taskMap.get(pid);
1607:                     parent.addSubTask(t);
1608:                 }
1609:             }
1610:             return roots;
1611:         }
1612:     }
1613:
1614:     public static class TaskService {
1615:         private final DatabaseManager db;
1616:         public TaskService(DatabaseManager db) { this.db = db; }
1617:
1618:         public Task saveTask(Task t, User initiator) throws UnauthorizedAccessException {
1619:             boolean isEdit = t.getId() != null;
1620:             Permission perm = isEdit ? Permission.EDIT_TASK : Permission.CREATE_TASK;
1621:             if (initiator == null || !initiator.hasPermission(perm)) {
1622:                 throw new UnauthorizedAccessException(initiator != null ? initiator.getUsername()
1623:             }
1624:             return db.saveTask(t);
1625:         }
1626:
1627:         public void updateTaskStatus(int taskId, TaskStatus newStatus, User initiator)
1628:                 throws UnauthorizedAccessException, InvalidTaskStateException, TaskNotFoundExcepti
1629:             if (initiator == null || !initiator.hasPermission(Permission.UPDATE_TASK_STATUS)) {
1630:                 throw new UnauthorizedAccessException(initiator != null ? initiator.getUsername()
1631:             }
1632:             Task task = db.getTasks().stream().filter(t -> t.getId() == taskId).findFirst().orElse
1633:             if (!task.getStatus().canTransitionTo(newStatus)) {
1634:                 throw new InvalidTaskStateException(task.getStatus().getLabel(), newStatus.getLabe
1635:             }
1636:             task.setStatus(newStatus);
1637:             db.saveTask(task);
1638:         }
1639:
1640:         public void logHours(int taskId, double hours, User initiator) throws UnauthorizedAccessEx
1641:             if (initiator == null || !initiator.hasPermission(Permission.LOG_HOURS)) {
1642:                 throw new UnauthorizedAccessException(initiator != null ? initiator.getUsername()
1643:             }
1644:             Task task = db.getTasks().stream().filter(t -> t.getId() == taskId).findFirst().orElse
1645:             task.logHours(hours);
1646:             db.saveTask(task);
1647:             Project project = db.getProjects().stream().filter(p -> p.getId() == task.getProjectId
1648:             db.recordTaskWorkLog(task, project, initiator, hours, "Engineering sprint task executi
1649:         }
1650:
1651:         public void deleteTask(int taskId, User initiator) throws UnauthorizedAccessException, Tas
1652:             if (initiator == null || !initiator.hasPermission(Permission.DELETE_TASK)) {
```

File: ProjectManagementSystem.java | Package: (default) (continued)

```
1653:                 throw new UnauthorizedAccessException(initiator != null ? initiator.getUsername()
1654:             }
1655:             boolean removed = db.deleteTask(taskId);
1656:             if (!removed) throw new TaskNotFoundException(taskId);
1657:         }
1658:
1659:         public List<Task> getAllTasks() { return db.getTasks(); }
1660:     }
1661:
1662:     // ================================================================================
1663:     // MODULE 4: ADAPTER DESIGN PATTERNS FOR JTABLE
1664:     // ================================================================================
1665:
1666:     public static class ProjectTableModelAdapter extends AbstractTableModel {
1667:         private final String[] columns = {"ID", "Project Title", "Client Name", "Budget ($)", "Sta
1668:         private final List<Project> projects = new ArrayList<>();
1669:
1670:         public void setProjects(List<Project> list) {
1671:             projects.clear();
1672:             if (list != null) projects.addAll(list);
1673:             fireTableDataChanged();
1674:         }
1675:         public Project getProjectAt(int row) { return (row >= 0 && row < projects.size()) ? projec
1676:         @Override public int getRowCount() { return projects.size(); }
1677:         @Override public int getColumnCount() { return columns.length; }
1678:         @Override public String getColumnName(int col) { return columns[col]; }
1679:         @Override public Object getValueAt(int row, int col) {
1680:             Project p = projects.get(row);
1681:             switch (col) {
1682:                 case 0: return p.getId();
1683:                 case 1: return p.getTitle();
1684:                 case 2: return p.getClientName();
1685:                 case 3: return String.format("$%,.2f", p.getBudget());
1686:                 case 4: return p.getStatus();
1687:                 case 5: return p.getTagsFormatted();
1688:                 default: return null;
1689:             }
1690:         }
1691:     }
1692:
1693:     public static class TaskTableModelAdapter extends AbstractTableModel {
1694:         private final String[] columns = {"ID", "Task Title", "Project", "Assigned Team", "Priorit
1695:         private final List<Task> tasks = new ArrayList<>();
1696:
1697:         public void setTasks(List<Task> list) {
1698:             tasks.clear();
1699:             if (list != null) tasks.addAll(list);
1700:             fireTableDataChanged();
1701:         }
1702:         public Task getTaskAt(int row) { return (row >= 0 && row < tasks.size()) ? tasks.get(row)
1703:         @Override public int getRowCount() { return tasks.size(); }
1704:         @Override public int getColumnCount() { return columns.length; }
1705:         @Override public String getColumnName(int col) { return columns[col]; }
1706:         @Override public Object getValueAt(int row, int col) {
1707:             Task t = tasks.get(row);
1708:             switch (col) {
1709:                 case 0: return t.getId();
1710:                 case 1: return t.getTitle();
1711:                 case 2: return "Project #" + t.getProjectId();
```

File: ProjectManagementSystem.java | Package: (default) (continued)

```
1712:                 case 3: return t.getAssignedTeam();
1713:                 case 4: return t.getPriority();
1714:                 case 5: return t.getStatus();
1715:                 case 6: return t.getEstimatedHours() + "h";
1716:                 case 7: return t.getLoggedHours() + "h";
1717:                 case 8: return t.calculateCompletionPercentage();
1718:                 default: return null;
1719:             }
1720:         }
1721:     }
1722:
1723:     // ================================================================================
1724:     // UI THEME, PALETTE & CUSTOM RENDERERS
1725:     // ================================================================================
1726:
1727:     public static final class UITheme {
1728:         public static final Color PRIMARY = new Color(30, 64, 175);       // Deep Enterprise Royal
1729:         public static final Color PRIMARY_ACCENT = new Color(59, 130, 246);
1730:         public static final Color SUCCESS = new Color(16, 185, 129);       // Emerald Green
1731:         public static final Color WARNING = new Color(245, 158, 11);       // Amber
1732:         public static final Color DANGER = new Color(220, 38, 38);         // Crimson Red
1733:         public static final Color BG_LIGHT = new Color(241, 245, 249);     // Slate 100
1734:         public static final Color CARD_BG = Color.WHITE;
1735:         public static final Color BORDER = new Color(203, 213, 225);       // Slate 300
1736:         public static final Color TEXT_DARK = new Color(15, 23, 42);       // Slate 900
1737:         public static final Color TEXT_MUTED = new Color(100, 116, 139);   // Slate 500
1738:
1739:         public static final Font FONT_HEADER = new Font("Segoe UI", Font.BOLD, 19);
1740:         public static final Font FONT_SECTION = new Font("Segoe UI", Font.BOLD, 15);
1741:         public static final Font FONT_BOLD = new Font("Segoe UI", Font.BOLD, 13);
1742:         public static final Font FONT_REGULAR = new Font("Segoe UI", Font.PLAIN, 13);
1743:         public static final Font FONT_SMALL = new Font("Segoe UI", Font.PLAIN, 11);
1744:         public static final Font FONT_BADGE = new Font("Segoe UI", Font.BOLD, 11);
1745:
1746:         public static JPanel createCard() {
1747:             JPanel p = new JPanel();
1748:             p.setBackground(CARD_BG);
1749:             p.setBorder(BorderFactory.createCompoundBorder(
1750:                     BorderFactory.createLineBorder(BORDER, 1),
1751:                     new EmptyBorder(14, 16, 14, 16)
1752:             ));
1753:             return p;
1754:         }
1755:
1756:         public static void styleButton(JButton btn, Color bg, Color fg, Color border, Color hoverB
1757:             btn.setUI(new javax.swing.plaf.basic.BasicButtonUI());
1758:             btn.setFont(FONT_BOLD);
1759:             btn.setBackground(bg);
1760:             btn.setForeground(fg);
1761:             btn.setOpaque(true);
1762:             btn.setContentAreaFilled(true);
1763:             btn.setFocusPainted(false);
1764:             btn.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
1765:             btn.setBorder(BorderFactory.createCompoundBorder(
1766:                     BorderFactory.createLineBorder(border, 1),
1767:                     BorderFactory.createEmptyBorder(8, 16, 8, 16)
1768:             ));
1769:             btn.addMouseListener(new MouseAdapter() {
1770:                 @Override public void mouseEntered(MouseEvent e) {
```

File: ProjectManagementSystem.java | Package: (default) (continued)

```
1771:                     if (btn.isEnabled()) btn.setBackground(hoverBg);
1772:                 }
1773:                 @Override public void mouseExited(MouseEvent e) {
1774:                     btn.setBackground(bg);
1775:                 }
1776:             });
1777:         }
1778:
1779:         public static JButton createPrimaryButton(String text) {
1780:             JButton btn = new JButton(text);
1781:             styleButton(btn, new Color(37, 99, 235), Color.WHITE, new Color(29, 78, 216), new Colo
1782:             return btn;
1783:         }
1784:
1785:         public static JButton createSecondaryButton(String text) {
1786:             JButton btn = new JButton(text);
1787:             styleButton(btn, new Color(241, 245, 249), new Color(15, 23, 42), new Color(148, 163,
1788:             return btn;
1789:         }
1790:
1791:         public static JButton createDangerButton(String text) {
1792:             JButton btn = new JButton(text);
1793:             styleButton(btn, new Color(220, 38, 38), Color.WHITE, new Color(185, 28, 28), new Colo
1794:             return btn;
1795:         }
1796:
1797:         public static JButton createSuccessButton(String text) {
1798:             JButton btn = new JButton(text);
1799:             styleButton(btn, new Color(16, 185, 129), Color.WHITE, new Color(5, 150, 105), new Col
1800:             return btn;
1801:         }
1802:
1803:         public static DefaultTableCellRenderer createStatusRenderer() {
1804:             return new DefaultTableCellRenderer() {
1805:                 @Override public Component getTableCellRendererComponent(JTable table, Object val,
1806:                     JLabel lbl = (JLabel) super.getTableCellRendererComponent(table, val, isSel, h
1807:                     lbl.setHorizontalAlignment(SwingConstants.CENTER);
1808:                     lbl.setFont(FONT_BADGE);
1809:                     lbl.setOpaque(true);
1810:
1811:                     if (val instanceof TaskStatus) {
1812:                         TaskStatus s = (TaskStatus) val;
1813:                         switch (s) {
1814:                             case COMPLETED:   lbl.setBackground(new Color(209, 250, 229)); lbl.set
1815:                             case REVIEW:      lbl.setBackground(new Color(243, 232, 255)); lbl.set
1816:                             case IN_PROGRESS: lbl.setBackground(new Color(254, 243, 199)); lbl.set
1817:                             default:          lbl.setBackground(new Color(224, 231, 255)); lbl.set
1818:                         }
1819:                     } else if (val instanceof ProjectStatus) {
1820:                         ProjectStatus s = (ProjectStatus) val;
1821:                         switch (s) {
1822:                             case COMPLETED:   lbl.setBackground(new Color(209, 250, 229)); lbl.set
1823:                             case IN_PROGRESS: lbl.setBackground(new Color(224, 242, 254)); lbl.set
1824:                             case ON_HOLD:     lbl.setBackground(new Color(254, 243, 199)); lbl.set
1825:                             case CANCELLED:   lbl.setBackground(new Color(254, 226, 226)); lbl.set
1826:                             default:          lbl.setBackground(new Color(241, 245, 249)); lbl.set
1827:                         }
1828:                     }
1829:                     return lbl;
```

File: ProjectManagementSystem.java | Package: (default) (continued)

```
1830:                 }
1831:             };
1832:         }
1833:
1834:         public static DefaultTableCellRenderer createProgressRenderer() {
1835:             final JProgressBar pb = new JProgressBar(0, 100);
1836:             pb.setStringPainted(true);
1837:             pb.setFont(FONT_SMALL);
1838:             return new DefaultTableCellRenderer() {
1839:                 @Override public Component getTableCellRendererComponent(JTable table, Object val,
1840:                     if (val instanceof Number) {
1841:                         int v = ((Number) val).intValue();
1842:                         pb.setValue(v);
1843:                         pb.setString(v + "%");
1844:                         pb.setForeground(v >= 100 ? SUCCESS : (v >= 50 ? PRIMARY_ACCENT : WARNING)
1845:                         return pb;
1846:                     }
1847:                     return super.getTableCellRendererComponent(table, val, isSel, hasFocus, r, c);
1848:                 }
1849:             };
1850:         }
1851:     }
1852:
1853:     // ================================================================================
1854:     // COMPREHENSIVE MODAL DIALOGS (ALL FIELDS ENTERABLE)
1855:     // ================================================================================
1856:
1857:     /**
1858:      * Comprehensive Project Dialog with all fields enterable.
1859:      */
1860:     public static class ProjectFormDialog extends JDialog {
1861:         private boolean saved = false;
1862:         private final Project project;
1863:
1864:         private JTextField txtTitle = new JTextField(22);
1865:         private JTextField txtClient = new JTextField(22);
1866:         private JTextField txtBudget = new JTextField(22);
1867:         private JComboBox<ProjectStatus> cmbStatus = new JComboBox<>(ProjectStatus.values());
1868:         private JTextField txtTags = new JTextField(22);
1869:         private JTextArea txtDesc = new JTextArea(4, 22);
1870:
1871:         public ProjectFormDialog(Frame parent, Project projectToEdit) {
1872:             super(parent, projectToEdit == null ? "Create New Project" : "Edit Project #" + projec
1873:             this.project = projectToEdit;
1874:
1875:             initComponents();
1876:             if (project != null) {
1877:                 txtTitle.setText(project.getTitle());
1878:                 txtClient.setText(project.getClientName());
1879:                 txtBudget.setText(String.valueOf(project.getBudget()));
1880:                 cmbStatus.setSelectedItem(project.getStatus());
1881:                 txtTags.setText(project.getTagsFormatted());
1882:                 txtDesc.setText(project.getDescription());
1883:             } else {
1884:                 txtBudget.setText("50000.0");
1885:                 txtClient.setText("Internal Directorate");
1886:             }
1887:             pack();
1888:             setLocationRelativeTo(parent);
```

File: ProjectManagementSystem.java | Package: (default) (continued)

```
1889:             setResizable(false);
1890:         }
1891:
1892:         private void initComponents() {
1893:             setLayout(new BorderLayout());
1894:             getContentPane().setBackground(UITheme.BG_LIGHT);
1895:
1896:             JPanel header = new JPanel(new FlowLayout(FlowLayout.LEFT, 15, 12));
1897:             header.setBackground(UITheme.PRIMARY);
1898:             JLabel lblH = new JLabel(project == null ? "Enter New Project Details" : "Update Proje
1899:             lblH.setFont(UITheme.FONT_SECTION);
1900:             lblH.setForeground(Color.WHITE);
1901:             header.add(lblH);
1902:             add(header, BorderLayout.NORTH);
1903:
1904:             JPanel form = new JPanel(new GridBagLayout());
1905:             form.setBackground(Color.WHITE);
1906:             form.setBorder(BorderFactory.createEmptyBorder(15, 20, 15, 20));
1907:
1908:             GridBagConstraints g = new GridBagConstraints();
1909:             g.insets = new Insets(5, 5, 5, 5);
1910:             g.fill = GridBagConstraints.HORIZONTAL;
1911:
1912:             int row = 0;
1913:             addRow(form, g, row++, "Project Title: *", txtTitle);
1914:             addRow(form, g, row++, "Client / Sponsor Name:", txtClient);
1915:             addRow(form, g, row++, "Total Budget ($): *", txtBudget);
1916:             addRow(form, g, row++, "Lifecycle Status:", cmbStatus);
1917:             addRow(form, g, row++, "Domain Tags (comma separated):", txtTags);
1918:
1919:             g.gridx = 0; g.gridy = row; g.anchor = GridBagConstraints.NORTHWEST;
1920:             form.add(new JLabel("Description & Objectives:"), g);
1921:             g.gridx = 1;
1922:             txtDesc.setLineWrap(true);
1923:             txtDesc.setWrapStyleWord(true);
1924:             form.add(new JScrollPane(txtDesc), g);
1925:
1926:             add(form, BorderLayout.CENTER);
1927:
1928:             JPanel footer = new JPanel(new FlowLayout(FlowLayout.RIGHT, 10, 10));
1929:             footer.setBackground(UITheme.BG_LIGHT);
1930:
1931:             JButton btnCancel = UITheme.createSecondaryButton("Cancel");
1932:             btnCancel.addActionListener(e -> dispose());
1933:
1934:             JButton btnSave = UITheme.createPrimaryButton("Save Project");
1935:             btnSave.addActionListener(e -> {
1936:                 String title = txtTitle.getText().trim();
1937:                 if (title.isEmpty()) {
1938:                     JOptionPane.showMessageDialog(this, "Project Title is required.", "Validation
1939:                     return;
1940:                 }
1941:                 double budget = 0.0;
1942:                 try {
1943:                     budget = Double.parseDouble(txtBudget.getText().trim());
1944:                     if (budget < 0) throw new NumberFormatException();
1945:                 } catch (NumberFormatException ex) {
1946:                     JOptionPane.showMessageDialog(this, "Please enter a valid numeric budget.", "V
1947:                     return;
```

File: ProjectManagementSystem.java | Package: (default) (continued)

```
1948:                 }
1949:
1950:                 String[] tags = txtTags.getText().trim().isEmpty() ? new String[0] : txtTags.getTe
1951:                 if (project == null) {
1952:                     createdProject = new Project(null, title, txtDesc.getText().trim(), budget, (P
1953:                 } else {
1954:                     project.setTitle(title);
1955:                     project.setClientName(txtClient.getText().trim());
1956:                     project.setBudget(budget);
1957:                     project.setStatus((ProjectStatus) cmbStatus.getSelectedItem());
1958:                     project.setTags(tags);
1959:                     project.setDescription(txtDesc.getText().trim());
1960:                 }
1961:                 saved = true;
1962:                 dispose();
1963:             });
1964:
1965:             footer.add(btnCancel);
1966:             footer.add(btnSave);
1967:             add(footer, BorderLayout.SOUTH);
1968:         }
1969:
1970:         private void addRow(JPanel p, GridBagConstraints g, int row, String label, JComponent comp
1971:             g.gridx = 0; g.gridy = row; g.anchor = GridBagConstraints.WEST;
1972:             p.add(new JLabel(label), g);
1973:             g.gridx = 1;
1974:             p.add(comp, g);
1975:         }
1976:
1977:         private Project createdProject;
1978:         public boolean isSaved() { return saved; }
1979:         public Project getProject() { return project != null ? project : createdProject; }
1980:     }
1981:
1982:     /**
1983:      * Comprehensive Task & Sub-task Dialog with all fields enterable.
1984:      */
1985:     public static class TaskFormDialog extends JDialog {
1986:         private boolean saved = false;
1987:         private final Task task;
1988:
1989:         private JComboBox<ProjectItem> cmbProj = new JComboBox<>();
1990:         private JComboBox<TaskItem> cmbParent = new JComboBox<>();
1991:         private JTextField txtTitle = new JTextField(22);
1992:         private JComboBox<String> cmbTeam = new JComboBox<>();
1993:         private JComboBox<UserItem> cmbUser = new JComboBox<>();
1994:         private JComboBox<TaskPriority> cmbPriority = new JComboBox<>(TaskPriority.values());
1995:         private JComboBox<TaskStatus> cmbStatus = new JComboBox<>(TaskStatus.values());
1996:         private JTextField txtEstHours = new JTextField("10.0", 22);
1997:         private JTextField txtLoggedHours = new JTextField("0.0", 22);
1998:         private JTextArea txtDesc = new JTextArea(4, 22);
1999:
2000:         public TaskFormDialog(Frame parent, Task taskToEdit, List<Project> projects, List<Task> ta
2001:             super(parent, taskToEdit == null ? "Create New Task / Sub-task" : "Edit Task #" + task
2002:             this.task = taskToEdit;
2003:
2004:             for (Project p : projects) cmbProj.addItem(new ProjectItem(p));
2005:             cmbParent.addItem(new TaskItem(null, "[None] Root / Top-Level Task"));
2006:             for (Task t : tasks) {
```

File: ProjectManagementSystem.java | Package: (default) (continued)

```
2007:                 if (taskToEdit == null || !t.getId().equals(taskToEdit.getId())) {
2008:                     cmbParent.addItem(new TaskItem(t.getId(), "#" + t.getId() + " | " + t.getTitle
2009:                 }
2010:             }
2011:             cmbUser.addItem(new UserItem(null, "[Unassigned]"));
2012:             for (User u : users) cmbUser.addItem(new UserItem(u.getId(), u.getFullName() + " (" +
2013:
2014:             cmbTeam.addItem("General Team");
2015:             for (Team tm : DatabaseManager.getInstance().getTeams()) {
2016:                 if (!"General Team".equalsIgnoreCase(tm.getName())) cmbTeam.addItem(tm.getName());
2017:             }
2018:             if (taskToEdit != null && taskToEdit.getAssignedTeam() != null) {
2019:                 cmbTeam.setSelectedItem(taskToEdit.getAssignedTeam());
2020:             }
2021:
2022:             // Project default
2023:             if (defaultProjectId != null) {
2024:                 for (int i = 0; i < cmbProj.getItemCount(); i++) {
2025:                     if (cmbProj.getItemAt(i).p.getId().equals(defaultProjectId)) {
2026:                         cmbProj.setSelectedIndex(i);
2027:                         break;
2028:                     }
2029:                 }
2030:             }
2031:
2032:             initComponents();
2033:             if (task != null) {
2034:                 txtTitle.setText(task.getTitle());
2035:                 txtDesc.setText(task.getDescription());
2036:                 cmbPriority.setSelectedItem(task.getPriority());
2037:                 cmbStatus.setSelectedItem(task.getStatus());
2038:                 txtEstHours.setText(String.valueOf(task.getEstimatedHours()));
2039:                 txtLoggedHours.setText(String.valueOf(task.getLoggedHours()));
2040:
2041:                 // Select project
2042:                 for (int i = 0; i < cmbProj.getItemCount(); i++) {
2043:                     if (cmbProj.getItemAt(i).p.getId().equals(task.getProjectId())) {
2044:                         cmbProj.setSelectedIndex(i);
2045:                         break;
2046:                     }
2047:                 }
2048:                 // Select parent
2049:                 if (task.getParentTaskId() != null) {
2050:                     for (int i = 0; i < cmbParent.getItemCount(); i++) {
2051:                         if (task.getParentTaskId().equals(cmbParent.getItemAt(i).id)) {
2052:                             cmbParent.setSelectedIndex(i);
2053:                             break;
2054:                         }
2055:                     }
2056:                 }
2057:                 // Select user
2058:                 if (task.getAssignedUserId() != null) {
2059:                     for (int i = 0; i < cmbUser.getItemCount(); i++) {
2060:                         if (task.getAssignedUserId().equals(cmbUser.getItemAt(i).id)) {
2061:                             cmbUser.setSelectedIndex(i);
2062:                             break;
2063:                         }
2064:                     }
2065:                 }
```

File: ProjectManagementSystem.java | Package: (default) (continued)

```
2066:             }
2067:
2068:             pack();
2069:             setLocationRelativeTo(parent);
2070:             setResizable(false);
2071:         }
2072:
2073:         private void initComponents() {
2074:             setLayout(new BorderLayout());
2075:             getContentPane().setBackground(UITheme.BG_LIGHT);
2076:
2077:             JPanel header = new JPanel(new FlowLayout(FlowLayout.LEFT, 15, 12));
2078:             header.setBackground(UITheme.PRIMARY);
2079:             JLabel lblH = new JLabel(task == null ? "Create New Task / Sub-task" : "Update Task #"
2080:             lblH.setFont(UITheme.FONT_SECTION);
2081:             lblH.setForeground(Color.WHITE);
2082:             header.add(lblH);
2083:             add(header, BorderLayout.NORTH);
2084:
2085:             JPanel form = new JPanel(new GridBagLayout());
2086:             form.setBackground(Color.WHITE);
2087:             form.setBorder(BorderFactory.createEmptyBorder(15, 20, 15, 20));
2088:
2089:             GridBagConstraints g = new GridBagConstraints();
2090:             g.insets = new Insets(5, 5, 5, 5);
2091:             g.fill = GridBagConstraints.HORIZONTAL;
2092:
2093:             int row = 0;
2094:             addRow(form, g, row++, "Target Project: *", cmbProj);
2095:             addRow(form, g, row++, "Parent Task (For Nested Sub-tasks):", cmbParent);
2096:             addRow(form, g, row++, "Task Title: *", txtTitle);
2097:             addRow(form, g, row++, "Assigned Team Member:", cmbUser);
2098:             addRow(form, g, row++, "Priority Level:", cmbPriority);
2099:             addRow(form, g, row++, "Execution Status:", cmbStatus);
2100:             addRow(form, g, row++, "Estimated Hours (Est):", txtEstHours);
2101:             if (task != null) addRow(form, g, row++, "Logged Hours (Actual):", txtLoggedHours);
2102:
2103:             g.gridx = 0; g.gridy = row; g.anchor = GridBagConstraints.NORTHWEST;
2104:             form.add(new JLabel("Task Description & Scope:"), g);
2105:             g.gridx = 1;
2106:             txtDesc.setLineWrap(true);
2107:             txtDesc.setWrapStyleWord(true);
2108:             form.add(new JScrollPane(txtDesc), g);
2109:
2110:             add(form, BorderLayout.CENTER);
2111:
2112:             JPanel footer = new JPanel(new FlowLayout(FlowLayout.RIGHT, 10, 10));
2113:             footer.setBackground(UITheme.BG_LIGHT);
2114:
2115:             JButton btnCancel = UITheme.createSecondaryButton("Cancel");
2116:             btnCancel.addActionListener(e -> dispose());
2117:
2118:             JButton btnSave = UITheme.createPrimaryButton("Save Task");
2119:             btnSave.addActionListener(e -> {
2120:                 String title = txtTitle.getText().trim();
2121:                 if (title.isEmpty()) {
2122:                     JOptionPane.showMessageDialog(this, "Task Title is required.", "Validation Err
2123:                     return;
2124:                 }
```

File: ProjectManagementSystem.java | Package: (default) (continued)

```
2125:                 double est = 0.0, logged = 0.0;
2126:                 try {
2127:                     est = Double.parseDouble(txtEstHours.getText().trim());
2128:                     if (est < 0) throw new NumberFormatException();
2129:                     if (task != null) logged = Double.parseDouble(txtLoggedHours.getText().trim())
2130:                 } catch (NumberFormatException ex) {
2131:                     JOptionPane.showMessageDialog(this, "Please enter valid numeric hours.", "Vali
2132:                     return;
2133:                 }
2134:
2135:                 ProjectItem projItem = (ProjectItem) cmbProj.getSelectedItem();
2136:                 int pid = projItem != null ? projItem.p.getId() : 1;
2137:                 TaskItem parentItem = (TaskItem) cmbParent.getSelectedItem();
2138:                 Integer parentId = parentItem != null ? parentItem.id : null;
2139:                 UserItem userItem = (UserItem) cmbUser.getSelectedItem();
2140:                 Integer uid = userItem != null ? userItem.id : null;
2141:
2142:                 String selectedTeam = (String) cmbTeam.getSelectedItem();
2143:                 if (selectedTeam == null || selectedTeam.trim().isEmpty()) selectedTeam = "General
2144:                 if (task == null) {
2145:                     createdTask = new Task.Builder(pid, title)
2146:                             .description(txtDesc.getText().trim())
2147:                             .parentTask(parentId)
2148:                             .assignedUser(uid)
2149:                             .assignedTeam(selectedTeam)
2150:                             .priority((TaskPriority) cmbPriority.getSelectedItem())
2151:                             .status((TaskStatus) cmbStatus.getSelectedItem())
2152:                             .estimatedHours(est)
2153:                             .loggedHours(0.0)
2154:                             .build();
2155:                 } else {
2156:                     task.setProjectId(pid);
2157:                     task.setParentTaskId(parentId);
2158:                     task.setTitle(title);
2159:                     task.setDescription(txtDesc.getText().trim());
2160:                     task.setAssignedUserId(uid);
2161:                     task.setAssignedTeam(selectedTeam);
2162:                     task.setPriority((TaskPriority) cmbPriority.getSelectedItem());
2163:                     task.setStatus((TaskStatus) cmbStatus.getSelectedItem());
2164:                     task.setEstimatedHours(est);
2165:                     task.setLoggedHours(logged);
2166:                 }
2167:                 saved = true;
2168:                 dispose();
2169:             });
2170:
2171:             footer.add(btnCancel);
2172:             footer.add(btnSave);
2173:             add(footer, BorderLayout.SOUTH);
2174:         }
2175:
2176:         private void addRow(JPanel p, GridBagConstraints g, int row, String label, JComponent comp
2177:             g.gridx = 0; g.gridy = row; g.anchor = GridBagConstraints.WEST;
2178:             p.add(new JLabel(label), g);
2179:             g.gridx = 1;
2180:             p.add(comp, g);
2181:         }
2182:
2183:         private Task createdTask;
```

File: ProjectManagementSystem.java | Package: (default) (continued)

```
2184:         public boolean isSaved() { return saved; }
2185:         public Task getTask() { return task != null ? task : createdTask; }
2186:
2187:         private static class ProjectItem {
2188:             final Project p;
2189:             ProjectItem(Project p) { this.p = p; }
2190:             @Override public String toString() { return "#" + p.getId() + " | " + p.getTitle(); }
2191:         }
2192:         private static class TaskItem {
2193:             final Integer id;
2194:             final String text;
2195:             TaskItem(Integer id, String text) { this.id = id; this.text = text; }
2196:             @Override public String toString() { return text; }
2197:         }
2198:         private static class UserItem {
2199:             final Integer id;
2200:             final String text;
2201:             UserItem(Integer id, String text) { this.id = id; this.text = text; }
2202:             @Override public String toString() { return text; }
2203:         }
2204:     }
2205:
2206:     /**
2207:      * Modal Dialog to Add New Team Members (Manager / Admin Only).
2208:      */
2209:
2210:     public static class TeamFormDialog extends JDialog {
2211:         private boolean saved = false;
2212:         private String teamName;
2213:         private String teamDescription;
2214:         private JTextField txtName = new JTextField(20);
2215:         private JTextArea txtDesc = new JTextArea(4, 20);
2216:
2217:         public TeamFormDialog(Frame parent, DatabaseManager db) {
2218:             super(parent, "Create New Functional Team", true);
2219:             setLayout(new BorderLayout());
2220:             getContentPane().setBackground(UITheme.BG_LIGHT);
2221:
2222:             JPanel header = new JPanel(new FlowLayout(FlowLayout.LEFT, 15, 12));
2223:             header.setBackground(UITheme.PRIMARY);
2224:             JLabel lblH = new JLabel("Add New Engineering / Business Team");
2225:             lblH.setFont(UITheme.FONT_SECTION);
2226:             lblH.setForeground(Color.WHITE);
2227:             header.add(lblH);
2228:             add(header, BorderLayout.NORTH);
2229:
2230:             JPanel form = new JPanel(new GridBagLayout());
2231:             form.setBackground(Color.WHITE);
2232:             form.setBorder(BorderFactory.createEmptyBorder(15, 20, 15, 20));
2233:             GridBagConstraints g = new GridBagConstraints();
2234:             g.insets = new Insets(6, 6, 6, 6);
2235:             g.fill = GridBagConstraints.HORIZONTAL;
2236:
2237:             g.gridx = 0; g.gridy = 0; form.add(new JLabel("Team Name: *"), g);
2238:             g.gridx = 1; form.add(txtName, g);
2239:
2240:             g.gridx = 0; g.gridy = 1; form.add(new JLabel("Description: *"), g);
2241:             g.gridx = 1; form.add(new JScrollPane(txtDesc), g);
2242:
```

File: ProjectManagementSystem.java | Package: (default) (continued)

```
2243:             add(form, BorderLayout.CENTER);
2244:
2245:             JPanel footer = new JPanel(new FlowLayout(FlowLayout.RIGHT, 10, 10));
2246:             footer.setBackground(UITheme.BG_LIGHT);
2247:             JButton btnCancel = UITheme.createSecondaryButton("Cancel");
2248:             btnCancel.addActionListener(e -> dispose());
2249:             JButton btnSave = UITheme.createPrimaryButton("Create Team");
2250:             btnSave.addActionListener(e -> {
2251:                 String nm = txtName.getText().trim();
2252:                 String ds = txtDesc.getText().trim();
2253:                 if (nm.isEmpty() || ds.isEmpty()) {
2254:                     JOptionPane.showMessageDialog(this, "Please enter both a team name and descrip
2255:                     return;
2256:                 }
2257:                 for (Team tm : db.getTeams()) {
2258:                     if (tm.getName().equalsIgnoreCase(nm)) {
2259:                         JOptionPane.showMessageDialog(this, "Team '" + nm + "' already exists!", "
2260:                         return;
2261:                     }
2262:                 }
2263:                 teamName = nm;
2264:                 teamDescription = ds;
2265:                 saved = true;
2266:                 dispose();
2267:             });
2268:             footer.add(btnCancel); footer.add(btnSave);
2269:             add(footer, BorderLayout.SOUTH);
2270:             pack();
2271:             setLocationRelativeTo(parent);
2272:             setResizable(false);
2273:         }
2274:         public boolean isSaved() { return saved; }
2275:         public String getTeamName() { return teamName; }
2276:         public String getTeamDescription() { return teamDescription; }
2277:     }
2278:
2279:     public static class AddTeamMemberDialog extends JDialog {
2280:         private boolean saved = false;
2281:         private User createdUser;
2282:         private final User userToEdit;
2283:
2284:         private JComboBox<String> cmbRole = new JComboBox<>();
2285:         private JComboBox<String> cmbDepartment = new JComboBox<>();
2286:         private JTextField txtFullName = new JTextField(20);
2287:         private JTextField txtUsername = new JTextField(20);
2288:         private JPasswordField txtPassword = new JPasswordField(20);
2289:
2290:         private final User currentUser;
2291:
2292:         public AddTeamMemberDialog(Frame parent, DatabaseManager db, User currentUser) {
2293:             this(parent, db, currentUser, null);
2294:         }
2295:
2296:         public AddTeamMemberDialog(Frame parent, DatabaseManager db, User currentUser, User userTo
2297:             super(parent, userToEdit != null ? ("Edit Account: " + userToEdit.getFullName())
2298:                     : ((currentUser != null && currentUser.hasPermission(Permission.MANAGE_ADMINS)
2299:             this.currentUser = currentUser;
2300:             this.userToEdit = userToEdit;
2301:             setLayout(new BorderLayout());
```

File: ProjectManagementSystem.java | Package: (default) (continued)

```
2302:             getContentPane().setBackground(UITheme.BG_LIGHT);
2303:
2304:             JPanel header = new JPanel(new FlowLayout(FlowLayout.LEFT, 15, 12));
2305:             header.setBackground(UITheme.PRIMARY);
2306:             JLabel lblH = new JLabel(userToEdit != null ? "Edit User Account Profile" : "Register
2307:             lblH.setFont(UITheme.FONT_SECTION);
2308:             lblH.setForeground(Color.WHITE);
2309:             header.add(lblH);
2310:             add(header, BorderLayout.NORTH);
2311:
2312:             JPanel form = new JPanel(new GridBagLayout());
2313:             form.setBackground(Color.WHITE);
2314:             form.setBorder(BorderFactory.createEmptyBorder(18, 20, 18, 20));
2315:
2316:             GridBagConstraints g = new GridBagConstraints();
2317:             g.insets = new Insets(6, 6, 6, 6);
2318:             g.fill = GridBagConstraints.HORIZONTAL;
2319:
2320:             if (userToEdit != null && userToEdit.getRole() == UserRole.SUPER_ADMIN) {
2321:                 cmbRole.addItem("Super Administrator");
2322:                 cmbRole.setEnabled(false);
2323:             } else if (currentUser != null && currentUser.hasPermission(Permission.MANAGE_ADMINS))
2324:                 cmbRole.addItem("Software Developer / User");
2325:                 cmbRole.addItem("Project Manager / Admin");
2326:             } else {
2327:                 cmbRole.addItem("Software Developer / User");
2328:             }
2329:
2330:             for (Team tm : db.getTeams()) cmbDepartment.addItem(tm.getName());
2331:             if (cmbDepartment.getItemCount() == 0) cmbDepartment.addItem("General Team");
2332:             cmbDepartment.setEditable(true);
2333:
2334:             int row = 0;
2335:             addRow(form, g, row++, "Full Name: *", txtFullName);
2336:             addRow(form, g, row++, "Unique Username: *", txtUsername);
2337:             addRow(form, g, row++, "Password (min 8 chars): *", txtPassword);
2338:             addRow(form, g, row++, "Account Role: *", cmbRole);
2339:             addRow(form, g, row++, "Department / Team: *", cmbDepartment);
2340:
2341:             // Pre-populate if editing an existing user
2342:             if (userToEdit != null) {
2343:                 txtFullName.setText(userToEdit.getFullName());
2344:                 txtUsername.setText(userToEdit.getUsername());
2345:                 txtPassword.setText(userToEdit.getPassword());
2346:                 cmbDepartment.setSelectedItem(userToEdit.getDepartment());
2347:
2348:                 if (userToEdit instanceof SuperAdmin) {
2349:                     cmbRole.setSelectedItem("Super Administrator");
2350:                 } else if (userToEdit instanceof Manager) {
2351:                     cmbRole.setSelectedItem("Project Manager / Admin");
2352:                 } else if (userToEdit instanceof Developer) {
2353:                     cmbRole.setSelectedItem("Software Developer / User");
2354:                 }
2355:             }
2356:
2357:             add(form, BorderLayout.CENTER);
2358:
2359:             JPanel footer = new JPanel(new FlowLayout(FlowLayout.RIGHT, 10, 10));
2360:             footer.setBackground(UITheme.BG_LIGHT);
```

File: ProjectManagementSystem.java | Package: (default) (continued)

```
2361:
2362:             JButton btnCancel = UITheme.createSecondaryButton("Cancel");
2363:             btnCancel.addActionListener(e -> dispose());
2364:
2365:             JButton btnSave = UITheme.createPrimaryButton(userToEdit != null ? "Save Changes" : "A
2366:             btnSave.addActionListener(e -> {
2367:                 String fn = txtFullName.getText().trim();
2368:                 String un = txtUsername.getText().trim();
2369:                 String pw = new String(txtPassword.getPassword());
2370:                 Object deptObj = cmbDepartment.getSelectedItem();
2371:                 String dept = (deptObj != null) ? deptObj.toString().trim() : "";
2372:
2373:                 if (fn.isEmpty() || un.isEmpty() || pw.isEmpty() || dept.isEmpty()) {
2374:                     JOptionPane.showMessageDialog(this, "Please fill in all required fields (Name,
2375:                     return;
2376:                 }
2377:
2378:                 if (pw.length() < 8) {
2379:                     JOptionPane.showMessageDialog(this, "Password must be at least 8 characters lo
2380:                     return;
2381:                 }
2382:
2383:                 boolean unChanged = (userToEdit != null) && un.equalsIgnoreCase(userToEdit.getUser
2384:                 if (!unChanged && db.isUsernameTaken(un)) {
2385:                     JOptionPane.showMessageDialog(this, "Username '" + un + "' is already taken! C
2386:                     return;
2387:                 }
2388:
2389:                 // Auto-register team if typed department doesn't exist yet
2390:                 if (db.getTeams().stream().noneMatch(t -> t.getName().equalsIgnoreCase(dept))) {
2391:                     db.saveTeam(new Team(dept, "Functional department for " + dept));
2392:                 }
2393:
2394:                 String em = (userToEdit != null && userToEdit.getEmail() != null && !userToEdit.ge
2395:                         ? userToEdit.getEmail() : (un.toLowerCase() + "@enterprise.com");
2396:
2397:                 Integer targetId = userToEdit != null ? userToEdit.getId() : null;
2398:                 if (userToEdit instanceof SuperAdmin) {
2399:                     createdUser = new SuperAdmin(targetId, un, pw, fn, em, dept);
2400:                 } else if ("Project Manager / Admin".equals(cmbRole.getSelectedItem())) {
2401:                     double limit = (userToEdit instanceof Manager) ? ((Manager) userToEdit).getBud
2402:                     createdUser = new Manager(targetId, un, pw, fn, em, dept, limit);
2403:                 } else {
2404:                     double rate = (userToEdit instanceof Developer) ? ((Developer) userToEdit).get
2405:                     createdUser = new Developer(targetId, un, pw, fn, em, dept, rate);
2406:                 }
2407:                 saved = true;
2408:                 dispose();
2409:             });
2410:
2411:             footer.add(btnCancel);
2412:             footer.add(btnSave);
2413:             add(footer, BorderLayout.SOUTH);
2414:
2415:             pack();
2416:             setLocationRelativeTo(parent);
2417:             setResizable(false);
2418:         }
2419:
```

File: ProjectManagementSystem.java | Package: (default) (continued)

```
2420:         private void addRow(JPanel p, GridBagConstraints g, int row, String label, JComponent comp
2421:             g.gridx = 0; g.gridy = row; g.anchor = GridBagConstraints.WEST;
2422:             p.add(new JLabel(label), g);
2423:             g.gridx = 1;
2424:             p.add(comp, g);
2425:         }
2426:
2427:         public boolean isSaved() { return saved; }
2428:         public User getCreatedUser() { return createdUser; }
2429:     }
2430:
2431:     // ================================================================================
2432:     // MAIN APPLICATION GUI (JFRAME) WITH ENHANCED DASHBOARD & WBS TREE
2433:     // ================================================================================
2434:
2435:     public static class MainFrame extends JFrame {
2436:         private final DatabaseManager db;
2437:         private final ProjectService projectService;
2438:         private final TaskService taskService;
2439:         private User currentUser;
2440:
2441:         private JLabel lblUserBadge;
2442:         private JTabbedPane tabbedPane;
2443:
2444:         // Dashboard Components
2445:         private JLabel lblTotalProj, lblTotalTasks, lblCompletedTasks, lblActiveUsers;
2446:         private JPanel progressCardsPanel;
2447:         private JLabel lblTodoCount, lblProgCount, lblReviewCount, lblDoneCount;
2448:
2449:         // Project Panel Components
2450:         private ProjectTableModelAdapter projectAdapter;
2451:         private JTable projectTable;
2452:
2453:         // Task Panel Components
2454:         private TaskTableModelAdapter taskAdapter;
2455:         private DefaultTreeModel treeModel;
2456:         private DefaultMutableTreeNode treeRoot;
2457:         private JTree hierarchyTree;
2458:         private JTable taskTable;
2459:         private JComboBox<String> cmbProjectFilter;
2460:
2461:         // Tasks Team Filter
2462:         private JComboBox<String> cmbTeamFilter;
2463:
2464:         // Team Panel Components
2465:         private DefaultTableModel teamModel;
2466:         private JTable teamTable;
2467:         private DefaultTableModel teamsTableModel;
2468:         private JTable teamsTable;
2469:
2470:         // Time Logs Components (Admin / Super Admin)
2471:         private DefaultTableModel sessionTableModel;
2472:         private JTable sessionTable;
2473:         private DefaultTableModel workLogTableModel;
2474:         private JTable workLogTable;
2475:         private JLabel lblTotalSessionsMetric, lblActiveSessionsMetric, lblTotalHoursMetric;
2476:         private JComboBox<String> cmbTimeLogUserFilter;
2477:
2478:         public MainFrame(DatabaseManager db, User initialUser) {
```

File: ProjectManagementSystem.java | Package: (default) (continued)

```
2479:             super("Enterprise Project Management System");
2480:             this.db = db;
2481:             this.projectService = new ProjectService(db);
2482:             this.taskService = new TaskService(db);
2483:             this.currentUser = initialUser;
2484:
2485:             initComponents();
2486:             setSize(1240, 820);
2487:             setMinimumSize(new Dimension(1040, 680));
2488:             setLocationRelativeTo(null);
2489:             setDefaultCloseOperation(JFrame.DO_NOTHING_ON_CLOSE);
2490:             addWindowListener(new WindowAdapter() {
2491:                 @Override public void windowClosing(WindowEvent e) {
2492:                     int confirm = JOptionPane.showConfirmDialog(MainFrame.this,
2493:                             "Are you sure you want to exit the application?",
2494:                             "Confirm Exit", JOptionPane.YES_NO_OPTION);
2495:                     if (confirm == JOptionPane.YES_OPTION) {
2496:                         db.recordLogoutSession(currentUser);
2497:                         System.exit(0);
2498:                     }
2499:                 }
2500:             });
2501:         }
2502:
2503:         private void initComponents() {
2504:             setLayout(new BorderLayout());
2505:             getContentPane().setBackground(UITheme.BG_LIGHT);
2506:
2507:             // Enterprise Header
2508:             JPanel header = new JPanel(new BorderLayout());
2509:             header.setBackground(UITheme.PRIMARY);
2510:             header.setBorder(BorderFactory.createEmptyBorder(12, 22, 12, 22));
2511:
2512:             JPanel leftHeader = new JPanel(new FlowLayout(FlowLayout.LEFT, 12, 0));
2513:             leftHeader.setOpaque(false);
2514:             JLabel logo = new JLabel("PMS");
2515:             logo.setFont(new Font("Segoe UI", Font.BOLD, 22));
2516:             logo.setForeground(Color.WHITE);
2517:             JLabel title = new JLabel("Enterprise Workforce & Project Governance Platform");
2518:             title.setFont(UITheme.FONT_SMALL);
2519:             title.setForeground(new Color(224, 231, 255));
2520:             leftHeader.add(logo);
2521:             leftHeader.add(title);
2522:             header.add(leftHeader, BorderLayout.WEST);
2523:
2524:             JPanel rightHeader = new JPanel(new FlowLayout(FlowLayout.RIGHT, 14, 0));
2525:             rightHeader.setOpaque(false);
2526:
2527:             // Dynamic Method Dispatch
2528:             lblUserBadge = new JLabel(currentUser.getProfileBadgeText());
2529:             lblUserBadge.setFont(UITheme.FONT_BOLD);
2530:             lblUserBadge.setForeground(Color.WHITE);
2531:
2532:             JButton btnLogout = UITheme.createDangerButton("Logout");
2533:             btnLogout.setFont(UITheme.FONT_BOLD);
2534:             btnLogout.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
2535:             btnLogout.addActionListener(e -> handleLogout());
2536:
2537:             rightHeader.add(lblUserBadge);
```

File: ProjectManagementSystem.java | Package: (default) (continued)

```
2538:             rightHeader.add(Box.createHorizontalStrut(8));
2539:             rightHeader.add(btnLogout);
2540:             header.add(rightHeader, BorderLayout.EAST);
2541:             add(header, BorderLayout.NORTH);
2542:
2543:             // Tabs
2544:             tabbedPane = new JTabbedPane();
2545:             tabbedPane.setFont(UITheme.FONT_BOLD);
2546:             tabbedPane.addTab("Executive Dashboard", createDashboardPanel());
2547:             tabbedPane.addTab("Project Management", createProjectsPanel());
2548:             tabbedPane.addTab("Tasks & WBS Hierarchy", createTasksPanel());
2549:             tabbedPane.addTab("Team Members & Roles", createTeamPanel());
2550:
2551:             if (currentUser.hasPermission(Permission.VIEW_TIME_LOGS)) {
2552:                 tabbedPane.addTab("User Time-In & Work Logs", createTimeLogsPanel());
2553:             }
2554:
2555:             tabbedPane.addChangeListener(e -> refreshAllViews());
2556:             add(tabbedPane, BorderLayout.CENTER);
2557:
2558:             // Status Bar
2559:             JPanel status = new JPanel(new BorderLayout());
2560:             status.setBackground(Color.WHITE);
2561:             status.setBorder(BorderFactory.createCompoundBorder(BorderFactory.createMatteBorder(1,
2562:             JLabel lblStat = new JLabel(db.getDatabaseStatusText());
2563:             lblStat.setFont(UITheme.FONT_SMALL);
2564:             lblStat.setForeground(UITheme.SUCCESS);
2565:             status.add(lblStat, BorderLayout.WEST);
2566:             add(status, BorderLayout.SOUTH);
2567:
2568:             refreshAllViews();
2569:         }
2570:
2571:         // ------------------------------------------------------------------------
2572:         // 1. ENHANCED PROFESSIONAL DASHBOARD
2573:         // ------------------------------------------------------------------------
2574:         private JPanel createDashboardPanel() {
2575:             JPanel panel = new JPanel(new BorderLayout(16, 16));
2576:             panel.setBackground(UITheme.BG_LIGHT);
2577:             panel.setBorder(BorderFactory.createEmptyBorder(20, 22, 20, 22));
2578:
2579:             // Top Stat Cards (4 columns)
2580:             JPanel metricsGrid = new JPanel(new GridLayout(1, 4, 16, 0));
2581:             metricsGrid.setOpaque(false);
2582:
2583:             lblTotalProj = new JLabel("0", SwingConstants.CENTER);
2584:             lblTotalTasks = new JLabel("0", SwingConstants.CENTER);
2585:             lblCompletedTasks = new JLabel("0", SwingConstants.CENTER);
2586:             lblActiveUsers = new JLabel("0", SwingConstants.CENTER);
2587:
2588:             metricsGrid.add(createStatCard("ACTIVE PROJECTS", lblTotalProj, UITheme.PRIMARY, "Port
2589:             metricsGrid.add(createStatCard("TOTAL TASKS", lblTotalTasks, UITheme.PRIMARY_ACCENT, "
2590:             metricsGrid.add(createStatCard("COMPLETED TASKS", lblCompletedTasks, UITheme.SUCCESS,
2591:             metricsGrid.add(createStatCard("TEAM MEMBERS", lblActiveUsers, new Color(139, 92, 246)
2592:             panel.add(metricsGrid, BorderLayout.NORTH);
2593:
2594:             // Center: Split between Project Progress Rollups and Task Status Breakdown
2595:             JPanel splitCenter = new JPanel(new GridLayout(1, 2, 16, 0));
2596:             splitCenter.setOpaque(false);
```

File: ProjectManagementSystem.java | Package: (default) (continued)

```
2597:
2598:             // Left Card: Project Rollups
2599:             JPanel progressSection = UITheme.createCard();
2600:             progressSection.setLayout(new BorderLayout(0, 12));
2601:             JLabel lblPTitle = new JLabel("Project Recursive Progress Tracking (Rollup from Sub-ta
2602:             lblPTitle.setFont(UITheme.FONT_SECTION);
2603:             lblPTitle.setForeground(UITheme.TEXT_DARK);
2604:             progressSection.add(lblPTitle, BorderLayout.NORTH);
2605:
2606:             progressCardsPanel = new JPanel();
2607:             progressCardsPanel.setLayout(new BoxLayout(progressCardsPanel, BoxLayout.Y_AXIS));
2608:             progressCardsPanel.setOpaque(false);
2609:             JScrollPane scrollProj = new JScrollPane(progressCardsPanel);
2610:             scrollProj.setBorder(BorderFactory.createEmptyBorder());
2611:             scrollProj.setOpaque(false);
2612:             scrollProj.getViewport().setOpaque(false);
2613:             progressSection.add(scrollProj, BorderLayout.CENTER);
2614:             splitCenter.add(progressSection);
2615:
2616:             // Right Card: Workflow Distribution & System Overview
2617:             JPanel summarySection = UITheme.createCard();
2618:             summarySection.setLayout(new BorderLayout(0, 15));
2619:             JLabel lblSTitle = new JLabel("Task Workflow Distribution & System Health");
2620:             lblSTitle.setFont(UITheme.FONT_SECTION);
2621:             lblSTitle.setForeground(UITheme.TEXT_DARK);
2622:             summarySection.add(lblSTitle, BorderLayout.NORTH);
2623:
2624:             JPanel distroPanel = new JPanel(new GridLayout(4, 1, 10, 10));
2625:             distroPanel.setOpaque(false);
2626:
2627:             lblTodoCount = new JLabel("To Do: 0");
2628:             lblProgCount = new JLabel("In Progress: 0");
2629:             lblReviewCount = new JLabel("In Review: 0");
2630:             lblDoneCount = new JLabel("Completed: 0");
2631:
2632:             distroPanel.add(createWorkflowItem("To Do (Pending Execution)", lblTodoCount, new Colo
2633:             distroPanel.add(createWorkflowItem("In Progress (Active Sprints)", lblProgCount, new C
2634:             distroPanel.add(createWorkflowItem("In Review (Quality Assurance)", lblReviewCount, ne
2635:             distroPanel.add(createWorkflowItem("Completed (Delivered)", lblDoneCount, new Color(20
2636:             summarySection.add(distroPanel, BorderLayout.CENTER);
2637:
2638:             splitCenter.add(summarySection);
2639:             panel.add(splitCenter, BorderLayout.CENTER);
2640:             return panel;
2641:         }
2642:
2643:         private JPanel createStatCard(String title, JLabel val, Color color, String sub) {
2644:             JPanel card = UITheme.createCard();
2645:             card.setLayout(new BorderLayout(0, 6));
2646:
2647:             JLabel t = new JLabel(title);
2648:             t.setFont(UITheme.FONT_BADGE);
2649:             t.setForeground(UITheme.TEXT_MUTED);
2650:
2651:             val.setFont(new Font("Segoe UI", Font.BOLD, 30));
2652:             val.setForeground(color);
2653:
2654:             JLabel s = new JLabel(sub);
2655:             s.setFont(UITheme.FONT_SMALL);
```

File: ProjectManagementSystem.java | Package: (default) (continued)

```
2656:             s.setForeground(UITheme.TEXT_MUTED);
2657:
2658:             card.add(t, BorderLayout.NORTH);
2659:             card.add(val, BorderLayout.CENTER);
2660:             card.add(s, BorderLayout.SOUTH);
2661:             return card;
2662:         }
2663:
2664:         private JPanel createWorkflowItem(String title, JLabel countLabel, Color bg, Color fg) {
2665:             JPanel item = new JPanel(new BorderLayout());
2666:             item.setBackground(bg);
2667:             item.setBorder(BorderFactory.createCompoundBorder(
2668:                     BorderFactory.createLineBorder(fg, 1),
2669:                     new EmptyBorder(8, 12, 8, 12)
2670:             ));
2671:             JLabel titleLbl = new JLabel(title);
2672:             titleLbl.setFont(UITheme.FONT_BOLD);
2673:             titleLbl.setForeground(fg);
2674:
2675:             countLabel.setFont(UITheme.FONT_BOLD);
2676:             countLabel.setForeground(fg);
2677:
2678:             item.add(titleLbl, BorderLayout.WEST);
2679:             item.add(countLabel, BorderLayout.EAST);
2680:             return item;
2681:         }
2682:
2683:         // ------------------------------------------------------------------------
2684:         // 2. ENHANCED PROJECT MANAGEMENT PANEL (FULL FIELDS & WARNINGS)
2685:         // ------------------------------------------------------------------------
2686:         private JPanel createProjectsPanel() {
2687:             JPanel panel = new JPanel(new BorderLayout(0, 14));
2688:             panel.setBackground(UITheme.BG_LIGHT);
2689:             panel.setBorder(BorderFactory.createEmptyBorder(18, 20, 18, 20));
2690:
2691:             JPanel topBar = new JPanel(new BorderLayout());
2692:             topBar.setOpaque(false);
2693:
2694:             JLabel heading = new JLabel("Enterprise Projects Directory");
2695:             heading.setFont(UITheme.FONT_HEADER);
2696:             heading.setForeground(UITheme.TEXT_DARK);
2697:             topBar.add(heading, BorderLayout.WEST);
2698:
2699:             JPanel actions = new JPanel(new FlowLayout(FlowLayout.RIGHT, 8, 0));
2700:             actions.setOpaque(false);
2701:
2702:             if (currentUser.hasPermission(Permission.CREATE_PROJECT)) {
2703:                 JButton btnNew = UITheme.createPrimaryButton("+ New Project");
2704:                 btnNew.addActionListener(e -> handleNewProject());
2705:                 actions.add(btnNew);
2706:             }
2707:
2708:             if (currentUser.hasPermission(Permission.EDIT_PROJECT)) {
2709:                 JButton btnEdit = UITheme.createSecondaryButton("Edit Project");
2710:                 btnEdit.addActionListener(e -> handleEditProject());
2711:                 actions.add(btnEdit);
2712:             }
2713:
2714:             if (currentUser.hasPermission(Permission.DELETE_PROJECT)) {
```

File: ProjectManagementSystem.java | Package: (default) (continued)

```
2715:                 JButton btnDel = UITheme.createDangerButton("Delete Project");
2716:                 btnDel.addActionListener(e -> handleDeleteProject());
2717:                 actions.add(btnDel);
2718:             }
2719:
2720:             if (actions.getComponentCount() > 0) {
2721:                 topBar.add(actions, BorderLayout.EAST);
2722:             }
2723:
2724:             panel.add(topBar, BorderLayout.NORTH);
2725:
2726:             projectAdapter = new ProjectTableModelAdapter();
2727:             projectTable = new JTable(projectAdapter);
2728:             projectTable.setRowHeight(34);
2729:             projectTable.setFont(UITheme.FONT_REGULAR);
2730:             projectTable.getTableHeader().setFont(UITheme.FONT_BOLD);
2731:             projectTable.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);
2732:             projectTable.getColumnModel().getColumn(4).setCellRenderer(UITheme.createStatusRendere
2733:
2734:             // Double click to edit (if permitted)
2735:             projectTable.addMouseListener(new MouseAdapter() {
2736:                 @Override public void mouseClicked(MouseEvent e) {
2737:                     if (e.getClickCount() == 2 && currentUser.hasPermission(Permission.EDIT_PROJE
2738:                         handleEditProject();
2739:                     }
2740:                 }
2741:             });
2742:
2743:             JScrollPane scroll = new JScrollPane(projectTable);
2744:             scroll.setBorder(BorderFactory.createLineBorder(UITheme.BORDER));
2745:             panel.add(scroll, BorderLayout.CENTER);
2746:             return panel;
2747:         }
2748:
2749:         private void handleNewProject() {
2750:             try {
2751:                 if (!currentUser.hasPermission(Permission.CREATE_PROJECT)) {
2752:                     throw new UnauthorizedAccessException(currentUser.getUsername(), "CREATE_PROJE
2753:                 }
2754:                 ProjectFormDialog dialog = new ProjectFormDialog(this, null);
2755:                 dialog.setVisible(true);
2756:                 if (dialog.isSaved()) {
2757:                     projectService.saveProject(dialog.getProject(), currentUser);
2758:                     refreshAllViews();
2759:                     JOptionPane.showMessageDialog(this, "Project '" + dialog.getProject().getTitle
2760:                 }
2761:             } catch (UnauthorizedAccessException ex) {
2762:                 JOptionPane.showMessageDialog(this, ex.getMessage() + "\n(Software Developers have
2763:             }
2764:         }
2765:
2766:         private void handleEditProject() {
2767:             int row = projectTable.getSelectedRow();
2768:             if (row < 0) {
2769:                 JOptionPane.showMessageDialog(this, "Please select a project from the table to edi
2770:                 return;
2771:             }
2772:             try {
2773:                 if (!currentUser.hasPermission(Permission.EDIT_PROJECT)) {
```

File: ProjectManagementSystem.java | Package: (default) (continued)

```
2774:                     throw new UnauthorizedAccessException(currentUser.getUsername(), "EDIT_PROJECT
2775:                 }
2776:                 Project p = projectAdapter.getProjectAt(row);
2777:                 ProjectFormDialog dialog = new ProjectFormDialog(this, p);
2778:                 dialog.setVisible(true);
2779:                 if (dialog.isSaved()) {
2780:                     projectService.saveProject(dialog.getProject(), currentUser);
2781:                     refreshAllViews();
2782:                     if (row >= 0 && row < projectTable.getRowCount()) {
2783:                         projectTable.setRowSelectionInterval(row, row);
2784:                     }
2785:                     JOptionPane.showMessageDialog(this, "Project updated successfully!", "Saved",
2786:                 }
2787:             } catch (UnauthorizedAccessException ex) {
2788:                 JOptionPane.showMessageDialog(this, ex.getMessage(), "Access Denied", JOptionPane.
2789:             }
2790:         }
2791:
2792:         private void handleDeleteProject() {
2793:             int row = projectTable.getSelectedRow();
2794:             if (row < 0) {
2795:                 JOptionPane.showMessageDialog(this, "Please select a project from the table to del
2796:                 return;
2797:             }
2798:             Project p = projectAdapter.getProjectAt(row);
2799:
2800:             // Mandatory Warning Message before Deleting
2801:             int confirm = JOptionPane.showConfirmDialog(this,
2802:                     String.format("WARNING: Are you sure you want to permanently delete Project #%
2803:                             "This action will also delete all associated nested tasks and work log
2804:                             p.getId(), p.getTitle()),
2805:                     "Confirm Project Deletion",
2806:                     JOptionPane.YES_NO_OPTION, JOptionPane.WARNING_MESSAGE);
2807:
2808:             if (confirm == JOptionPane.YES_OPTION) {
2809:                 try {
2810:                     projectService.deleteProject(p.getId(), currentUser);
2811:                     refreshAllViews();
2812:                     JOptionPane.showMessageDialog(this, "Project #" + p.getId() + " and its tasks
2813:                 } catch (UnauthorizedAccessException ex) {
2814:                     JOptionPane.showMessageDialog(this, ex.getMessage(), "Access Denied", JOptionP
2815:                 } catch (ProjectNotFoundException ex) {
2816:                     JOptionPane.showMessageDialog(this, ex.getMessage(), "Error", JOptionPane.ERRO
2817:                 }
2818:             }
2819:         }
2820:
2821:         // ---------------------------------------------------------------------
2822:         // 3. ENHANCED TASKS & PERFECT WBS TREE PANEL
2823:         // ---------------------------------------------------------------------
2824:         private JPanel createTasksPanel() {
2825:             JPanel panel = new JPanel(new BorderLayout(0, 14));
2826:             panel.setBackground(UITheme.BG_LIGHT);
2827:             panel.setBorder(BorderFactory.createEmptyBorder(18, 20, 18, 20));
2828:
2829:             JPanel topBar = new JPanel(new BorderLayout(10, 0));
2830:             topBar.setOpaque(false);
2831:
2832:             JPanel filterArea = new JPanel(new FlowLayout(FlowLayout.LEFT, 8, 0));
```

File: ProjectManagementSystem.java | Package: (default) (continued)

```
2833:             filterArea.setOpaque(false);
2834:             filterArea.add(new JLabel("Filter Project:"));
2835:             cmbProjectFilter = new JComboBox<>();
2836:             cmbProjectFilter.setFont(UITheme.FONT_REGULAR);
2837:             cmbProjectFilter.addActionListener(e -> applyTaskFilter());
2838:             filterArea.add(cmbProjectFilter);
2839:
2840:             filterArea.add(new JLabel("Team:"));
2841:             cmbTeamFilter = new JComboBox<>();
2842:             cmbTeamFilter.setFont(UITheme.FONT_REGULAR);
2843:             cmbTeamFilter.addActionListener(e -> applyTaskFilter());
2844:             filterArea.add(cmbTeamFilter);
2845:             topBar.add(filterArea, BorderLayout.WEST);
2846:
2847:             JPanel actions = new JPanel(new FlowLayout(FlowLayout.RIGHT, 8, 0));
2848:             actions.setOpaque(false);
2849:
2850:             if (currentUser.hasPermission(Permission.CREATE_TASK)) {
2851:                 JButton btnNewTask = UITheme.createPrimaryButton("+ New Task / Sub-task");
2852:                 btnNewTask.addActionListener(e -> handleNewTask());
2853:                 actions.add(btnNewTask);
2854:             }
2855:
2856:             if (currentUser.hasPermission(Permission.EDIT_TASK)) {
2857:                 JButton btnEditTask = UITheme.createSecondaryButton("Edit Task");
2858:                 btnEditTask.addActionListener(e -> handleEditTask());
2859:                 actions.add(btnEditTask);
2860:             }
2861:
2862:             if (currentUser.hasPermission(Permission.UPDATE_TASK_STATUS)) {
2863:                 JButton btnStatus = UITheme.createSecondaryButton("Transition Status");
2864:                 btnStatus.addActionListener(e -> handleTransitionStatus());
2865:                 actions.add(btnStatus);
2866:             }
2867:
2868:             if (currentUser.hasPermission(Permission.LOG_HOURS)) {
2869:                 JButton btnHours = UITheme.createSecondaryButton("Log Hours");
2870:                 btnHours.addActionListener(e -> handleLogHours());
2871:                 actions.add(btnHours);
2872:             }
2873:
2874:             if (currentUser.hasPermission(Permission.DELETE_TASK)) {
2875:                 JButton btnDelTask = UITheme.createDangerButton("Delete Task");
2876:                 btnDelTask.addActionListener(e -> handleDeleteTask());
2877:                 actions.add(btnDelTask);
2878:             }
2879:
2880:             if (actions.getComponentCount() > 0) {
2881:                 topBar.add(actions, BorderLayout.EAST);
2882:             }
2883:
2884:             panel.add(topBar, BorderLayout.NORTH);
2885:
2886:             // JTree WBS on Left, JTable on Right
2887:             treeRoot = new DefaultMutableTreeNode("Work Breakdown Structure (WBS)");
2888:             treeModel = new DefaultTreeModel(treeRoot);
2889:             hierarchyTree = new JTree(treeModel);
2890:             hierarchyTree.setFont(UITheme.FONT_REGULAR);
2891:             hierarchyTree.setRowHeight(24);
```

File: ProjectManagementSystem.java | Package: (default) (continued)

```
2892:
2893:             // Clicking any tree node synchronizes and selects that task in the table!
2894:             hierarchyTree.addTreeSelectionListener(new TreeSelectionListener() {
2895:                 @Override public void valueChanged(TreeSelectionEvent e) {
2896:                     DefaultMutableTreeNode node = (DefaultMutableTreeNode) hierarchyTree.getLastSe
2897:                     if (node != null && node.getUserObject() instanceof TaskNodeItem) {
2898:                         TaskNodeItem item = (TaskNodeItem) node.getUserObject();
2899:                         highlightTaskInTable(item.task.getId());
2900:                     }
2901:                 }
2902:             });
2903:
2904:             taskAdapter = new TaskTableModelAdapter();
2905:             taskTable = new JTable(taskAdapter);
2906:             taskTable.setRowHeight(34);
2907:             taskTable.setFont(UITheme.FONT_REGULAR);
2908:             taskTable.getTableHeader().setFont(UITheme.FONT_BOLD);
2909:             taskTable.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);
2910:             taskTable.getColumnModel().getColumn(4).setCellRenderer(UITheme.createStatusRenderer()
2911:             taskTable.getColumnModel().getColumn(7).setCellRenderer(UITheme.createProgressRenderer
2912:
2913:             taskTable.addMouseListener(new MouseAdapter() {
2914:                 @Override public void mouseClicked(MouseEvent e) {
2915:                     if (e.getClickCount() == 2) handleEditTask();
2916:                 }
2917:             });
2918:
2919:             JScrollPane scrollTree = new JScrollPane(hierarchyTree);
2920:             scrollTree.setBorder(BorderFactory.createCompoundBorder(
2921:                     BorderFactory.createTitledBorder(BorderFactory.createLineBorder(UITheme.BORDER
2922:                     new EmptyBorder(6, 6, 6, 6)
2923:             ));
2924:             scrollTree.setPreferredSize(new Dimension(380, 400));
2925:
2926:             JScrollPane scrollTable = new JScrollPane(taskTable);
2927:             scrollTable.setBorder(BorderFactory.createLineBorder(UITheme.BORDER));
2928:
2929:             JSplitPane split = new JSplitPane(JSplitPane.HORIZONTAL_SPLIT, scrollTree, scrollTable
2930:             split.setDividerLocation(380);
2931:             split.setOpaque(false);
2932:             panel.add(split, BorderLayout.CENTER);
2933:             return panel;
2934:         }
2935:
2936:         private void highlightTaskInTable(int taskId) {
2937:             for (int i = 0; i < taskAdapter.getRowCount(); i++) {
2938:                 Task t = taskAdapter.getTaskAt(i);
2939:                 if (t != null && t.getId().equals(taskId)) {
2940:                     taskTable.setRowSelectionInterval(i, i);
2941:                     taskTable.scrollRectToVisible(taskTable.getCellRect(i, 0, true));
2942:                     break;
2943:                 }
2944:             }
2945:         }
2946:
2947:         private void handleNewTask() {
2948:             try {
2949:                 if (!currentUser.hasPermission(Permission.CREATE_TASK)) {
2950:                     throw new UnauthorizedAccessException(currentUser.getUsername(), "CREATE_TASK"
```

File: ProjectManagementSystem.java | Package: (default) (continued)

```
2951:                 }
2952:                 List<Project> projs = projectService.getAllProjects();
2953:                 if (projs.isEmpty()) {
2954:                     JOptionPane.showMessageDialog(this, "Please create a project first.", "No Proj
2955:                     return;
2956:                 }
2957:                 TaskFormDialog dialog = new TaskFormDialog(this, null, projs, taskService.getAllTa
2958:                 dialog.setVisible(true);
2959:                 if (dialog.isSaved()) {
2960:                     taskService.saveTask(dialog.getTask(), currentUser);
2961:                     refreshAllViews();
2962:                     JOptionPane.showMessageDialog(this, "Task created successfully!", "Success", J
2963:                 }
2964:             } catch (UnauthorizedAccessException ex) {
2965:                 JOptionPane.showMessageDialog(this, ex.getMessage(), "Access Denied", JOptionPane.
2966:             }
2967:         }
2968:
2969:         private void handleEditTask() {
2970:             int row = taskTable.getSelectedRow();
2971:             if (row < 0) {
2972:                 JOptionPane.showMessageDialog(this, "Please select a task from the table to edit."
2973:                 return;
2974:             }
2975:             try {
2976:                 if (!currentUser.hasPermission(Permission.EDIT_TASK)) {
2977:                     throw new UnauthorizedAccessException(currentUser.getUsername(), "EDIT_TASK");
2978:                 }
2979:                 Task t = taskAdapter.getTaskAt(row);
2980:                 TaskFormDialog dialog = new TaskFormDialog(this, t, projectService.getAllProjects(
2981:                 dialog.setVisible(true);
2982:                 if (dialog.isSaved()) {
2983:                     taskService.saveTask(dialog.getTask(), currentUser);
2984:                     refreshAllViews();
2985:                     if (row >= 0 && row < taskTable.getRowCount()) {
2986:                         taskTable.setRowSelectionInterval(row, row);
2987:                     }
2988:                     JOptionPane.showMessageDialog(this, "Task updated successfully!", "Saved", JOp
2989:                 }
2990:             } catch (UnauthorizedAccessException ex) {
2991:                 JOptionPane.showMessageDialog(this, ex.getMessage(), "Access Denied", JOptionPane.
2992:             }
2993:         }
2994:
2995:         private void handleTransitionStatus() {
2996:             int row = taskTable.getSelectedRow();
2997:             if (row < 0) {
2998:                 JOptionPane.showMessageDialog(this, "Select a task to transition status.", "Select
2999:                 return;
3000:             }
3001:             Task t = taskAdapter.getTaskAt(row);
3002:             TaskStatus chosen = (TaskStatus) JOptionPane.showInputDialog(
3003:                     this, String.format("Current Status: %s\nSelect target status for Task #%d '%s
3004:                     "State Machine Workflow Transition", JOptionPane.QUESTION_MESSAGE,
3005:                     null, TaskStatus.values(), t.getStatus());
3006:             if (chosen != null && chosen != t.getStatus()) {
3007:                 try {
3008:                     taskService.updateTaskStatus(t.getId(), chosen, currentUser);
3009:                     refreshAllViews();
```

File: ProjectManagementSystem.java | Package: (default) (continued)

```
3010:                     if (row >= 0 && row < taskTable.getRowCount()) {
3011:                         taskTable.setRowSelectionInterval(row, row);
3012:                     }
3013:                     JOptionPane.showMessageDialog(this, "Status successfully changed to: " + chose
3014:                 } catch (InvalidTaskStateException ex) {
3015:                     JOptionPane.showMessageDialog(this, "State Machine Violation:\n" + ex.getMessa
3016:                 } catch (UnauthorizedAccessException ex) {
3017:                     JOptionPane.showMessageDialog(this, ex.getMessage(), "Access Denied", JOptionP
3018:                 } catch (Exception ex) {
3019:                     JOptionPane.showMessageDialog(this, ex.getMessage(), "Error", JOptionPane.ERRO
3020:                 }
3021:             }
3022:         }
3023:
3024:         private void handleLogHours() {
3025:             int row = taskTable.getSelectedRow();
3026:             if (row < 0) {
3027:                 JOptionPane.showMessageDialog(this, "Select a task to log hours against.", "Select
3028:                 return;
3029:             }
3030:             Task t = taskAdapter.getTaskAt(row);
3031:             String input = JOptionPane.showInputDialog(this,
3032:                     String.format("Task: #%d - %s\nCurrent Logged: %.2fh (Est: %.2fh)\n\nEnter add
3033:                             t.getId(), t.getTitle(), t.getLoggedHours(), t.getEstimatedHours()), "
3034:             if (input != null) {
3035:                 try {
3036:                     double hrs = Double.parseDouble(input.trim());
3037:                     if (hrs <= 0) throw new NumberFormatException();
3038:                     taskService.logHours(t.getId(), hrs, currentUser);
3039:                     refreshAllViews();
3040:                     if (row >= 0 && row < taskTable.getRowCount()) {
3041:                         taskTable.setRowSelectionInterval(row, row);
3042:                     }
3043:                     JOptionPane.showMessageDialog(this, String.format("Logged %.2f hours on task #
3044:                 } catch (Exception ex) {
3045:                     JOptionPane.showMessageDialog(this, "Please enter a valid numeric number of ho
3046:                 }
3047:             }
3048:         }
3049:
3050:         private void handleDeleteTask() {
3051:             int row = taskTable.getSelectedRow();
3052:             if (row < 0) {
3053:                 JOptionPane.showMessageDialog(this, "Select a task to delete.", "Selection Require
3054:                 return;
3055:             }
3056:             Task t = taskAdapter.getTaskAt(row);
3057:
3058:             // Mandatory Warning Message before Deleting
3059:             int confirm = JOptionPane.showConfirmDialog(this,
3060:                     String.format("WARNING: Are you sure you want to delete Task #%d '%s'?\n" +
3061:                             "Any associated nested sub-tasks will also be removed!",
3062:                             t.getId(), t.getTitle()),
3063:                     "Confirm Task Deletion",
3064:                     JOptionPane.YES_NO_OPTION, JOptionPane.WARNING_MESSAGE);
3065:
3066:             if (confirm == JOptionPane.YES_OPTION) {
3067:                 try {
3068:                     taskService.deleteTask(t.getId(), currentUser);
```

File: ProjectManagementSystem.java | Package: (default) (continued)

```
3069:                     refreshAllViews();
3070:                     JOptionPane.showMessageDialog(this, "Task #" + t.getId() + " was deleted.", "D
3071:                 } catch (UnauthorizedAccessException ex) {
3072:                     JOptionPane.showMessageDialog(this, ex.getMessage(), "Access Denied", JOptionP
3073:                 } catch (TaskNotFoundException ex) {
3074:                     JOptionPane.showMessageDialog(this, ex.getMessage(), "Error", JOptionPane.ERRO
3075:                 }
3076:             }
3077:         }
3078:
3079:         // ------------------------------------------------------------------
3080:         // 4. ENHANCED TEAM PANEL (ADD TEAM MEMBER - ADMIN ONLY)
3081:         // ------------------------------------------------------------------
3082:         private JPanel createTeamPanel() {
3083:             JPanel panel = new JPanel(new BorderLayout(0, 14));
3084:             panel.setBackground(UITheme.BG_LIGHT);
3085:             panel.setBorder(BorderFactory.createEmptyBorder(18, 20, 18, 20));
3086:
3087:             JPanel topBar = new JPanel(new BorderLayout());
3088:             topBar.setOpaque(false);
3089:
3090:             JPanel titles = new JPanel(new GridLayout(2, 1, 0, 2));
3091:             titles.setOpaque(false);
3092:             JLabel heading = new JLabel("Teams Directory & Organization Workforce");
3093:             heading.setFont(UITheme.FONT_HEADER);
3094:             heading.setForeground(UITheme.TEXT_DARK);
3095:             JLabel sub = new JLabel(currentUser.hasPermission(Permission.MANAGE_USERS)
3096:                     ? "Admin and Super Admin can manage members and teams. Super Admin manages Adm
3097:                     : "Organization Workforce and Functional Teams Roster (View-only for Software
3098:             sub.setFont(UITheme.FONT_SMALL);
3099:             sub.setForeground(UITheme.TEXT_MUTED);
3100:             titles.add(heading);
3101:             titles.add(sub);
3102:             topBar.add(titles, BorderLayout.WEST);
3103:
3104:             JPanel actions = new JPanel(new FlowLayout(FlowLayout.RIGHT, 8, 0));
3105:             actions.setOpaque(false);
3106:
3107:             if (currentUser.hasPermission(Permission.MANAGE_USERS)) {
3108:                 JButton btnAddMember = UITheme.createPrimaryButton("+ Add User");
3109:                 btnAddMember.addActionListener(e -> handleAddTeamMember());
3110:
3111:                 JButton btnEditMember = UITheme.createSecondaryButton("Edit User");
3112:                 btnEditMember.addActionListener(e -> handleEditUser());
3113:
3114:                 JButton btnDeleteUser = UITheme.createDangerButton("- Delete User");
3115:                 btnDeleteUser.addActionListener(e -> handleDeleteUser());
3116:
3117:                 JButton btnAddTeam = UITheme.createSuccessButton("+ Add New Team");
3118:                 btnAddTeam.addActionListener(e -> handleAddTeam());
3119:
3120:                 JButton btnDeleteTeam = UITheme.createSecondaryButton("- Delete Team");
3121:                 btnDeleteTeam.addActionListener(e -> handleDeleteTeam());
3122:
3123:                 actions.add(btnAddMember);
3124:                 actions.add(btnEditMember);
3125:                 actions.add(btnDeleteUser);
3126:                 actions.add(btnAddTeam);
3127:                 actions.add(btnDeleteTeam);
```

File: ProjectManagementSystem.java | Package: (default) (continued)

```
3128:                 topBar.add(actions, BorderLayout.EAST);
3129:             }
3130:             panel.add(topBar, BorderLayout.NORTH);
3131:
3132:             // Split: Top Teams Table, Bottom Members Table
3133:             JPanel centerSplit = new JPanel(new GridLayout(2, 1, 0, 14));
3134:             centerSplit.setOpaque(false);
3135:
3136:             // 1. Teams Section
3137:             JPanel teamsCard = UITheme.createCard();
3138:             teamsCard.setLayout(new BorderLayout(0, 8));
3139:             JLabel lblTeamsHead = new JLabel("Registered Functional Teams (Departments / Squads)")
3140:             lblTeamsHead.setFont(UITheme.FONT_SECTION);
3141:             lblTeamsHead.setForeground(UITheme.TEXT_DARK);
3142:             teamsCard.add(lblTeamsHead, BorderLayout.NORTH);
3143:
3144:             Vector<String> tmCols = new Vector<>(Arrays.asList("Team ID", "Team Name", "Domain / M
3145:             teamsTableModel = new DefaultTableModel(tmCols, 0) {
3146:                 @Override public boolean isCellEditable(int r, int c) { return false; }
3147:             };
3148:             teamsTable = new JTable(teamsTableModel);
3149:             teamsTable.setRowHeight(32);
3150:             teamsTable.setFont(UITheme.FONT_REGULAR);
3151:             teamsTable.getTableHeader().setFont(UITheme.FONT_BOLD);
3152:             teamsTable.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);
3153:             teamsCard.add(new JScrollPane(teamsTable), BorderLayout.CENTER);
3154:             centerSplit.add(teamsCard);
3155:
3156:             // 2. Members Section
3157:             JPanel membersCard = UITheme.createCard();
3158:             membersCard.setLayout(new BorderLayout(0, 8));
3159:             JLabel lblMembersHead = new JLabel(currentUser.hasPermission(Permission.MANAGE_USERS)
3160:                     ? "Personnel & Role Roster (Select a user to Edit, Delete, or View)"
3161:                     : "Personnel & Role Roster (Directory View)");
3162:             lblMembersHead.setFont(UITheme.FONT_SECTION);
3163:             lblMembersHead.setForeground(UITheme.TEXT_DARK);
3164:             membersCard.add(lblMembersHead, BorderLayout.NORTH);
3165:
3166:             Vector<String> memCols = new Vector<>(Arrays.asList("User ID", "Name", "Username", "Ro
3167:             teamModel = new DefaultTableModel(memCols, 0) {
3168:                 @Override public boolean isCellEditable(int r, int c) { return false; }
3169:             };
3170:             teamTable = new JTable(teamModel);
3171:             teamTable.setRowHeight(32);
3172:             teamTable.setFont(UITheme.FONT_REGULAR);
3173:             teamTable.getTableHeader().setFont(UITheme.FONT_BOLD);
3174:             teamTable.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);
3175:
3176:             // Double click to edit user
3177:             teamTable.addMouseListener(new MouseAdapter() {
3178:                 @Override public void mouseClicked(MouseEvent e) {
3179:                     if (e.getClickCount() == 2 && currentUser.hasPermission(Permission.MANAGE_USER
3180:                         handleEditUser();
3181:                     }
3182:                 }
3183:             });
3184:
3185:             membersCard.add(new JScrollPane(teamTable), BorderLayout.CENTER);
3186:             centerSplit.add(membersCard);
```

File: ProjectManagementSystem.java | Package: (default) (continued)

```
3187:
3188:             panel.add(centerSplit, BorderLayout.CENTER);
3189:             return panel;
3190:         }
3191:
3192:         private void handleAddTeamMember() {
3193:             if (!currentUser.hasPermission(Permission.MANAGE_USERS)) {
3194:                 JOptionPane.showMessageDialog(this, "Access Denied: Only Administrators have permi
3195:                 return;
3196:             }
3197:             AddTeamMemberDialog dialog = new AddTeamMemberDialog(this, db, currentUser);
3198:             dialog.setVisible(true);
3199:             if (dialog.isSaved()) {
3200:                 User u = dialog.getCreatedUser();
3201:                 db.saveUser(u);
3202:                 refreshAllViews();
3203:                 JOptionPane.showMessageDialog(this, "Account '" + u.getFullName() + "' (" + u.getF
3204:             }
3205:         }
3206:
3207:         private void handleEditUser() {
3208:             if (!currentUser.hasPermission(Permission.MANAGE_USERS)) {
3209:                 JOptionPane.showMessageDialog(this, "Access Denied: You do not have permission to
3210:                 return;
3211:             }
3212:             int row = teamTable.getSelectedRow();
3213:             if (row < 0) {
3214:                 JOptionPane.showMessageDialog(this, "Please select a user from the table to edit."
3215:                 return;
3216:             }
3217:             int userId = (Integer) teamModel.getValueAt(row, 0);
3218:             User targetUser = db.getUsers().stream().filter(u -> u.getId().equals(userId)).findFir
3219:             if (targetUser == null) {
3220:                 JOptionPane.showMessageDialog(this, "User not found.", "Error", JOptionPane.ERROR_
3221:                 return;
3222:             }
3223:             if (targetUser.getRole() == UserRole.SUPER_ADMIN && currentUser.getRole() != UserRole.
3224:                 JOptionPane.showMessageDialog(this, "Access Denied: Only Super Administrators can
3225:                 return;
3226:             }
3227:             if (targetUser.getRole() == UserRole.MANAGER && !currentUser.hasPermission(Permission.
3228:                 JOptionPane.showMessageDialog(this,
3229:                         "Access Denied: Only Super Administrators have permission to edit other Ac
3230:                         "Your Current Role: " + currentUser.getRole().getLabel(),
3231:                         "Super Admin Privilege Required", JOptionPane.WARNING_MESSAGE);
3232:                 return;
3233:             }
3234:             AddTeamMemberDialog dialog = new AddTeamMemberDialog(this, db, currentUser, targetUser
3235:             dialog.setVisible(true);
3236:             if (dialog.isSaved()) {
3237:                 User updated = dialog.getCreatedUser();
3238:                 db.saveUser(updated);
3239:                 if (updated.getId().equals(currentUser.getId())) {
3240:                     currentUser.setFullName(updated.getFullName());
3241:                     currentUser.setEmail(updated.getEmail());
3242:                     lblUserBadge.setText(currentUser.getProfileBadgeText());
3243:                 }
3244:                 refreshAllViews();
3245:                 if (row >= 0 && row < teamTable.getRowCount()) {
```

File: ProjectManagementSystem.java | Package: (default) (continued)

```
3246:                     teamTable.setRowSelectionInterval(row, row);
3247:                 }
3248:                 JOptionPane.showMessageDialog(this, "Account '" + updated.getFullName() + "' (" +
3249:             }
3250:         }
3251:
3252:         private void handleDeleteUser() {
3253:             if (!currentUser.hasPermission(Permission.MANAGE_USERS)) {
3254:                 JOptionPane.showMessageDialog(this, "Access Denied: You do not have permission to
3255:                 return;
3256:             }
3257:             int row = teamTable.getSelectedRow();
3258:             if (row < 0) {
3259:                 JOptionPane.showMessageDialog(this, "Please select a user from the table to delete
3260:                 return;
3261:             }
3262:             int userId = (Integer) teamModel.getValueAt(row, 0);
3263:             User targetUser = db.getUsers().stream().filter(u -> u.getId().equals(userId)).findFir
3264:             if (targetUser == null) {
3265:                 JOptionPane.showMessageDialog(this, "User not found.", "Error", JOptionPane.ERROR_
3266:                 return;
3267:             }
3268:             if (targetUser.getId().equals(currentUser.getId())) {
3269:                 JOptionPane.showMessageDialog(this, "You cannot delete your own active account!",
3270:                 return;
3271:             }
3272:             if (targetUser.getRole() == UserRole.SUPER_ADMIN) {
3273:                 JOptionPane.showMessageDialog(this, "Access Denied: Super Administrator accounts a
3274:                 return;
3275:             }
3276:             if (targetUser.getRole() == UserRole.MANAGER) {
3277:                 if (!currentUser.hasPermission(Permission.MANAGE_ADMINS)) {
3278:                     JOptionPane.showMessageDialog(this,
3279:                             "Access Denied: Only Super Administrators have permission to delete Ad
3280:                             "Your Current Role: " + currentUser.getRole().getLabel(),
3281:                             "Super Admin Privilege Required", JOptionPane.WARNING_MESSAGE);
3282:                     return;
3283:                 }
3284:             }
3285:             int confirm = JOptionPane.showConfirmDialog(this,
3286:                     String.format("WARNING: Are you sure you want to permanently delete user:\n\n"
3287:                                   "  ID: #%d\n" +
3288:                                   "  Name: %s\n" +
3289:                                   "  Username: %s\n" +
3290:                                   "  Role: %s\n\n" +
3291:                                   "This user will be permanently deleted from the SQLite database.
3292:                             targetUser.getId(), targetUser.getFullName(), targetUser.getUsername()
3293:                     "Confirm User Deletion",
3294:                     JOptionPane.YES_NO_OPTION, JOptionPane.WARNING_MESSAGE);
3295:             if (confirm == JOptionPane.YES_OPTION) {
3296:                 db.deleteUser(targetUser.getId());
3297:                 refreshAllViews();
3298:                 JOptionPane.showMessageDialog(this, "User '" + targetUser.getFullName() + "' was s
3299:             }
3300:         }
3301:
3302:         private void handleAddTeam() {
3303:             if (!currentUser.hasPermission(Permission.MANAGE_USERS)) {
3304:                 JOptionPane.showMessageDialog(this, "Access Denied: Only Administrators can create
```

File: ProjectManagementSystem.java | Package: (default) (continued)

```
3305:                 return;
3306:             }
3307:             TeamFormDialog dlg = new TeamFormDialog(this, db);
3308:             dlg.setVisible(true);
3309:             if (dlg.isSaved()) {
3310:                 Team tm = new Team(dlg.getTeamName(), dlg.getTeamDescription());
3311:                 db.saveTeam(tm);
3312:                 refreshAllViews();
3313:                 JOptionPane.showMessageDialog(this, "Team '" + tm.getName() + "' created successfu
3314:             }
3315:         }
3316:
3317:         private void handleDeleteTeam() {
3318:             if (!currentUser.hasPermission(Permission.MANAGE_USERS)) {
3319:                 JOptionPane.showMessageDialog(this, "Access Denied: Only Administrators can delete
3320:                 return;
3321:             }
3322:             int row = teamsTable.getSelectedRow();
3323:             if (row < 0) {
3324:                 JOptionPane.showMessageDialog(this, "Please select a team from the Teams table to
3325:                 return;
3326:             }
3327:             int tid = (Integer) teamsTableModel.getValueAt(row, 0);
3328:             Team tm = db.getTeams().stream().filter(t -> t.getId().equals(tid)).findFirst().orElse
3329:             if (tm == null) return;
3330:             int confirm = JOptionPane.showConfirmDialog(this,
3331:                     "Are you sure you want to delete Team '" + tm.getName() + "'?\nAny associated
3332:                     "Confirm Team Deletion", JOptionPane.YES_NO_OPTION, JOptionPane.WARNING_MESSAG
3333:             if (confirm == JOptionPane.YES_OPTION) {
3334:                 db.deleteTeam(tid);
3335:                 refreshAllViews();
3336:                 JOptionPane.showMessageDialog(this, "Team '" + tm.getName() + "' was removed.", "T
3337:             }
3338:         }
3339:
3340:         private JPanel createTimeLogsPanel() {
3341:             JPanel panel = new JPanel(new BorderLayout(0, 14));
3342:             panel.setBackground(UITheme.BG_LIGHT);
3343:             panel.setBorder(BorderFactory.createEmptyBorder(18, 20, 18, 20));
3344:
3345:             // Top Header & Controls
3346:             JPanel topBar = new JPanel(new BorderLayout());
3347:             topBar.setOpaque(false);
3348:
3349:             JPanel headerTextPanel = new JPanel(new GridLayout(2, 1, 0, 2));
3350:             headerTextPanel.setOpaque(false);
3351:             JLabel heading = new JLabel("User Time-In Sessions & Task Hours Audit");
3352:             heading.setFont(UITheme.FONT_HEADER);
3353:             heading.setForeground(UITheme.TEXT_DARK);
3354:             JLabel sub = new JLabel("Administrative visibility into user attendance, active login
3355:             sub.setFont(UITheme.FONT_SMALL);
3356:             sub.setForeground(UITheme.TEXT_MUTED);
3357:             headerTextPanel.add(heading);
3358:             headerTextPanel.add(sub);
3359:             topBar.add(headerTextPanel, BorderLayout.WEST);
3360:
3361:             JPanel controls = new JPanel(new FlowLayout(FlowLayout.RIGHT, 10, 0));
3362:             controls.setOpaque(false);
3363:
```

File: ProjectManagementSystem.java | Package: (default) (continued)

```
3364:             JLabel lblFilter = new JLabel("Filter by User:");
3365:             lblFilter.setFont(UITheme.FONT_BOLD);
3366:             cmbTimeLogUserFilter = new JComboBox<>();
3367:             cmbTimeLogUserFilter.addItem("All Users");
3368:             for (User u : db.getUsers()) {
3369:                 cmbTimeLogUserFilter.addItem(u.getFullName() + " (" + u.getUsername() + ")");
3370:             }
3371:             cmbTimeLogUserFilter.addActionListener(e -> refreshTimeLogs());
3372:
3373:             JButton btnRefreshLogs = UITheme.createPrimaryButton("Refresh Logs");
3374:             btnRefreshLogs.addActionListener(e -> refreshTimeLogs());
3375:
3376:             JButton btnAuditSummary = UITheme.createSecondaryButton("Audit Summary");
3377:             btnAuditSummary.addActionListener(e -> showAuditSummaryDialog());
3378:
3379:             controls.add(lblFilter);
3380:             controls.add(cmbTimeLogUserFilter);
3381:             controls.add(btnRefreshLogs);
3382:             controls.add(btnAuditSummary);
3383:             topBar.add(controls, BorderLayout.EAST);
3384:
3385:             panel.add(topBar, BorderLayout.NORTH);
3386:
3387:             // Center Content: Top Metrics + 2 Tables (Sessions & Work Logs)
3388:             JPanel centerPanel = new JPanel(new BorderLayout(0, 12));
3389:             centerPanel.setOpaque(false);
3390:
3391:             // 3 KPI Cards
3392:             JPanel metricsGrid = new JPanel(new GridLayout(1, 3, 14, 0));
3393:             metricsGrid.setOpaque(false);
3394:             lblTotalSessionsMetric = new JLabel("0", SwingConstants.CENTER);
3395:             lblActiveSessionsMetric = new JLabel("0", SwingConstants.CENTER);
3396:             lblTotalHoursMetric = new JLabel("0.0h", SwingConstants.CENTER);
3397:
3398:             metricsGrid.add(createStatCard("TOTAL SESSIONS RECORDED", lblTotalSessionsMetric, UITh
3399:             metricsGrid.add(createStatCard("ACTIVE SESSIONS NOW", lblActiveSessionsMetric, UITheme
3400:             metricsGrid.add(createStatCard("TOTAL WORK HOURS LOGGED", lblTotalHoursMetric, UITheme
3401:             centerPanel.add(metricsGrid, BorderLayout.NORTH);
3402:
3403:             // Split Tables: Sessions Table (Top) and Work Logs Table (Bottom)
3404:             JPanel tablesContainer = new JPanel(new GridLayout(2, 1, 0, 12));
3405:             tablesContainer.setOpaque(false);
3406:
3407:             // Section 1: User Login & Time-In Sessions
3408:             JPanel sessionSection = UITheme.createCard();
3409:             sessionSection.setLayout(new BorderLayout(0, 8));
3410:             JLabel lblSessionTitle = new JLabel("User Login & Time-In Sessions (Attendance & Prese
3411:             lblSessionTitle.setFont(UITheme.FONT_SECTION);
3412:             lblSessionTitle.setForeground(UITheme.TEXT_DARK);
3413:             sessionSection.add(lblSessionTitle, BorderLayout.NORTH);
3414:
3415:             Vector<String> sessionCols = new Vector<>(Arrays.asList(
3416:                     "Session ID", "Username", "Full Name", "Role", "Time-In (Login)", "Time-Out (L
3417:             ));
3418:             sessionTableModel = new DefaultTableModel(sessionCols, 0) {
3419:                 @Override public boolean isCellEditable(int r, int c) { return false; }
3420:             };
3421:             sessionTable = new JTable(sessionTableModel);
3422:             sessionTable.setRowHeight(30);
```

File: ProjectManagementSystem.java | Package: (default) (continued)

```
3423:             sessionTable.setFont(UITheme.FONT_REGULAR);
3424:             sessionTable.getTableHeader().setFont(UITheme.FONT_BOLD);
3425:             sessionTable.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);
3426:             sessionSection.add(new JScrollPane(sessionTable), BorderLayout.CENTER);
3427:             tablesContainer.add(sessionSection);
3428:
3429:             // Section 2: Task Work Logs
3430:             JPanel workLogSection = UITheme.createCard();
3431:             workLogSection.setLayout(new BorderLayout(0, 8));
3432:             JLabel lblWorkTitle = new JLabel("User Task Work Hours Logged (Task Activity Audit)");
3433:             lblWorkTitle.setFont(UITheme.FONT_SECTION);
3434:             lblWorkTitle.setForeground(UITheme.TEXT_DARK);
3435:             workLogSection.add(lblWorkTitle, BorderLayout.NORTH);
3436:
3437:             Vector<String> workCols = new Vector<>(Arrays.asList(
3438:                     "Log ID", "Team Member", "Task Title", "Project", "Hours Logged", "Timestamp",
3439:             ));
3440:             workLogTableModel = new DefaultTableModel(workCols, 0) {
3441:                 @Override public boolean isCellEditable(int r, int c) { return false; }
3442:             };
3443:             workLogTable = new JTable(workLogTableModel);
3444:             workLogTable.setRowHeight(30);
3445:             workLogTable.setFont(UITheme.FONT_REGULAR);
3446:             workLogTable.getTableHeader().setFont(UITheme.FONT_BOLD);
3447:             workLogTable.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);
3448:             workLogSection.add(new JScrollPane(workLogTable), BorderLayout.CENTER);
3449:             tablesContainer.add(workLogSection);
3450:
3451:             centerPanel.add(tablesContainer, BorderLayout.CENTER);
3452:             panel.add(centerPanel, BorderLayout.CENTER);
3453:
3454:             return panel;
3455:         }
3456:
3457:         private void refreshTimeLogs() {
3458:             if (sessionTableModel == null || workLogTableModel == null) return;
3459:
3460:             List<TimeInSession> allSessions = db.getSessionLogs();
3461:             List<TaskWorkLog> allWorkLogs = db.getWorkLogs();
3462:
3463:             // Filter by user if selected
3464:             int filterIdx = (cmbTimeLogUserFilter != null) ? cmbTimeLogUserFilter.getSelectedInde
3465:             String filterUsername = null;
3466:             if (filterIdx > 0) {
3467:                 List<User> users = db.getUsers();
3468:                 if (filterIdx - 1 < users.size()) {
3469:                     filterUsername = users.get(filterIdx - 1).getUsername();
3470:                 }
3471:             }
3472:
3473:             DateTimeFormatter fmt = DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss");
3474:
3475:             // Update Metrics
3476:             long totalSessions = allSessions.size();
3477:             long activeSessions = allSessions.stream().filter(s -> "ACTIVE".equals(s.getStatus()))
3478:             double totalHours = allWorkLogs.stream().mapToDouble(TaskWorkLog::getHoursLogged).sum(
3479:
3480:             lblTotalSessionsMetric.setText(String.valueOf(totalSessions));
3481:             lblActiveSessionsMetric.setText(String.valueOf(activeSessions));
```

File: ProjectManagementSystem.java | Package: (default) (continued)

```
3482:             lblTotalHoursMetric.setText(String.format("%.1fh", totalHours));
3483:
3484:             // Populate Sessions Table
3485:             Vector<Vector<Object>> sData = new Vector<>();
3486:             for (TimeInSession s : allSessions) {
3487:                 if (filterUsername != null && !s.getUsername().equalsIgnoreCase(filterUsername)) c
3488:                 Vector<Object> row = new Vector<>();
3489:                 row.add(s.getSessionId());
3490:                 row.add(s.getUsername());
3491:                 row.add(s.getFullName());
3492:                 row.add(s.getRole().getLabel());
3493:                 row.add(s.getTimeIn().format(fmt));
3494:                 row.add(s.getTimeOut() != null ? s.getTimeOut().format(fmt) : "[Active Now]");
3495:                 row.add(s.getFormattedDuration());
3496:                 row.add(s.getStatus());
3497:                 sData.add(row);
3498:             }
3499:             sessionTableModel.setDataVector(sData, new Vector<>(Arrays.asList(
3500:                     "Session ID", "Username", "Full Name", "Role", "Time-In (Login)", "Time-Out (L
3501:             )));
3502:             sessionTable.getColumnModel().getColumn(7).setCellRenderer(new DefaultTableCellRendere
3503:                 @Override public Component getTableCellRendererComponent(JTable table, Object val,
3504:                     JLabel lbl = (JLabel) super.getTableCellRendererComponent(table, val, isSel, h
3505:                     lbl.setHorizontalAlignment(SwingConstants.CENTER);
3506:                     lbl.setFont(UITheme.FONT_BADGE);
3507:                     lbl.setOpaque(true);
3508:                     if ("ACTIVE".equals(val)) {
3509:                         lbl.setBackground(new Color(209, 250, 229));
3510:                         lbl.setForeground(new Color(6, 95, 70));
3511:                     } else {
3512:                         lbl.setBackground(new Color(241, 245, 249));
3513:                         lbl.setForeground(new Color(71, 85, 105));
3514:                     }
3515:                     return lbl;
3516:                 }
3517:             });
3518:
3519:             // Populate Work Logs Table
3520:             Vector<Vector<Object>> wData = new Vector<>();
3521:             for (TaskWorkLog w : allWorkLogs) {
3522:                 if (filterUsername != null) {
3523:                     Optional<User> uOpt = db.getUsers().stream().filter(u -> u.getId().equals(w.ge
3524:                     if (uOpt.isPresent() && !uOpt.get().getUsername().equalsIgnoreCase(filterUserr
3525:                 }
3526:                 Vector<Object> row = new Vector<>();
3527:                 row.add(w.getLogId());
3528:                 row.add(w.getUserName());
3529:                 row.add(w.getTaskTitle());
3530:                 row.add(w.getProjectName());
3531:                 row.add(String.format("%.1f hrs", w.getHoursLogged()));
3532:                 row.add(w.getTimestamp().format(fmt));
3533:                 row.add(w.getNotes());
3534:                 wData.add(row);
3535:             }
3536:             workLogTableModel.setDataVector(wData, new Vector<>(Arrays.asList(
3537:                     "Log ID", "Team Member", "Task Title", "Project", "Hours Logged", "Timestamp",
3538:             )));
3539:         }
3540:
```

File: ProjectManagementSystem.java | Package: (default) (continued)

```
3541:         private void showAuditSummaryDialog() {
3542:             List<TimeInSession> sessions = db.getSessionLogs();
3543:             List<TaskWorkLog> workLogs = db.getWorkLogs();
3544:             long activeCount = sessions.stream().filter(s -> "ACTIVE".equals(s.getStatus())).count
3545:             double totalHours = workLogs.stream().mapToDouble(TaskWorkLog::getHoursLogged).sum();
3546:
3547:             StringBuilder sb = new StringBuilder();
3548:             sb.append("=====================================================\n");
3549:             sb.append("   EXECUTIVE AUDIT & TIME SUMMARY REPORT   \n");
3550:             sb.append("=====================================================\n\n");
3551:             sb.append(String.format("Generated By: %s [%s]\n", currentUser.getFullName(), currentU
3552:             sb.append(String.format("Generated At: %s\n\n", LocalDateTime.now().format(DateTimeFor
3553:             sb.append(String.format("Total Login Sessions Recorded: %d\n", sessions.size()));
3554:             sb.append(String.format("Currently Active User Sessions: %d\n", activeCount));
3555:             sb.append(String.format("Total Task Work Hours Logged:  %.2f hours\n\n", totalHours));
3556:             sb.append("-----------------------------------------------------\n");
3557:             sb.append("User Work Breakdown:\n");
3558:             Map<String, Double> hoursByUser = new HashMap<>();
3559:             for (TaskWorkLog w : workLogs) {
3560:                 hoursByUser.put(w.getUserName(), hoursByUser.getOrDefault(w.getUserName(), 0.0) +
3561:             }
3562:             for (Map.Entry<String, Double> entry : hoursByUser.entrySet()) {
3563:                 sb.append(String.format(" | %-28s : %.1f hrs\n", entry.getKey(), entry.getValue())
3564:             }
3565:             sb.append("=====================================================\n");
3566:
3567:             JTextArea txt = new JTextArea(sb.toString(), 16, 45);
3568:             txt.setFont(new Font("Consolas", Font.PLAIN, 12));
3569:             txt.setEditable(false);
3570:             JOptionPane.showMessageDialog(this, new JScrollPane(txt), "Audit Summary Report", JOpt
3571:         }
3572:
3573:         private void handleLogout() {
3574:             int confirm = JOptionPane.showConfirmDialog(
3575:                     this,
3576:                     String.format("Are you sure you want to log out, %s?\n\nTo switch accounts or
3577:                     "Confirm Logout",
3578:                     JOptionPane.YES_NO_OPTION,
3579:                     JOptionPane.QUESTION_MESSAGE
3580:             );
3581:             if (confirm == JOptionPane.YES_OPTION) {
3582:                 AppLogger.info("User '%s' logged out.", currentUser.getUsername());
3583:                 db.recordLogoutSession(currentUser);
3584:                 dispose();
3585:                 SwingUtilities.invokeLater(() -> ProjectManagementSystem.showLoginAndLaunch(db));
3586:             }
3587:         }
3588:
3589:         // ---------------------------------------------------------------------
3590:         // REFRESH & WBS TREE BUILDER
3591:         // ---------------------------------------------------------------------
3592:         private void refreshAllViews() {
3593:             List<Project> projs = projectService.getAllProjects();
3594:             List<Task> tasks = taskService.getAllTasks();
3595:             List<User> users = db.getUsers();
3596:
3597:             // 1. Dashboard Metrics
3598:             lblTotalProj.setText(String.valueOf(projs.size()));
3599:             lblTotalTasks.setText(String.valueOf(tasks.size()));
```

File: ProjectManagementSystem.java | Package: (default) (continued)

```
3600:             lblCompletedTasks.setText(String.valueOf(tasks.stream().filter(t -> t.getStatus() == T
3601:             lblActiveUsers.setText(String.valueOf(users.size()));
3602:
3603:             long todo = tasks.stream().filter(t -> t.getStatus() == TaskStatus.TODO).count();
3604:             long prog = tasks.stream().filter(t -> t.getStatus() == TaskStatus.IN_PROGRESS).count(
3605:             long rev = tasks.stream().filter(t -> t.getStatus() == TaskStatus.REVIEW).count();
3606:             long done = tasks.stream().filter(t -> t.getStatus() == TaskStatus.COMPLETED).count();
3607:
3608:             lblTodoCount.setText(todo + " Tasks (" + (tasks.isEmpty() ? 0 : (todo * 100 / tasks.si
3609:             lblProgCount.setText(prog + " Tasks (" + (tasks.isEmpty() ? 0 : (prog * 100 / tasks.si
3610:             lblReviewCount.setText(rev + " Tasks (" + (tasks.isEmpty() ? 0 : (rev * 100 / tasks.si
3611:             lblDoneCount.setText(done + " Tasks (" + (tasks.isEmpty() ? 0 : (done * 100 / tasks.si
3612:
3613:             // 2. Dashboard Project Progress Rollup Cards
3614:             progressCardsPanel.removeAll();
3615:             for (Project p : projs) {
3616:                 double pct = projectService.calculateProjectCompletion(p.getId()); // Recursive ro
3617:                 JPanel r = new JPanel(new BorderLayout(14, 0));
3618:                 r.setOpaque(false);
3619:                 r.setBorder(BorderFactory.createEmptyBorder(7, 0, 7, 0));
3620:
3621:                 JLabel l = new JLabel(String.format("<html><b>#%d: %s</b> <font color='#64748B'>[%
3622:                         p.getId(), p.getTitle(), p.getStatus(), p.getBudget()));
3623:                 l.setPreferredSize(new Dimension(320, 24));
3624:                 l.setFont(UITheme.FONT_REGULAR);
3625:
3626:                 JProgressBar bar = new JProgressBar(0, 100);
3627:                 bar.setValue((int) Math.round(pct));
3628:                 bar.setStringPainted(true);
3629:                 bar.setString(String.format("%.1f%%", pct));
3630:                 bar.setPreferredSize(new Dimension(240, 22));
3631:                 bar.setForeground(pct >= 100 ? UITheme.SUCCESS : (pct >= 50 ? UITheme.PRIMARY_ACCE
3632:
3633:                 r.add(l, BorderLayout.WEST);
3634:                 r.add(bar, BorderLayout.CENTER);
3635:                 progressCardsPanel.add(r);
3636:             }
3637:             progressCardsPanel.revalidate();
3638:             progressCardsPanel.repaint();
3639:
3640:             // 3. Project Table
3641:             projectAdapter.setProjects(projs);
3642:
3643:             // 4. Update Filter Combos
3644:             String prevSelected = (String) cmbProjectFilter.getSelectedItem();
3645:             cmbProjectFilter.removeAllItems();
3646:             cmbProjectFilter.addItem("All Projects (" + projs.size() + ")");
3647:             for (Project p : projs) cmbProjectFilter.addItem("#" + p.getId() + " | " + p.getTitle(
3648:             if (prevSelected != null) cmbProjectFilter.setSelectedItem(prevSelected);
3649:
3650:             List<Team> teams = db.getTeams();
3651:             if (cmbTeamFilter != null) {
3652:                 String prevTeam = (String) cmbTeamFilter.getSelectedItem();
3653:                 cmbTeamFilter.removeAllItems();
3654:                 cmbTeamFilter.addItem("All Teams (" + teams.size() + ")");
3655:                 for (Team tm : teams) cmbTeamFilter.addItem(tm.getName());
3656:                 if (prevTeam != null) cmbTeamFilter.setSelectedItem(prevTeam);
3657:             }
3658:
```

File: ProjectManagementSystem.java | Package: (default) (continued)

```
3659:             applyTaskFilter();
3660:
3661:             // 5. Build Perfect WBS Tree
3662:             treeRoot.removeAllChildren();
3663:             for (Project p : projs) {
3664:                 DefaultMutableTreeNode pNode = new DefaultMutableTreeNode(String.format("Project #
3665:                 List<Task> roots = projectService.buildTaskHierarchy(p.getId());
3666:                 for (Task rootTask : roots) {
3667:                     pNode.add(buildTreeNodesRecursively(rootTask));
3668:                 }
3669:                 treeRoot.add(pNode);
3670:             }
3671:             treeModel.reload();
3672:             for (int i = 0; i < hierarchyTree.getRowCount(); i++) hierarchyTree.expandRow(i);
3673:
3674:             // 6. Registered Teams Table & Personnel Roster
3675:             Vector<Vector<Object>> tmData = new Vector<>();
3676:             for (Team tm : teams) {
3677:                 Vector<Object> r = new Vector<>();
3678:                 r.add(tm.getId());
3679:                 r.add(tm.getName());
3680:                 r.add(tm.getDescription());
3681:                 long activeTasks = tasks.stream().filter(t -> tm.getName().equalsIgnoreCase(t.getA
3682:                 r.add(activeTasks + " Task(s)");
3683:                 tmData.add(r);
3684:             }
3685:             teamsTableModel.setDataVector(tmData, new Vector<>(Arrays.asList("Team ID", "Team Name
3686:
3687:             Vector<Vector<Object>> data = new Vector<>();
3688:             for (User u : users) {
3689:                 Vector<Object> row = new Vector<>();
3690:                 row.add(u.getId());
3691:                 row.add(u.getFullName());
3692:                 row.add(u.getUsername());
3693:                 row.add(u.getRole().getLabel());
3694:                 row.add(u.getDepartment());
3695:                 data.add(row);
3696:             }
3697:             teamModel.setDataVector(data, new Vector<>(Arrays.asList("User ID", "Name", "Username"
3698:
3699:             // 7. Time Logs (Admin / Super Admin Only)
3700:             if (currentUser.hasPermission(Permission.VIEW_TIME_LOGS)) {
3701:                 refreshTimeLogs();
3702:             }
3703:         }
3704:
3705:         private void applyTaskFilter() {
3706:             List<Task> all = taskService.getAllTasks();
3707:             int idx = cmbProjectFilter != null ? cmbProjectFilter.getSelectedIndex() : 0;
3708:             if (idx > 0) {
3709:                 List<Project> projs = projectService.getAllProjects();
3710:                 if (idx - 1 < projs.size()) {
3711:                     int pid = projs.get(idx - 1).getId();
3712:                     all.removeIf(t -> t.getProjectId() != pid);
3713:                 }
3714:             }
3715:             if (cmbTeamFilter != null && cmbTeamFilter.getSelectedIndex() > 0) {
3716:                 String teamSel = (String) cmbTeamFilter.getSelectedItem();
3717:                 all.removeIf(t -> !teamSel.equalsIgnoreCase(t.getAssignedTeam()));
```

File: ProjectManagementSystem.java | Package: (default) (continued)

```
3718:             }
3719:             taskAdapter.setTasks(all);
3720:         }
3721:
3722:         private DefaultMutableTreeNode buildTreeNodesRecursively(Task t) {
3723:             String label = String.format("#%d: %s [%s] (%.0f%% | Depth: L%d)",
3724:                     t.getId(), t.getTitle(), t.getStatus().getLabel(), t.calculateCompletionPercer
3725:             DefaultMutableTreeNode node = new DefaultMutableTreeNode(new TaskNodeItem(t, label));
3726:             for (Task child : t.getSubTasks()) {
3727:                 node.add(buildTreeNodesRecursively(child)); // Recursive descent
3728:             }
3729:             return node;
3730:         }
3731:
3732:         private static class TaskNodeItem {
3733:             final Task task;
3734:             final String label;
3735:             TaskNodeItem(Task task, String label) { this.task = task; this.label = label; }
3736:             @Override public String toString() { return label; }
3737:         }
3738:     }
3739:
3740:     // ================================================================================
3741:     // LOGIN DIALOG
3742:     // ================================================================================
3743:
3744:     public static class LoginDialog extends JDialog {
3745:         private final DatabaseManager db;
3746:         private User authenticatedUser;
3747:
3748:         private JComboBox<String> cmbRoleSelect;
3749:         private JTextField txtUser;
3750:         private JPasswordField txtPass;
3751:
3752:         public LoginDialog(Frame parent, DatabaseManager db) {
3753:             super(parent, "Enterprise Project Management - Login Portal", true);
3754:             this.db = db;
3755:             initComponents();
3756:             pack();
3757:             setSize(500, 360);
3758:             setLocationRelativeTo(parent);
3759:             setResizable(false);
3760:         }
3761:
3762:         private void initComponents() {
3763:             setLayout(new BorderLayout());
3764:             getContentPane().setBackground(UITheme.BG_LIGHT);
3765:
3766:             JPanel banner = new JPanel(new BorderLayout());
3767:             banner.setBackground(UITheme.PRIMARY);
3768:             banner.setBorder(BorderFactory.createEmptyBorder(16, 22, 16, 22));
3769:             JLabel t = new JLabel("Enterprise Project Management System");
3770:             t.setFont(UITheme.FONT_HEADER);
3771:             t.setForeground(Color.WHITE);
3772:             JLabel s = new JLabel("Enterprise Project Management Portal");
3773:             s.setFont(UITheme.FONT_SMALL);
3774:             s.setForeground(new Color(224, 231, 255));
3775:             banner.add(t, BorderLayout.NORTH);
3776:             banner.add(s, BorderLayout.SOUTH);
```

File: ProjectManagementSystem.java | Package: (default) (continued)

```
3777:             add(banner, BorderLayout.NORTH);
3778:
3779:             JPanel form = new JPanel(new GridBagLayout());
3780:             form.setBackground(Color.WHITE);
3781:             form.setBorder(BorderFactory.createEmptyBorder(18, 25, 18, 25));
3782:             GridBagConstraints g = new GridBagConstraints();
3783:             g.insets = new Insets(6, 6, 6, 6);
3784:             g.fill = GridBagConstraints.HORIZONTAL;
3785:
3786:             // 1. Account Type Dropdown
3787:             g.gridx = 0; g.gridy = 0;
3788:             JLabel lblRole = new JLabel("Account Type: *");
3789:             lblRole.setFont(UITheme.FONT_BOLD);
3790:             form.add(lblRole, g);
3791:
3792:             g.gridx = 1;
3793:             cmbRoleSelect = new JComboBox<>(new String[]{
3794:                     "[Select Account Type]",
3795:                     "User (Software Developer)",
3796:                     "Admin (Project Manager)",
3797:                     "Super Admin (System Administrator)"
3798:             });
3799:             cmbRoleSelect.setFont(UITheme.FONT_REGULAR);
3800:             form.add(cmbRoleSelect, g);
3801:
3802:             // 2. Clean Instruction (NO USER MODE or credential disclosure)
3803:             g.gridx = 0; g.gridy = 1; g.gridwidth = 2;
3804:             JLabel lblInstr = new JLabel("Please select your account type and enter your credentia
3805:             lblInstr.setFont(UITheme.FONT_SMALL);
3806:             lblInstr.setForeground(UITheme.TEXT_MUTED);
3807:             form.add(lblInstr, g);
3808:             g.gridwidth = 1;
3809:
3810:             // 3. Username Field (Starts empty)
3811:             g.gridx = 0; g.gridy = 2;
3812:             JLabel lblU = new JLabel("Username: *");
3813:             lblU.setFont(UITheme.FONT_BOLD);
3814:             form.add(lblU, g);
3815:
3816:             g.gridx = 1;
3817:             txtUser = new JTextField(18);
3818:             txtUser.setFont(UITheme.FONT_REGULAR);
3819:             form.add(txtUser, g);
3820:
3821:             // 4. Password Field (Starts empty)
3822:             g.gridx = 0; g.gridy = 3;
3823:             JLabel lblP = new JLabel("Password: *");
3824:             lblP.setFont(UITheme.FONT_BOLD);
3825:             form.add(lblP, g);
3826:
3827:             g.gridx = 1;
3828:             txtPass = new JPasswordField(18);
3829:             txtPass.setFont(UITheme.FONT_REGULAR);
3830:             form.add(txtPass, g);
3831:
3832:             cmbRoleSelect.addActionListener(e -> {
3833:                 txtUser.setText("");
3834:                 txtPass.setText("");
3835:                 txtUser.requestFocusInWindow();
```

File: ProjectManagementSystem.java | Package: (default) (continued)

```
3836:             });
3837:
3838:             txtPass.addActionListener(e -> attemptLogin());
3839:
3840:             add(form, BorderLayout.CENTER);
3841:
3842:             JPanel actions = new JPanel(new FlowLayout(FlowLayout.RIGHT, 12, 12));
3843:             actions.setBackground(UITheme.BG_LIGHT);
3844:
3845:             JButton btnExit = UITheme.createSecondaryButton("Exit");
3846:             btnExit.addActionListener(e -> {
3847:                 authenticatedUser = null;
3848:                 dispose();
3849:             });
3850:
3851:             JButton btnLogin = UITheme.createPrimaryButton("Sign In");
3852:             btnLogin.addActionListener(e -> attemptLogin());
3853:
3854:             actions.add(btnExit);
3855:             actions.add(btnLogin);
3856:             add(actions, BorderLayout.SOUTH);
3857:         }
3858:
3859:         private void attemptLogin() {
3860:             int selectedIdx = cmbRoleSelect.getSelectedIndex();
3861:             if (selectedIdx <= 0) {
3862:                 JOptionPane.showMessageDialog(this,
3863:                         "Please select your Account Type before signing in.",
3864:                         "Account Type Required", JOptionPane.WARNING_MESSAGE);
3865:                 return;
3866:             }
3867:
3868:             String u = txtUser.getText().trim();
3869:             String p = new String(txtPass.getPassword());
3870:
3871:             if (u.isEmpty() || p.isEmpty()) {
3872:                 JOptionPane.showMessageDialog(this,
3873:                         "Please enter both username and password.",
3874:                         "Input Required", JOptionPane.WARNING_MESSAGE);
3875:                 return;
3876:             }
3877:
3878:             Optional<User> auth = db.authenticate(u, p);
3879:             if (!auth.isPresent()) {
3880:                 JOptionPane.showMessageDialog(this,
3881:                         "Authentication Failed: Invalid username or password.",
3882:                         "Authentication Error", JOptionPane.ERROR_MESSAGE);
3883:                 return;
3884:             }
3885:
3886:             User user = auth.get();
3887:             boolean roleMatches = false;
3888:             String expectedType = "";
3889:
3890:             if (selectedIdx == 1) { // User (Developer)
3891:                 expectedType = "User (Software Developer)";
3892:                 roleMatches = (user.getRole() == UserRole.DEVELOPER);
3893:             } else if (selectedIdx == 2) { // Admin (Manager)
3894:                 expectedType = "Admin (Project Manager)";
```

File: ProjectManagementSystem.java | Package: (default) (continued)

```
3895:                 roleMatches = (user.getRole() == UserRole.MANAGER);
3896:             } else if (selectedIdx == 3) { // Super Admin
3897:                 expectedType = "Super Admin (System Administrator)";
3898:                 roleMatches = (user.getRole() == UserRole.SUPER_ADMIN);
3899:             }
3900:
3901:             if (!roleMatches) {
3902:                 JOptionPane.showMessageDialog(this,
3903:                         String.format("Authentication Denied:\nAccount '%s' does not match the sel
3904:                                 u, expectedType),
3905:                         "Account Type Mismatch", JOptionPane.WARNING_MESSAGE);
3906:                 return;
3907:             }
3908:
3909:             this.authenticatedUser = user;
3910:             dispose();
3911:         }
3912:
3913:         public User getAuthenticatedUser() { return authenticatedUser; }
3914:     }
3915:
3916:     // ================================================================================
3917:     // APPLICATION ENTRY POINT: MAIN METHOD & SESSION LAUNCHER
3918:     // ================================================================================
3919:
3920:     public static void showLoginAndLaunch(DatabaseManager db) {
3921:         LoginDialog login = new LoginDialog(null, db);
3922:         login.setVisible(true);
3923:
3924:         User user = login.getAuthenticatedUser();
3925:         if (user != null) {
3926:             AppLogger.info("Authenticated successfully as: %s [%s]", user.getFullName(), user.getR
3927:             db.recordLoginSession(user);
3928:             MainFrame frame = new MainFrame(db, user);
3929:             frame.setVisible(true);
3930:         } else {
3931:             System.exit(0);
3932:         }
3933:     }
3934:
3935:     public static void main(String[] args) {
3936:         System.out.println("============================================================================
3937:         System.out.println("  Enterprise Project Management System (Production Standard)       ");
3938:         System.out.println("============================================================================
3939:
3940:         try {
3941:             UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName());
3942:         } catch (Exception ignored) {}
3943:
3944:         DatabaseManager db = DatabaseManager.getInstance();
3945:
3946:         SwingUtilities.invokeLater(() -> showLoginAndLaunch(db));
3947:     }
3948: }
```

Chapter 8
CONCLUSION

The **Project Management System (PMS)** successfully demonstrates the practical, industry-aligned application of Object-Oriented Programming (OOP) concepts using the Java programming language. By combining Java Swing for an interactive, responsive desktop user interface with an embedded SQLite database through JDBC, the system provides a production-grade, self-contained solution for managing complex engineering projects, deliverables, and workforce resources.

The application drastically eliminates operational bottlenecks associated with manual spreadsheets by automating project scheduling, financial budget tracking, milestone decomposition, and developer attendance auditing. Granular Role-Based Access Control (RBAC) guarantees organizational security, ensuring that Super Administrators retain root governance, Project Managers direct project execution and team squad allocations, and Software Developers execute work items without unauthorized administrative access.

Every core topic outlined in the **KTU PBCST304 Object-Oriented Programming syllabus** has been rigorously implemented: from fundamental wrapper classes, control structures, and varargs loggers (Module 1); to deep inheritance hierarchies, constructor chaining, recursion, and dynamic method dispatch (Module 2); through custom exception hierarchies, SOLID principles, the Singleton pattern, and the Adapter pattern (Module 3); culminating in an event-driven Swing user interface and transactional JDBC persistence (Module 4).

The modular architecture ensures that the system is easily maintainable and extensible. Future enhancements may incorporate networked multi-client synchronization, Gantt chart scheduling visualizations, and automated email notifications. Overall, this microproject bridges the gap between academic object-oriented theory and commercial software engineering practice, delivering a robust, secure, and production-ready enterprise application.